



TRAINING & CONSULTING

Spring Boot for BNP

ABIS Training & Consulting
www.abis.be
training@abis.be

© ABIS 2026

Document number: 1618_03b.fm
19 August 2026

Address comments concerning the contents of this publication to:
ABIS Training & Consulting, Diestsevest 32 / 4b, B-3000 Leuven, Belgium
Tel.: (+32)-16-245610

© Copyright ABIS N.V.

TABLE OF CONTENTS

PREFACE	IX
INTRODUCTION TO SPRING	11
1 ____ <i>Some history</i> _____	12
2 ____ <i>Design goals</i> _____	14
3 ____ <i>The Spring Framework Landscape</i> _____	15
4 ____ <i>Spring and/vs Spring Boot</i> _____	19
4.1 Spring Starter Dependencies	22
4.2 Auto Configuration	23
GETTING STARTED WITH SPRING BOOT	25
1 ____ <i>Philosophy</i> _____	26
2 ____ <i>Ingredients</i> _____	27
3 ____ <i>Tools</i> _____	28
4 ____ <i>Hello World Example</i> _____	29
4.1 Maven Configuration - pom.xml	30
4.2 Application Class	32
4.3 Model	33
4.4 Service Layer	34
4.5 Test	35

TESTING IN SPRING	37
1 ____ <i>Testing principles</i> _____	38
2 ____ <i>Introduction to Spring Testing</i> _____	40
3 ____ <i>JUnit</i> _____	41
3.1 JUnit overview	43
3.2 JUnit 5 implementation	46
3.3 JUnit Assert	47
3.4 JUnit test class	49
3.5 JUnit extensions	51
4 ____ <i>Mockito</i> _____	53
5 ____ <i>Spring Integration Testing</i> _____	55
5.1 Spring Integration Testing Goals	56
5.2 Spring TestContext Framework	57
5.3 Test Slices in Spring Boot	62
5.4 Spring MVC Test Framework	63

SPRING CORE	65
1 ____ <i>The Container</i> _____	66
2 ____ <i>Bean Lifecycle</i> _____	67
3 ____ <i>Declaring and Wiring Beans</i> _____	69
3.1 Java Based Configuration	70
3.2 Using Annotations for Autodiscovering and Wiring Beans	72
3.3 Stereotypes	73
3.4 Meta-Annotations	74
3.5 Wiring Beans with Annotations	75
3.6 Explicit definition and wiring of beans in the Java Config class	82
3.7 Using multiple Configuration files	86

4	<i>Addressing Ambiguity</i>	87
5	<i>Profiles</i>	91
6	<i>Runtime Value Injection</i>	95
6.1	Injecting external values	96

ASPECT-ORIENTED SPRING 101

1	<i>What is AOP</i>	102
1.1	General	102
1.2	Terminology	103
1.3	Spring's AOP support	104
2	<i>Writing pointcuts</i>	105
3	<i>Defining Aspects</i>	107
4	<i>Annotating Advices</i>	108
4.1	Before and After(Returning/Throwing) Advice	109
4.2	Around Advice	110
4.3	Example	111

INTRODUCTION TO REST AND JSON 117

1	<i>Application Development and Distributed Computing</i>	118
1.1	Structuring an Application: Layers	118
1.2	Design Goals	119
1.3	Distributed Computing	120
1.4	Layers vs. Tiers	121
1.5	Communicating Objects & Data Types	122
2	<i>RESTful Web Services / APIs</i>	124
2.1	What is REST?	125
2.2	The REST Architectural Style	127
2.3	Resources	128
2.4	HTTP Methods	129
2.5	The Representation of Resources	130
2.6	REST Resource Naming Best Practices	132
3	<i>XML and/vs. JSON</i>	143

3.1	XML	143
3.2	JSON	145

SPRING REST BASICS 149

1	<i>Spring REST basics</i>	150
1.1	Configuration	151
1.2	Writing a Controller	153
1.3	Hello World REST API	154
2	<i>Testing REST APIs</i>	155
2.1	SoapUI	155
2.2	Postman	156

MORE REST API FUNCTIONALITY 157

1	<i>Defining RESTful Resources</i>	158
2	<i>Mapping the HTTP Request Methods</i>	160
2.1	@GetMapping	161
2.2	@PostMapping	162
2.3	@PutMapping	163
2.4	@DeleteMapping	164
3	<i>Accessing Request Data</i>	165
3.1	@PathVariable	166
3.2	@RequestParam	167
3.3	@RequestBody	169
4	<i>Returning Responses</i>	170
4.1	@ResponseBody	170
5	<i>More JSON Mapping</i>	171
5.1	Jackson Annotations	172
5.2	Adding a Wrapper Element around a List of Objects	174
5.3	Jackson ObjectMapper	176
6	<i>Producing and Consuming XML</i>	179
6.1	Using JAXB for mapping XML	180

CREATING REST CLIENTS 187

1	REST Client Project Settings	188
2	RestTemplate	189
3	Getting Data	191
3.1	Basic retrieval of data with getForObject() method	191
3.2	More Possibilities with the exchange() Method	193
4	Posting Data	195
5	Updating Data	197
6	Deleting Data	199

EXCEPTION HANDLING AND VALIDATION 201

1	HTTP Status Codes	202
1.1	HTTP Status Codes - 2xx Success	203
1.2	HTTP Status Codes - 4xx: Client Errors	206
1.3	HTTP Status Codes - 5xx: Server Errors	212
2	The ResponseEntity class	214
3	Exception Handling	217
3.1	Creating an Error Object	218
3.2	Exception Handling with ResponseEntity	220
3.3	Centralizing Exception Handling	224
4	Validation	226

ADDITIONAL TOPICS 233

1	Swagger for documenting REST APIs	234
2	Circuit Breakers	235

INTRODUCTION TO ORM AND (SPRING) JPA 237

1	Persisting objects	238
2	Object - relational mapping (ORM)	240
3	Features of persistence mechanisms	243
4	(Spring) JPA	245
4.1	JPA Architecture	246
4.2	Spring's JPA integration	248

CONFIGURATION 249

1	Spring JPA: Basic Configuration	250
2	Configuring a data source	251
2.1	General	251
2.2	Maven dependencies	252
3	Data Source Types	253
3.1	Embedded Data Source	253
3.2	Basic Data Source	254
3.3	Pooled Data Source	256
3.4	JNDI Data Source	258
4	Simplifications from Spring Boot	259
5	Extra configuration options	260
6	Testing The Connection	262

BASIC OR MAPPING AND JPA REPOSITORIES 263

1	Mapping concepts	264
2	Class to table mapping	265
3	Object identity	266
3.1	Object identity - mapping	267

3.2	Composite primary key	269
4	<i>Property mapping</i>	274
4.1	Type conversion - built-in Hibernate types	276
4.2	Property mapping - derived properties	278
5	<i>JPA repositories</i>	279
5.1	General	279
5.2	Repository Interfaces	280
5.3	Query Methods	282
6	<i>Spring Data JUnit Testing</i>	284

MAPPING ASSOCIATIONS 285

1	<i>Value types</i>	286
2	<i>One class for two tables</i>	288
3	<i>Association mapping</i>	289
3.1	One-to-one mapping	290
3.2	Many-to-one mapping	295
3.3	One-to-many mapping	297
3.4	Many-to-many associations	301
4	<i>Additional options</i>	304
4.1	Cascade strategy	304
4.2	Fetching strategy	305

QUERYING IN SPRING JPA 307

1	<i>@Query</i>	308
2	<i>JPQL</i>	309
2.1	Basic JPQL	310
2.2	Selection: From	311
2.3	Restriction: Where	312
2.4	Joins in JPQL	314
2.5	Parameter binding in JPQL	315
3	<i>Named queries</i>	316

4	<i>Native queries</i>	317
5	<i>Changing the “select” object</i>	318
5.1	Aggregate functions	319
5.2	Limiting/Changing selected columns	320
6	<i>Using DTOs and mappers</i>	321
6.1	Data Transfer Objects (DTOs)	321
6.2	Custom Mapper Class	322
6.3	@NamedNativeQuery and @SqlResultSetMapping	324
6.4	Mapping JPQL query results	325

TRANSACTION MANAGEMENT AND EXCEPTION HANDLING 327

1	<i>Exception Handling</i>	328
2	<i>Modifying queries</i>	329
3	<i>Understanding transactions</i>	330
3.1	Spring's Transaction Philosophy	331
3.2	Transaction Managers	332
4	<i>Transaction Status</i>	333
5	<i>Transaction Definition</i>	334
6	<i>Declarative transactions</i>	337
7	<i>Programming transactions</i>	340

SECURING REST APIS 343

1	<i>Securing APIs</i>	344
2	<i>API Keys</i>	346
2.1	API keys in the API	347
2.2	API Keys in the Client	349
3	<i>Introduction to Spring Security</i>	350

4	<i>HTTP Basic and Digest Authentication</i>	351
4.1	Configuring Minimal Security	354
4.2	Creating a User Store	355
4.3	Intercepting Requests	357
4.4	HTTP Basic Authentication in the Client	359
5	<i>Authorizing Access</i>	363
5.1	Securing Methods with Spring	364
5.2	Client Side Authorization	365
6	<i>Introduction to OAuth 2.0 and OpenID Connect</i>	366
6.1	OAuth 2.0	366
6.2	OpenID Connect (OIDC)	368
6.3	Integration in Spring	370
7	<i>HTTPS - Trust stores and Key Stores</i>	373
8	<i>CORS</i>	374

APPENDIX A.EXERCISES 375

1	<i>Spring Getting Started</i>	375
2	<i>Spring Boot Testing</i>	375
3	<i>Spring Core</i>	375
4	<i>AOP</i>	376
5	<i>Spring REST</i>	376
6	<i>Creating REST Clients</i>	377
7	<i>Exception Handling and Validation</i>	377
8	<i>Additional Topics</i>	377
9	<i>Database Configuration</i>	377
10	<i>Entity mappings and Basic JPA repositories</i>	378

11	<i>Association Mapping and @Query</i>	378
12	<i>More querying techniques and performance</i>	378
13	<i>Exception Handling and Transactions</i>	379
14	<i>Spring Security</i>	379

PREFACE

This course text is used as a practical workbook for mastering the Spring framework, covering aspects on application architecture, front-end and back-end development. This includes the creation of APIs and the integration with a database using JPA.

A good practical experience with Java SE is a must, to make full profit of this mastering course.

More information can be found in:

- **The other Abis Courses**
- <https://docs.spring.io/spring-framework/reference/index.html>
- <https://docs.spring.io/spring-boot/index.html>
- <https://www.baeldung.com>
- <https://chatgpt.com>

Introduction to Spring

Objectives :

- **Goals of the Spring Framework**
- **The Spring Framework Landscape**
- **Spring and/vs Spring Boot**

Some history

1

- **Rod Johnson (and Juergen Hoeller)**
book: Expert One-on-One J2EE Design and Development in October 2002
- **reaction against complexity of EJBs at that time**
- **originally called Interface21 Framework**
Yann Caroff: “Spring” - association with nature
+ fresh start after winter
- **first released under the Apache 2.0 license in June 2003**

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

Versions

- **1.0: March 2004**
- **2.0: October 2006; 2.5: November 2007**
- **3.0: December 2009; 3.1, 3.2: December 2012**
- **4.0: December 2013; 4.3: June 2016**
- **5.0: September 2017; 5.3.32: February 2024**
- **6.0: November 2022; 6.2.16: February 2026**
- **7.0: November 2025; 7.0.5: February 2026**

<https://github.com/spring-projects/spring-framework/releases>

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

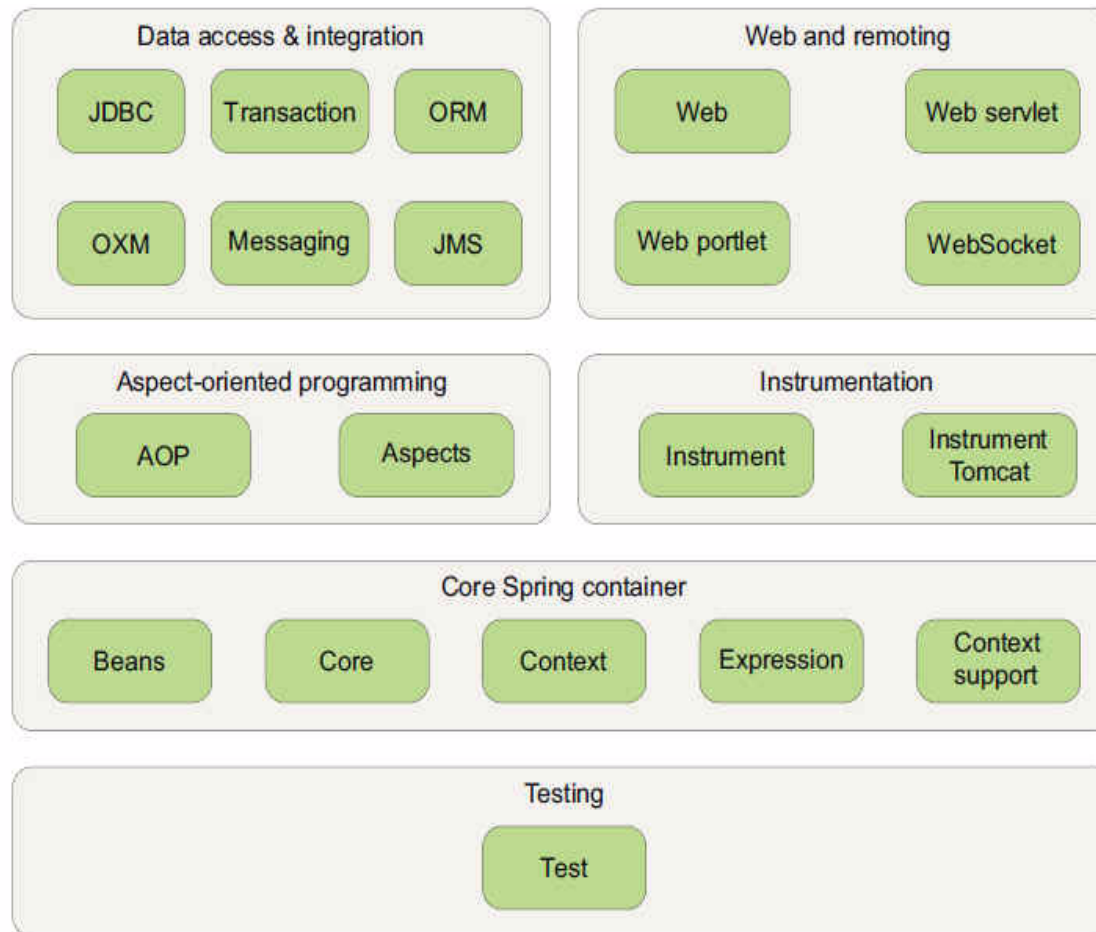
- **MISSION:** simplifying Java development of enterprise applications from front to back
- **FOUR KEY STRATEGIES:**
 - *Lightweight and minimally invasive development with POJOs*
Spring avoids littering application code with its API
 - *Loose coupling through dependency injection (inversion of control) and interface orientation*
Code becomes easier to understand, to test and to reuse. Whack-a-mole bug behaviour is avoided
 - *Declarative programming through aspects*
Separate out cross-cutting concerns like logging, transaction management and security
 - *Boilerplate reduction through aspects and templates*
Spring encapsulates commonly repeated code (DRY principle), so you can focus on the real business logic

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

The Spring Framework Landscape

3

- **core framework is composed of several distinct modules (20 Jar files), grouped in 6 categories of functionality**



- **free to choose the modules that suit your application**

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

Spring Core Modules

- ***Core Spring Container***
create, configure and manage beans
- ***Spring's AOP module***
decoupling application-wide concerns (transactions, security,...)
- ***Data Access and Integration***
abstract away boilerplate code, provide hooks into popular ORM frameworks, JMS support, object-to-XML mapping
- ***Web and Remoting***
MVC framework (servlet-based and portlet-based), support for RMI, Hessian and Burlap, JAX-WS, building REST APIs
- ***Testing***
mock implementations for unit tests, support for integration testing

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

The Spring portfolio

- **build upon the core framework and upon each other**
- **covers almost every facet of Java development**
- **more info can be found on: <http://spring.io/projects>**
- ***Spring Boot***
 - **use automatic configuration techniques that can eliminate most (and in many cases, all) Spring configuration**
 - **helps to reduce the size of your Spring project build files**
- ***Spring Web Services***
 - **offers contract-first model for building Web Services**
- ***Spring HATEOAS***
 - **simplify creating REST applications that follow HATEOAS rules**
- ***Spring REST docs***
- ***Spring Security***
 - **declarative security mechanism based on AOP**

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and vs Spring Boot

The Spring portfolio (cont.)

- ***Spring Web Flow***
 - building conversational, flow-based web applications
- ***Spring Integration***
 - interaction with other enterprise applications
- ***Spring Batch***
 - to perform bulk operations on data
- ***Spring Data***
 - more simplification of JPA
 - connecting to NoSQL databases
- ***Spring Cloud***
 - provides tools for developers to quickly build some of the common patterns in distributed systems; useful for building and deploying microservices
- ***Spring LDAP***
 - template-based approach to simplify applications using LDAP

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

- **Spring Boot = extension of the Spring framework which eliminates the boilerplate configurations required for setting up a Spring application**
- **based on convention-over-configuration principle**
- **advantages**
 - **easy to understand and develop spring applications**
 - **increases productivity**
 - **reduces development time**
- **no more XML configuration needed**

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

Spring Boot Versions

- 1.0 released in 2013 (Spring 4.0)
- 2.0 released in 2018 (Spring 5.0)
- 3.0 released in November 2022 (Spring 6.0, needs jdk17)
 - latest version: 3.5.11 (February 2026)
- 4.0 released in November 2025 (Spring 7.0)
 - latest version: 4.0.3 (February 2026)
 - remodularization of the Spring Boot codebase
 - > some maven dependencies changed!

<https://github.com/spring-projects/spring-boot/releases>

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and vs Spring Boot

Spring and/vs Spring Boot (cont.)

- **four main features:**
 - **Spring Boot Starters**
aggregate common groupings of dependencies
 - **Auto Configuration**
automatically configures beans needed by your application
 - **Command-line Interface (CLI)**
simplifies even further, based on groovy
 - **Spring Boot Actuator**
adds management features
- **integrated in STS/IntelliJ (Spring Initializr)**
- **running as Spring Boot App**
will bootstrap the embedded Tomcat server,
deploy your app and map the URLs

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

- **starters aggregate commonly needed dependencies**
 - **based on the idea of transitive dependencies**
 - **takes care of version compatibility**
 - **more than 30 starters available**
 - **for Spring Core, but also the portfolio projects**
 - **examples**
 - **spring-boot-starter-web**
 - **spring-boot-starter-aop**
 - **spring-boot-starter-jdbc**
 - **spring-boot-starter-data-jpa**
 - **spring-boot-starter-ws**
 - **spring-boot-starter-rest (integrated in spring-boot-starter-web)**
1. Some history
 2. Design goals
 3. The Spring Framework Landscape
 4. Spring and/vs Spring Boot

- **Spring Boot starters trigger auto configuration**
- **while adding dependencies on classpath (pom.xml), Spring makes assumptions about what configuration you will need**
- **cuts down the amount of configuration**
- **example:**
when adding spring-boot-starter-web, ViewResolver, resource handlers and message converters automatically added
- **watch out for “side effects”!**

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and vs Spring Boot

Introduction to Spring

1. Some history
2. Design goals
3. The Spring Framework Landscape
4. Spring and/vs Spring Boot

Getting Started with Spring Boot

Objectives :

- **Setting up a Spring Boot application**
- **Hello world**

- **design:**

- **Lasagna / ravioli code**
- **Loose coupling**
- **Coding to interfaces**
- **Dependency Injection - Inversion of control (IoC)**

objects do not create other objects on which they rely to do their work. Instead, they get the objects that they need from an outside source

- **DRY: avoid boilerplate code**

-> templates, configuration taken out of the code, aop

- **easy to test**

-> due to design + extra features on top of JUnit

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

- **Java Beans**
 - reusable software components
 - classes conforming to a particular convention: default constructor, private variables with getters and setters, serializable
- **Services - Repository/DAO**
 - Service Layer decoupled from model
 - Repository: CRUD methods
- **Configuration**
 - XML (deprecated)
 - annotations
 - Java based
 - application.properties
- **Container**
 - ApplicationContext

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

- **IDE**
 - IntelliJ
 - Eclipse + STS plugin
 - Spring Tool Suite (STS)
- **Maven**
 - build automation and software comprehension tool
 - uses Project Object Model (POM) to describe the software project being built, its dependencies on other external modules and components, and the build order
 - dynamically downloads Java libraries from one or more repositories <https://central.sonatype.com>
- **Server**
 - only application server needed
 - embedded Tomcat Server provided

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

Recipe

- create a new Spring Initializr project/module
- make sure that the Spring Boot version in the pom.xml file is the correct one and add the needed dependencies
- Spring will add SpringHelloApplication and SpringHelloApplicationTests classes under the “base” package of the application
- add your own model and service classes under subpackages of the “base” package, only there Spring Boot will look for “registered” classes
- the Service layer should have an Interface + Implementation class(es), the class is “registered” with Spring via the @Component annotation
- use the SpringHelloApplicationTests class to write your Junit 5 test cases. “Wire” in the service INTERFACE in the test class (via @Autowired), and use that in your test. Spring Boot will “automatically” find the correct implementation class to use

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.7</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>

  <modelVersion>4.0.0</modelVersion>
  <groupId>be.abis</groupId>
  <artifactId>hellospring</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>
  <name>hellospring</name>

  <properties>
    <java.version>21</java.version>
  </properties>

  <!-- continued on next page-->
```

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

Maven Configuration - pom.xml (cont.)

```
<!-- continuation -->
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

Getting Started with Spring Boot

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

Application Class

4.2

```
package be.abis.hellospring;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HellospringApplication {
    public static void main(String[] args) {
        SpringApplication.run(HellospringApplication.class, args);
    }
}
```

**This class will “start up” the application
+ will “register”, instantiate and “wire” all necessary classes
which it will look for in subpackages of the package
where this class is in.**

Getting Started with Spring Boot

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example


```
package be.abis.hellospring.model;

public class Person {

    private String firstName;

    public Person(String firstName) {
        this.firstName = firstName;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
}
```

Getting Started with Spring Boot

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

Interface

```
package be.abis.hellospring.service;
import be.abis.hellospring.model.Person;
public interface HelloService {
    Person findPerson(int id);
    void sayHelloTo(Person person);
}
```

Implementation

```
package be.abis.hellospring.service;
import be.abis.hellospring.model.Person;

@Component
public class AbisHelloService implements HelloService {
    public Person findPerson(int id) {
        return new Person("John");
    }
    public void sayHelloTo(Person person) {
        System.out.println("Welcome to Spring, " + person.getFirstName());
    }
}
```

```
import be.abis.hellospring.model.Person;
import be.abis.hellospring.service.HelloService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import static org.junit.jupiter.api.Assertions.assertEquals

@SpringBootTest
public class HellospringApplicationTests {

    @Autowired
    private HelloService helloService;
    private Person person;

    @Test
    void contextLoads(){}    //will check whether all beans are registered correctly

    @BeforeEach
    void init(){ person = new Person("John");}

    @Test
    void PersonNamelsJohn(){
        assertEquals("John", helloService.findPerson(1).getFirstName());
    }

    @Test
    void sayHelloToJohn(){
        helloService.sayHelloTo(person);
    }
}
```

Getting Started with Spring Boot

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

Getting Started with Spring Boot

1. Philosophy
2. Ingredients
3. Tools
4. Hello World Example

Testing in Spring

Objectives :

- **Testing principles**
- **Introduction to Spring Testing**
- **JUnit**
- **Mockito**
- **Spring Integration Testing**

types of tests:

- **code testing**
 - **unit test**
 - **integration test**
- **regression testing**
- **performance testing**
- **defect tracking**
- ...

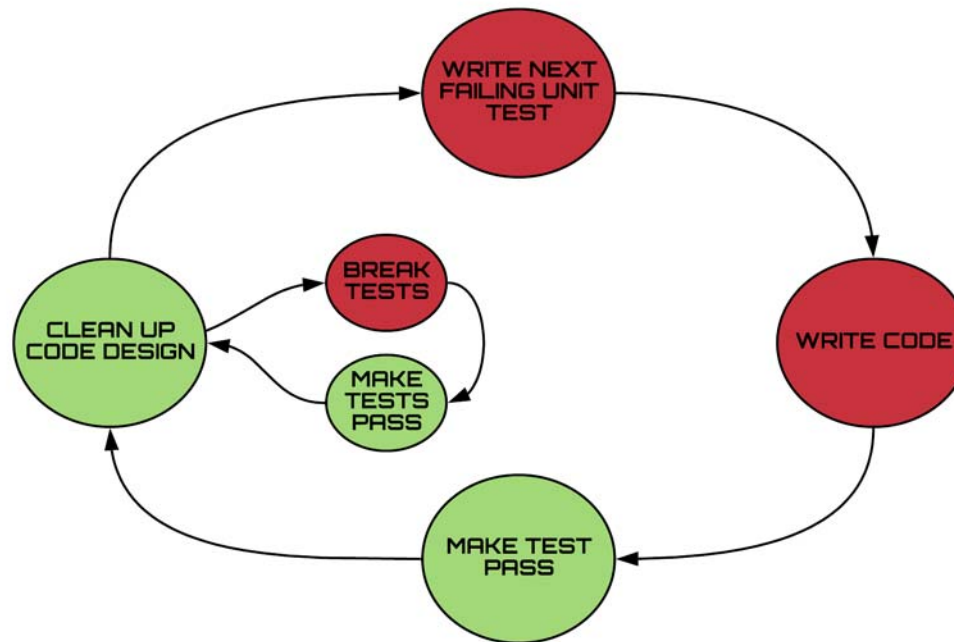
Testing based on test plans/scenarios, derived from use cases

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Test Driven Design (TDD)

Red-Green-Refactor



Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- **provided via spring-boot-starter-test dependency**
 - **contains JUnit Jupiter, AssertJ, Hamcrest, and a number of other useful libraries**

<https://docs.spring.io/spring-boot/3.5/reference/testing/test-scope-dependencies.html>

- **@SpringBootTest**
 - **starts the entire Spring application**
 - **creates the application context**
 - **wires all beans**
 - **loads configuration**
- **Spring adds test features on top of JUnit and Mockito**
- **use Spring Test + Spring Boot Test to:**
 - **test HTTP endpoints**
 - **test database calls**
 - **isolate application layers**

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

web site: www.junit.org

open source testing framework used to develop and execute unit tests in Java

What are unit tests?

- low-level
- investigate the behaviour of a single component (=unit) within a class, Repository, Controller,...
- based on component specification
- written before the component is developed (test-driven development)

Why use unit testing?

- improvement in productivity and overall code quality

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Why use a unit testing framework?

Advantages

- easier to write tests
 - easier to run tests
 - easier to rerun tests after change
- +
- consistency, maintenance, ramp-up time, automation

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Instantiate object -> invoke method -> verify assertions

JUnit 4 (2006): refactored to take advantage of Java SE 5 features

- **annotations** (no inheritance, no naming conventions)

- increase of flexibility
- more lightweight

- **new functionality**

- parameterized tests
- simplified exception testing
- timeout tests
- flexible fixtures
- easy way to ignore tests
- new way to logically group tests

**-> packages `org.junit.*`
(and `junit.framework.*` for compatibility with JUnit 3)**

JUnit 5

- **released September 2017, requires Java 8 or higher**
- **JUnit 5 is composed of several different modules from three different sub-projects**
- **JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage**
 - **JUnit Platform**
 - serves as a foundation for launching testing frameworks on the JVM
 - defines the TestEngine API for developing a testing framework that runs on the platform
 - provides a Console Launcher to launch the platform from the command line and build plugins for Gradle and Maven as well as a JUnit 4 based Runner for running any TestEngine on the platform
 - **JUnit Jupiter**
 - combination of the new programming model and extension model for writing tests and extensions
 - provides a TestEngine for running Jupiter based tests
 - **JUnit Vintage**
 - provides a TestEngine for running JUnit 3 and JUnit 4 based tests

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

JUnit 5 (..)

- **new in JUnit 5**
 - **lambda support**
 - **test interfaces with default methods**
 - **nested unit tests**
 - **conditional test execution**
 - **parameterized tests**
 - **possibility to write custom extensions**
 - **repeated tests**
 - **dynamic tests**
- **watch out: some names of annotations have changed!**

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

junit-jupiter-*.jar in classpath

Most important classes of the `org.junit.jupiter.api.*` framework

- **Assert** with several static test methods
- Write test methods in test classes with annotations

Execution of the test cases

- with the command line
 - `org.junit.runner.JUnitCore`
- or via IDE tooling (Eclipse, IntelliJ, Netbeans,...)

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- The Assert class contains only static methods to be invoked in the test methods (use static import)
- General principle:
 - AAA: Arrange - Act - Assert
 - if the test fails ---> report the failure

Example

```
import static org.junit.jupiter.api.Assert.*;

Person p = new Person("John","Travolta"); //arrange
String firstName = p.getFirstName();      //act
assertEquals(firstName,"Johan");          //assert
```

Two types of reports:

- Failures: failures of anticipated test conditions
org.opentest4j.AssertionFailedError: expected:<Joh[a]n> but was:<Joh[]n>
- Errors: unexpected error or exceptions

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

JUnit Assert (..)

Different tests

Type of test	Normal	Negated
Condition returns true	<code>assertTrue(boolean)</code>	<code>assertFalse(boolean)</code>
Object does not exist	<code>assertNull(Object)</code>	<code>assertNotNull(Object)</code>
Both objects refer to the same instance	<code>assertSame(Object, Object)</code>	<code>assertNotSame(Object, Object)</code>
Both objects are equal	<code>assertEquals(Object, Object)</code>	<code>assertNotEquals(Object, Object)</code>
Exception is thrown	<code>assertThrows()</code>	<code>assertDoesNotThrow()</code>
Fail unconditionally	<code>fail()</code>	-

Note:

- **`assertEquals` is overloaded to compare**

Objects, booleans, longs, doubles,....

<https://junit.org/junit5/docs/5.8.2/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Create 1 test class for each class to be tested with

- annotation **@Test** before test method
- name test methods after the goal to be reached
e.g. `nameOfPerson1IsJohn()`
- initialize and finalize the fixture (test context)
 - defined via annotations **@BeforeEach** and **@AfterEach**
-> `setUp()` / `tearDown()` methods
 - called before and after each test

to make sure there are no side effects between test runs

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Example

```
import static org.junit.jupiter.api.Assert.*;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import be.abis.demo.Person;
```

```
public class PersonTest {
    Person p;
    @BeforeEach
    public void setUp() throws Exception {
        p = new Person("John", "Travolta");
    }
    @AfterEach
    public void tearDown() throws Exception {
        p = null;
    }
    @Test
    public void testGetFirstName() {
        assertThrows(NameTooLongException.class, () -> p.getFirstName());
    }
}
```

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Annotations

- **@Timeout** : to fail after a certain timeout
- **@Disabled** : to skip a test method or a complete test class
- **@TestMethodOrder + @Order** : to order the execution of tests (disadvise!)
- **@Tag** : to group test together for filtering
- **@DisplayName** : for changing the display name of a test (method)

<https://junit.org/junit5/docs/current/user-guide/>

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Example

```
import static org.junit.jupiter.api.Assert.*;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;
...
@TestMethodOrder(OrderAnnotation.class)
public class CourseTest {
    @Test
    @Order(2)
    public void courseName() {
        Course c1 = new Course("Eclipse");
        assertEquals("Oclipse", c1.getCourseName());
    }
    @Test
    @Tag("exceptions")
    @Order(1)
    public void exceptionExpected() {
        Course course = new Course(null);
        course.getCourseName().length();
        fail();
    }
    @Timeout(100)
    public void timed() {...}
    @Disabled("database not yet available")
    public void ignoreThis() {...}
}
```

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- **JUnit test classes should be independent of the execution of methods in other classes**
- **use Mockito framework to achieve this**
 - **open source testing framework released under the MIT License**
 - **jars:**
 - mockito-core.jar
 - mockito-junit-jupiter-api.jar
 - + other dependent libraries
- **allows creation of test double objects (mock objects) in automated unit tests**
- **mock object = fake implementation class of interface**

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Mockito - Example

```
@SpringBootTest
class HelloServiceTests {

    @Autowired
    private HelloService helloService;

    @MockitoBean
    private Person mockPerson;

    @Test
    void sayHelloToJohn() throws NotBigEnoughException {
        when(mockPerson.calculateHeightInInches()).thenReturn(70.0);
        when(mockPerson.getFirstName()).thenReturn("John");
        helloService.sayHelloTo(mockPerson);
    }

    @Test
    void sayHelloToOtherThrowsException(){
        when(mockPerson.calculateHeightInInches()).thenReturn(62.9);
        assertThrows(NotBigEnoughException.class,
            ()-> {helloService.sayHelloTo(mockPerson);});
    }
}
```

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- **test application components together without deploying to an external application server**
- **validate integration with databases, messaging systems, REST APIs, and Spring infrastructure**
- **Spring Boot provides extensive testing support through:**
 - **spring-boot-starter-test**
 - **Spring TestContext Framework (spring-test)**
 - **JDBC and transaction testing support**
 - **Spring MVC / WebFlux testing support**
 - **Testcontainers integration**

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- **manage Spring IoC container caching between test executions**
 - **load `ApplicationContext` using `@SpringBootTest`, `@ContextConfiguration`, or test slices**
 - **reuse contexts across tests for improved performance**
- **provide dependency injection into test classes**
 - **use `@Autowired` in test classes**
 - **constructor injection is supported in JUnit Jupiter**
- **provide transaction management for tests**
 - **by default, transactional tests roll back after execution**
 - **use `@Commit` or `@Rollback(false)` to persist changes**

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Core Features

- context loading and caching
- dependency injection for tests
- transaction management
- SQL script execution
- active profile management
- property source customization
- test execution listeners

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Context loading and caching

- **@SpringBootTest**
 - loads the complete Spring Boot application context
 - suitable for full integration testing
 - can start embedded web servers with `webEnvironment`
- **@ContextConfiguration**
 - defines how the `ApplicationContext` should be loaded
 - can load configuration classes, XML files, or custom initializers
 - mostly used in lower-level Spring Framework tests
- **@DirtiesContext**
 - marks the Spring context as modified during a test
 - forces Spring to reload the context afterwards
 - useful when tests change shared state
 - can slow down test execution if overused

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Dependency Injection for Tests

- **@Autowired**
 - injects Spring-managed beans into test classes
 - works with fields, constructors, and setters
 - constructor injection is recommended in Spring Boot 3.x
- **@MockitoBean**
 - replaces a Spring bean with a Mockito mock inside the application context
 - commonly used to isolate services from external dependencies
 - automatically reset between tests
- **@MockitoSpyBean**
 - wraps an existing Spring bean with a Mockito spy
 - allows partial mocking while keeping real behavior
 - useful for verifying interactions

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Transaction management

- **@Transactional**
 - executes test methods within a transaction
 - by default, Spring rolls back the transaction after the test finishes
 - keeps database state isolated between tests
- **@Rollback**
 - explicitly controls rollback behavior
 - **@Rollback(false)** disables automatic rollback
- **@Commit**
 - commits the transaction after the test completes
 - useful for debugging or preparing shared test data

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Active profile management

- **@ActiveProfiles**
 - activates one or more Spring profiles for a test
 - enables profile-specific beans and configuration files
 - commonly used with application-test.properties

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- **Spring Boot provides specialized test slices**
 - **@WebMvcTest** — MVC controllers
 - **@DataJpaTest** — JPA repositories
 - **@JdbcTest** — JDBC components
 - **@JsonTest** — JSON serialization
 - **@RestClientTest** — REST clients
 - **@GraphQLTest** — GraphQL components
- **Benefits**
 - faster startup
 - reduced context size
 - better isolation

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

- **MockMvc**
 - built on Servlet API mock objects
 - no running servlet container required
 - efficient controller and REST API testing
 - fast and lightweight
 - features: request/response verification, JSON assertions, form submission testing, validation testing,...
- **WebTestClient**
 - reactive and servlet support
 - preferred for WebFlux applications
 - can test live servers or mock environments
- **Selenium / Browser Testing**
 - end-to-end browser automation
 - suitable for UI and workflow testing

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Testing in Spring

1. Testing principles
2. Introduction to Spring Testing
3. JUnit
4. Mockito
5. Spring Integration Testing

Spring Core

Objectives :

- **Container and Bean lifecycle**
- **Declaring and wiring Beans**
- **Addressing Ambiguity**
- **Profiles**
- **Runtime Value Injection**

- **objects live within the Spring container**
- **container will create the objects, wire them together, configure them, and manage their complete lifecycle, using DI**
- **several container implementations, categorized in two types**
 - ***BeanFactory***
 - `org.springframework.beans.factory.BeanFactory`
 - basic support for DI, low-level
 - ***ApplicationContext***
 - `org.springframework.context.ApplicationContext`
 - extends `BeanFactory`
 - adds application framework services
 - different implementation classes present
- **Spring Boot Application automatically creates an `ApplicationContext` implementation behind the scenes**

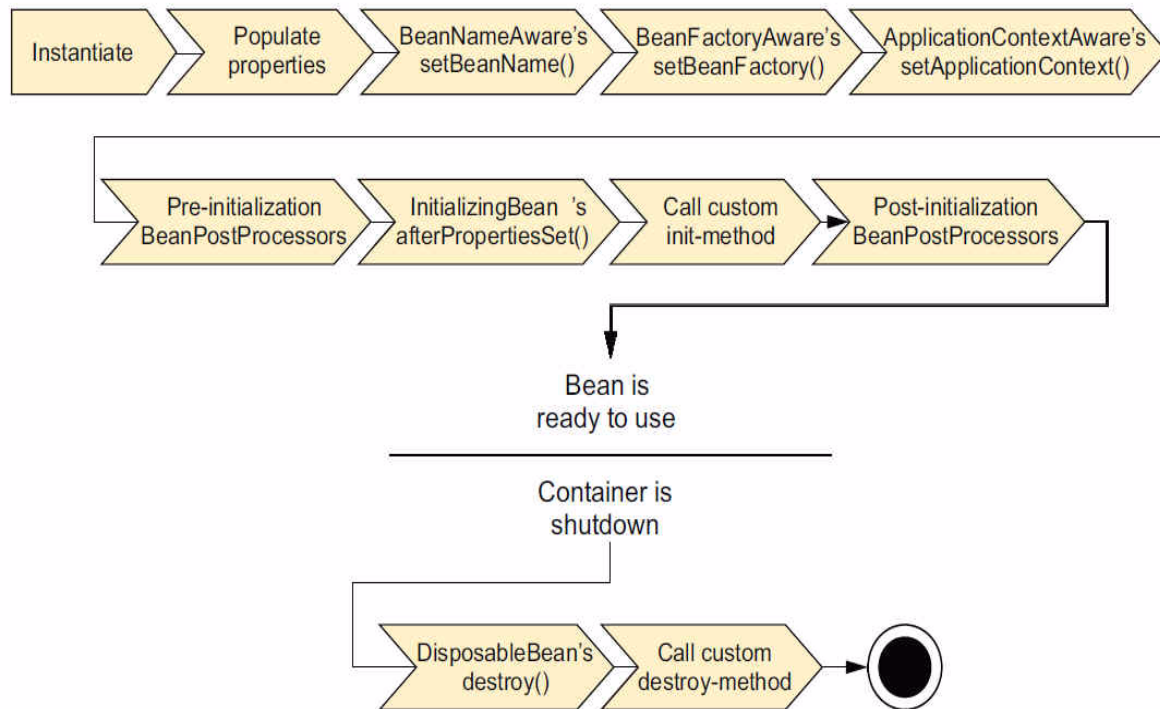
Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Bean Lifecycle

2

- **Spring (Boot) by default uses eager loading when ApplicationContext is created (i.e. when running the application or test)**
- **Lifecycle:**



- **by default, beans will be singletons!**

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Bean Scopes

- **singleton**
 - same instance of bean is returned each time
 - **DEFAULT! -> service oriented nature of Spring!**
- **prototype**
 - one instance of bean is created each time it is injected into or retrieved from the application context
- **request/session**
 - in web application, one instance for each HTTP request/session
- **globalSession**
 - in web application, one instance for each session, but only in portlet context
- **application**
 - in web application, one instance during life of a ServletContext
- **websocket**
 - one instance during life of a WebSocket

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **beans that will be managed by Spring need to be configured/declared**
- **beans themselves always defined as interface + implementation**
- **historically configuration in XML, nowadays Java-based (Application class in Spring Boot)**
- **combine with annotations on classes for automatic bean discovery**
- **configuration loaded automatically when using JUnit in Spring**
-> **@SpringBootTest**
- **different configurations can be combined if you want**

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- configuration of beans is done in a Java class
- Spring Boot already creates this class by default (...Application) in the base package of your application
- class is annotated with @SpringBootApplication (or @SpringBootTest)
- beans can be discovered via annotations in base package or its subpackages, or specifically defined in the configuration class via @Bean
- both techniques discussed above can be combined
 - autodiscovery of annotated classes for beans defined yourself
 - @Bean for 3rd party beans you want to use
(more on this, see: [3.6 Explicit definition and wiring of beans in the Java Config class](#) on page 82)

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Java Based Configuration - Example

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication
public class SpringRestTemplateClientApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringRestTemplateClientApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder.build();
    }
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **explicit configuration of beans, whether Java Config or XML is very verbose**
- **use annotations on bean classes to make them autodiscoverable and to wire them together**
- **use one of the “core” stereotypes, or a more specialized one like `@RestController` (when using `spring-boot-starter-web`) for discovery**
- **wiring will be done via `@Autowired`**
- **approach has its pros and cons (compared to explicit configuration)**
- **could use CDI annotations `@Named` and `@Inject` instead, but not often done in Spring**
- **automatically configured in Spring Boot, scans current package and sub-packages**
- **if other base-package, add:
`@ComponentScan(basePackages="be.abis.otherpackage")` on the configuration class**

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **annotations from the `org.springframework.stereotype` package**
- **4 basic types of beans:**
 - **@Component**
 - most generic annotation
 - ensures that the class will be found during classpath scanning and that it will be registered in the context as a bean
 - **@Service**
 - beans in Service Layer (alias for @Component)
 - **@Repository**
 - for DAO or Repository classes
 - represent the database access layer in an application
 - has automatic persistence exception translation enabled
 - **@Controller**
 - tells the container that the class is an MVC controller
 - usually used together with the @RequestMapping annotation

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **meta-annotation can combine annotations that are frequently used together**
- **meta-annotation can be applied to other annotations**
- **syntax:**
 - **state the Target of the annotation (Type, Method, Constructor)**
 - **add Retention Policy (annotation valid only for this class or for whole runtime)**
 - **add all annotations + all imports**
 - **public @interface MyAnnotation { } will define name of annotation**
 - **can have a body (containing annotation attributes)**
- **actually:**
 - @SpringBootApplication=**
 - @SpringBootConfiguration (special form of @Configuration)**
 - + @ComponentScan**
 - + @EnableAutoconfiguration**

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **@Value** to wire basic properties (String, int,...)
- **@Autowired** annotation used for wiring bean properties
- **wire interface, not implementation class!**
- **can be put on constructor, setter, field and even methods**
- **alternatively you could use**
 - **@Inject** (CDI, JSR 330)
 - **@Resource**
 - Common Annotations for the JavaPlatform, JSR-250
 - to inject data sources, connectors, or any other desired resources available in the JNDI namespace
 - takes a name attribute: `@Resource(name="myBean")`
- **use @PostConstruct and @PreDestroy for init and destroy methods**

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Which Type of Dependency Injection to choose?

- **constructor injection**
 - for required dependencies
 - can be used for final fields
 - avoids circular dependencies
 - as of Spring 4.3: implicit constructor injection if only 1 constructor present!
- **setter injection**
 - for optional or changeable dependencies
 - class should still be able to function when omitted
- **field injection**
 - shorter, since no setter necessary
 - tightly coupled to DI container
 - classes cannot be instantiated (e.g. in unit tests) without reflection
 - to be avoided in most cases?

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example

- **Coffee.java**

```
public class Coffee {  
    private String flavour;  
  
    public Coffee(String flavour) {this.flavour = flavour;}  
  
    public String getFlavour() {return flavour; }  
    public void setFlavour(String flavour) {this.flavour = flavour; }  
}
```

- **Person.java**

```
public class Person {  
    private String firstName;  
    private String coffeePreference;  
  
    public Person(String firstName) {this.firstName = firstName;}  
    public Person(String firstName, String coffeePreference) {  
        this(firstName);  
        this.coffeePreference=coffeePreference;  
    }  
  
    public String getFirstName() {return firstName;}  
    public void setFirstName(String firstName) {this.firstName=firstName;}  
    public String getCoffeePreference() {return coffeePreference;}  
    public void setCoffeePreference(String coffeePreference) {  
        this.coffeePreference=coffeePreference;  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example - Services - @Value

- **HelloService.java**

```
public interface HelloService {  
    Person findPerson(int id);  
    void sayHelloTo(Person person);  
}
```

- **AbisHelloService.java**

@Component

```
public class AbisHelloService implements HelloService {  
    private String helloMessage;  
    public String getHelloMessage() {return helloMessage;}  
    @Value("Welcome to Abis")  
    public void setHelloMessage(String helloMessage) {  
        this.helloMessage = helloMessage;  
    }  
    @Override  
    public Person findPerson(int id) {  
        return new Person("John");  
    }  
    @Override  
    public void sayHelloTo(Person person) {  
        System.out.println(helloMessage + ", " + person.getFirstName() + ".");  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example - Services - Init and Destroy

- **CoffeeService.java**

```
public interface CoffeeService {  
    Coffee serveCoffeeToVisitor(Person person);  
}
```

- **AbisCoffeeService.java**

```
@Service  
public class AbisCoffeeService implements CoffeeService {  
  
    @PostConstruct  
    public void init(){  
        System.out.println("preparing coffee");  
    }  
  
    @Override  
    public Coffee serveCoffeeToVisitor(Person person) {  
        return new Coffee(person.getCoffeePreference());  
    }  
  
    @PreDestroy  
    public void destroy(){  
        System.out.println("cleaning coffee things");  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example - Services - @Autowired

- **ReceptionService.java**

```
public interface ReceptionService {  
    HelloService getHelloService();  
    void welcomePersonAndGetCoffee(Person person);  
}
```

- **AbisReceptionService.java**

@Service

```
public class AbisReceptionService implements ReceptionService {  
    private HelloService helloService;  
    @Autowired private CoffeeService coffeeservice;
```

//@Autowired

```
public AbisReceptionService(HelloService helloService){  
    this.helloService=helloService;  
}
```

```
public HelloService getHelloService() {return helloService; }  
public void setHelloService(HelloService helloService) {  
    this.helloService=helloService;  
}
```

```
public void welcomePersonAndGetCoffee(Person person) {  
    helloService.sayHelloTo(person);  
    Coffee coffee= coffeeservice.serveCoffeeToVisitor(person);  
    System.out.println("Here is your " + coffee.getFlavour() + " coffee.");  
}  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example - Configuration and Tests

- **AnnotationsApplication.java**

```
@SpringBootApplication
public class AnnotationsApplication {
    public static void main(String[] args) {
        SpringApplication.run(AnnotationsApplication.class,args);
    }
}
```

- **AnnotationsApplicationTests.java**

```
@SpringBootTest
class AnnotationsApplicationTests {
    private Person person;
    @Autowired ReceptionService receptionService;

    @BeforeEach
    public void init(){ person=new Person("John","black");}

    @Test
    void personNameIsJohn(){
        assertEquals("John",
            receptionService.getHelloService().findPerson(1).getFirstName());
    }

    @Test
    void welcomeJohn(){
        receptionService.welcomePersonAndGetCoffee(person);
    }
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **define the beans via @Bean**
 - **method name (without the parentheses) = bean/instance name**
 - **use “new” to create instance as in plain java**
 - **all standard Java methods can be used**
- **wire beans together via setter injection or constructor injection, using method call to the previously defined beans**
- **Use @Bean(initMethod="init", destroyMethod="destroy") to call init and destroy methods. Make sure those are mentioned in the interface as well.**
- **use the bean in your test via @Autowired, or retrieve it explicitly:**
 - **retrieve the applicationContext by referencing the config class in your JUnit Test's @BeforeAll method**
 - **retrieve bean from applicationContext**
 - by referencing the Interface class
 - by bean name -> cast necessary!

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example

- **model + service classes as mentioned in [Example](#) on page 77, except for a small change in the CoffeeService interface:**

```
public interface CoffeeService {  
    Coffee serveCoffeeToVisitor(Person person);  
    void init();  
    void destroy();  
}
```

Make sure to take out all Spring annotations (@Service, @Autowired,...)

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example (cont.)

- **configuration class will define all the beans explicitly**

```
@SpringBootApplication
public class JavaconfigApplication {

    public static void main(String[] args) {
        SpringApplication.run(JavaconfigApplication.class,args);
    }

    @Bean
    public HelloService helloService(){
        AbisHelloService ahs = new AbisHelloService();
        ahs.setHelloMessage("Welcome to Abis");
        return ahs;
    }

    @Bean(initMethod="init" destroyMethod="destroy")
    public CoffeeService coffeeService(){
        return new AbisCoffeeService();
    }

    @Bean
    public ReceptionService receptionService(){
        ReceptionService rs = new AbisReceptionService(helloService());
        ((AbisReceptionService)rs).setCoffeeservice(coffeeService());
        return rs;
    }
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Example (cont.)

- **test**

```
class JavaconfigApplicationTests {  
  
    private static ReceptionService receptionService;  
    private Person person;  
  
    @BeforeAll  
    public static void retrieveBean(){  
        ApplicationContext context =  
            new AnnotationConfigApplicationContext(JavaconfigApplication.class);  
        receptionService = (ReceptionService) context.getBean("receptionService");  
    }  
  
    @BeforeEach  
    public void init(){ person = new Person("John","black");}  
  
    @Test  
    void personNameIsJohn(){  
        assertEquals("John",  
            receptionService.getHelloService().findPerson(1).getFirstName());  
    }  
  
    @Test  
    void welcomeJohn(){  
        receptionService.welcomePersonAndGetCoffee(person);  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- in case a Java config class gets too big, you can split it up
- these are automatically discovered by Spring Boot

@Configuration

```
public class SecondConfig {  
    @Bean  
    public HelloService helloService() {  
        AbisHelloService ahs = new AbisHelloService();  
        ahs.setHelloMessage("Welcome to Abis");  
        return ahs;  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **@Autowired will by default use type inference to find implementation class**
- **if more implementation of the interface present, a `NoUniqueBeanDefinitionException` will be thrown**
- **solve with:**
 - **@Qualifier** to distinguish between beans
 - **@Primary** to designate primary (“favourite”) bean

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

@Qualifier Example

- **CampinaCoffeeService.java**

```
@Service
@Qualifier("campina")
public class CampinaCoffeeService implements CoffeeService {
    @Override
    public Coffee serveCoffeeToVisitor(Person person) {
        return new Coffee(person.getCoffeePreference() + " with extra milk");
    }
}
```

- **AbisCoffeeService.java**

```
@Service
@Qualifier("Abis")
public class AbisCoffeeService implements CoffeeService {
    //content: see earlier
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

@Qualifier Example (cont.)

- **AbisReceptionService.java**

```
@Service
public class AbisReceptionService implements ReceptionService {
    private HelloService helloService;
    @Autowired
    @Qualifier("campina")
    private CoffeeService coffeeservice;
    // rest of content: see earlier
}
```

- **QualifierApplicationTests.java**

```
@SpringBootTest
class QualifierApplicationTests {
    private Person person;
    @Autowired ReceptionService receptionService;

    @BeforeEach
    public void init(){ person=new Person("John","black");}

    @Test
    void welcomeJohn(){
        receptionService.welcomePersonAndGetCoffee(person);
    }
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

@Primary Example

- **CampinaCoffeeService.java**

```
@Service
public class CampinaCoffeeService implements CoffeeService {
    @Override
    public Coffee serveCoffeeToVisitor(Person person) {
        return new Coffee(person.getCoffeePreference() + " with extra milk");
    }
}
```

- **AbisCoffeeService.java**

```
@Service
@Primary
public class AbisCoffeeService implements CoffeeService {
    //content: see earlier
}
```

- **AbisReceptionService.java**

```
@Service
public class AbisReceptionService implements ReceptionService {
    private HelloService helloService;
    @Autowired
    private CoffeeService coffeeservice;
    // rest of content: see earlier
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- **profile = named, logical group of bean definitions**
- **environment determines which profiles (if any) are currently active, and which profiles (if any) should be active by default**
- **use cases**
 - **working against an in-memory datasource in development vs looking up that same datasource from JNDI when in QA or production**
 - **registering monitoring infrastructure only when deploying an application into a performance environment**
 - **registering customized implementations of beans for customer A vs. customer B deployments**

application does not need to be rebuilt for each environment!

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Configuring and Activating Profiles

- **Java Config (class or method level)**
`@Profile("development")`
- **default profile: `@Profile("default")`**
- **activation**
 - **NoSuchBeanDefinitionException** thrown when profile not activated, unless default profile specified
 - **programmatic, via Environment object**
 - **declarative, via `spring.profiles.active` property (command-line or in `application.properties` when using Spring Boot)**
 - **via `@ActiveProfiles` in Test**
 - **can have more than one profile active at once**

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Profiles - Example

- **CampinaCoffeeService.java**

```
@Service
@Profile("test")
public class CampinaCoffeeService implements CoffeeService {
    @Override
    public Coffee serveCoffeeToVisitor(Person person) {
        return new Coffee(person.getCoffeePreference() + " with extra milk");
    }
}
```

- **AbisCoffeeService.java**

```
@Service
@Profile("production")
public class AbisCoffeeService implements CoffeeService {
    //content: see earlier
}
```

- **AbisReceptionService.java**

```
@Service
public class AbisReceptionService implements ReceptionService {
    private HelloService helloService;

    @Autowired
    private CoffeeService coffeeservice;

    // rest of content: see earlier
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Profiles - Example (cont.)

- **application.properties**

spring.profiles.active=test

- **ProfilesApplicationTests.java**

@SpringBootTest

//@ActiveProfiles("production") // will override the setting of application.properties

class ProfilesApplicationTests {

private Person person;

@Autowired ReceptionServices receptionService;

@BeforeEach

public void init(){ person=new Person("John","black");}

@Test

void welcomeJohn(){

receptionService.welcomePersonAndGetCoffee(person);

}

}

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- to avoid hard-coded values, determined at runtime instead
- via property placeholders

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

- add a new property to `application.properties`
- use `${myPropertyName}` to call it
- in case you want to use a specific file for these properties:
 - create `my.properties` under the `resources` folder
 - add in your configuration class via `@PropertySource("classpath:my.properties")`
 - `application.properties` has precedence over this new file
- to programmatically resolve a property:
 - wire `Environment env` in your class
 - get properties via `env.getProperty("mykey");`

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Property from application.properties - Example

- **application.properties**

```
spring.profiles.active=production  
hello.company=Abis
```

- **GeneralHelloService.java**

```
@Service  
public class GeneralHelloService implements HelloService {  
    private String helloMessage;  
    public String getHelloMessage() {  
        return helloMessage;  
    }  
    @Value("Welcome to ${hello.company}")  
    public void setHelloMessage(String helloMessage) {  
        this.helloMessage=helloMessage;  
    }  
  
    public Person findPerson(int id) {  
        return new Person("John");  
    }  
  
    public void sayHelloTo(Person person) {  
        System.out.println(helloMessage + ", " + person.getFirstName());  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Property from my.properties - Example

- **application.properties**

spring.profiles.active=production

- **my.properties**

hello.company=Abis

- **RuntimevalueinjectionApplication.java**

```
@SpringBootApplication
@PropertySource("classpath:my.properties")
public class RuntimevalueinjectionApplication {
    public static void main(String[] args) {
        SpringApplication.run(RuntimevalueinjectionApplication.class, args);
    }
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Property from code - Example

- **application.properties**

```
spring.profiles.active=production  
hello.company=Abis
```

- **GeneralHelloService.java**

```
@Service  
public class GeneralHelloService implements HelloService {  
  
    @Autowired private Environment env;  
    private String helloMessage;  
    public String getHelloMessage() {  
        return helloMessage;  
    }  
    public void setHelloMessage(String helloMessage) {  
        this.helloMessage=helloMessage;  
    }  
  
    public Person findPerson(int id) {  
        return new Person("John");  
    }  
  
    public void sayHelloTo(Person person) {  
        System.out.println("Welcome at " + env.getProperty("hello.company")  
            + ", " + person.getFirstName());  
    }  
}
```

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Spring Core

1. The Container
2. Bean Lifecycle
3. Declaring and Wiring Beans
4. Addressing Ambiguity
5. Profiles
6. Runtime Value Injection

Aspect-oriented Spring

Objectives :

- What is AOP
- Terminology
- Configuring AOP in Spring

What is AOP

1

General

1.1

- **aspect-oriented programming**
- **separate cross-cutting concerns from the business logic**
logging, transactions, security
- **declaratively define how and where functionality is applied**
- **cross-cutting concerns modularized into special classes**
-> aspects

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- *Aspect*

The general functionality that has to be handled

- *Advice*

The what and when of an aspect

what: code to be executed

when: before, after, after-returning, after-throwing, around

- *Join Points*

point in execution where aspect **could** be plugged in

- *Pointcuts*

matches the join points **where** the advice is applied

- *Weaving*

applying aspects via a proxied object

can be done at compile-time, classload time or runtime

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- **AOP frameworks differ in join point model, weaving approach,...**
- **Three major players**
 - **AspectJ**
 - **JBoss AOP**
 - **Spring AOP**
- **Spring AOP borrows a lot from AspectJ**
- **Advice is written in Java: Aspect is a class, advice are methods**
- **Spring advises objects at runtime (weaving is proxy-based)**
 - **only methods defined in the interface can be used**
 - **methods calling methods in the same class, won't be advised**
- **Spring only supports method join points**
- **Advice should not influence behaviour of original method!**

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- **AspectJ pointcut expression language is used**
- **Spring supports a subset of AspectJ pointcut designators**
args(), @args(), execution(), this(), target(), @target(), within(), @within() and @annotation()
additional: bean()
- **within()**
within(com.xyz.service.*) -> any method within the service package
within(com.xyz.service..*) -> any method within the service package or a sub-package
- **target()**
target(com.xyz.service.AccountService) -> any method where the target object implements the AccountService interface
- **bean()**
bean(myHelloBean) -> any method on a bean named myHelloBean (id-based!)
bean(*Service) -> any method on Spring beans having names that match the wildcard expression '*Service'
- **args() -> to pass args to advice, used together with execution()**
args(name)

Writing pointcuts (cont.)

- **execution()**
 - **execution(modifiers-pattern? ret-type-pattern declaring-type-pattern? name-pattern(param-pattern) throws-pattern?)**
 - * wildcard can be used for return type + as all or part of a name-pattern
 - param-pattern:
 - () : method without arguments
 - (..) : method with any number of arguments (zero or more)
 - (*) : one parameter of any type
 - (* ,String): two parameters, first any type, second of type String
 - examples
 - execution(public * *(..)) -> execution of any public method
 - execution(* set*(..)) -> execution of any method with name beginning with set
 - execution(* Instrument.play(..)) -> execution of play method in Instrument class
 - execution(* com.xyz.service.*.(..)) -> execution of any method in service package

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- **AspectJ's annotation-oriented model**
- **@Aspect** makes class an aspect
- **@Pointcut** defines pointcut (combined with type: after, before,...)
- **Maven Dependency**

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-aop</artifactId>  
</dependency>
```

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- **Types of advices:**
 - **@Before, @After**
 - **@AfterReturning, @AfterThrowing**
 - **@Around**
- **put just before the advice method, combined with a pointcut**
- **possibility to define named pointcuts that can be combined**

```
public class MyPointcuts {  
    @Pointcut("within(com.xyz.service.*)")  
    public void inService(){}  
    @Pointcut("execution(* perform*(..))")  
    public void execPerform(){}  
    @Pointcut(inService() && execPerform())  
    public void execPerformInService(){}  
}
```

-> use via **@Pointcut(MyPointCuts.execPerform())**

- **passing arguments: @AspectJ leans on Java syntax to determine details of parameters passed, arg-names not necessary**

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- **aspect is a pure Java class with (possibly) multiple methods (the advices)**
- **@Before: executed BEFORE executing the original method called by the PointCut**
- **@After: executed AFTER executing the original method called by the PointCut**
- **@AfterReturning: executed AFTER the original method called by the PointCut returned correctly**
- **@AfterThrowing: executed AFTER the original method called by the PointCut returned threw an Exception**
- **@AfterTrowing: possibility to work with / access the exception**
- **advice method's return-type is void**
- **(Proceeding)JoinPoint can be used to give information about the method/class that is under advice**
- **(Proceeding)JoinPoint has to be FIRST argument of advice!**

- single method for before and after advice
- if information needs to be shared
- return-type is **void** or **Object**
- **Object** -> if original method returns something, to fetch returned value
- Uses **ProceedingJoinPoint** instead of **JoinPoint**
- **ProceedingJoinPoint pjp -> (Object o =) pjp.proceed();**
should be called explicitly

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

- **HelloService.java**

```
public interface HelloService {  
    Person findPerson(int id);  
    void sayHelloTo(Person person);  
}
```

- **AbisHelloService.java**

```
@Service  
public class AbisHelloService implements HelloService {  
    private String helloMessage;  
    public String getHelloMessage() {  
        return helloMessage;  
    }  
    public void setHelloMessage(String helloMessage) {  
        this.helloMessage=helloMessage;  
    }  
  
    public Person findPerson(int id) {  
        return new Person("John");  
    }  
  
    public void sayHelloTo(Person person) {  
        System.out.println("Welcome at Abis, " + person.getFirstName());  
    }  
}
```

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

}Example (cont.)

- **AbisReceptionService.java**

```
@Service
public class AbisReceptionService implements ReceptionService {
    private HelloService helloService;
    @Autowired CoffeeService coffeeService;

    //@Autowired
    public AbisReceptionService(HelloService helloService) {
        this.helloService = helloService;
    }

    //getters and setters

    @Override
    public void welcomePersonAndGetCoffee(Person person)
        throws PersonNotWelcomeException {
        if (!person.getFirstName().equals("Mr. Bean")) {
            helloService.sayHelloTo(person);
            Coffee coffee = coffeeService.serveCoffeeToVisitor(person);
            System.out.println("Here is your " + coffee.getFlavour() + ".");
        } else {
            throw new PersonNotWelcomeException
                (person.getFirstName()+" , you are not welcome.");
        }
    }
}
```

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

Example (cont.)

- **AopPointcuts.java**

```
public class AopPointcuts {  
    @Pointcut("execution(* welcome*(..))")  
    public void execWelcome(){}  
    @Pointcut("execution(* be.abis.demo.service.*.get*(..))")  
    public void execGet(){}  
    @Pointcut(  
        "execution(* be.abis.demo.service.*.find*(..))"  
    )  
    public void findThings(){}  
}
```

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

Example (cont.)

- **MyAspect.java**

@Component

@Aspect

public class MyAspect {

@Before("execution(public * be.abis.demo.service.*.welcome*(..))")

public void beforeWelcome(){

System.out.println("Receptionist stands up:");

}

@AfterReturning("AopPointcuts.execWelcome()")

public void afterWelcomeOK(JoinPoint jp){

System.out.println("Guest is guided to lounge.");

}

@AfterThrowing(pointcut="AopPointcuts.execWelcome()",throwing="exc")

public void afterNotWelcome(GuestNotWelcomeException exc){

System.out.println("You are escorted back to the door " +exc.getMessage());

}

@After("AopPointcuts.execGet()")

public void afterGetting(JoinPoint jp){

System.out.println("finished calling getter: " + jp.getSignature().getName());

}

//continued on next page!

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

Example (cont.)

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

```
// MyAspect.java continued
@Around("AopPointcuts.findThings()")
public Object timing(ProceedingJoinPoint pjp){
    Object o =null;
    try {
        long start = System.nanoTime();
        o = pjp.proceed();
        long end= System.nanoTime();
        System.out.println("time taken by " + pjp.getSignature().getName() + " : " +
            ((end-start)/1000000.0) + " milliseconds");
    } catch (Throwable e) {
        e.printStackTrace();
    }
    return o;
}
}
```

Example (cont.)

- **ReceptionTest.java**

```
@SpringBootTest
class ReceptionTest {
    private Person person;
    @Autowired ReceptionService receptionService
    @BeforeEach public void init() { person=new Person("John" ,"black coffee"); }

    @Test void personNameIsJohn(){
        assertEquals("John",
            receptionService.getHelloService().findPerson(1).getFirstName());
    }

    @Test void welcomeJohn() throws PersonNotWelcomeException {
        receptionService.welcomePersonAndGetCoffee(person;
    }

    @Test
    void welcomeMrBeanException() {
        Person person2 = new Person("Mr. Bean", "none");
        assertThrows(PersonNotWelcomeException.class,
            () -> {receptionService.welcomePersonAndGetCoffee(person2);});
    }
}
```

Aspect-oriented Spring

1. What is AOP
2. Writing pointcuts
3. Defining Aspects
4. Annotating Advices

Introduction to REST and JSON

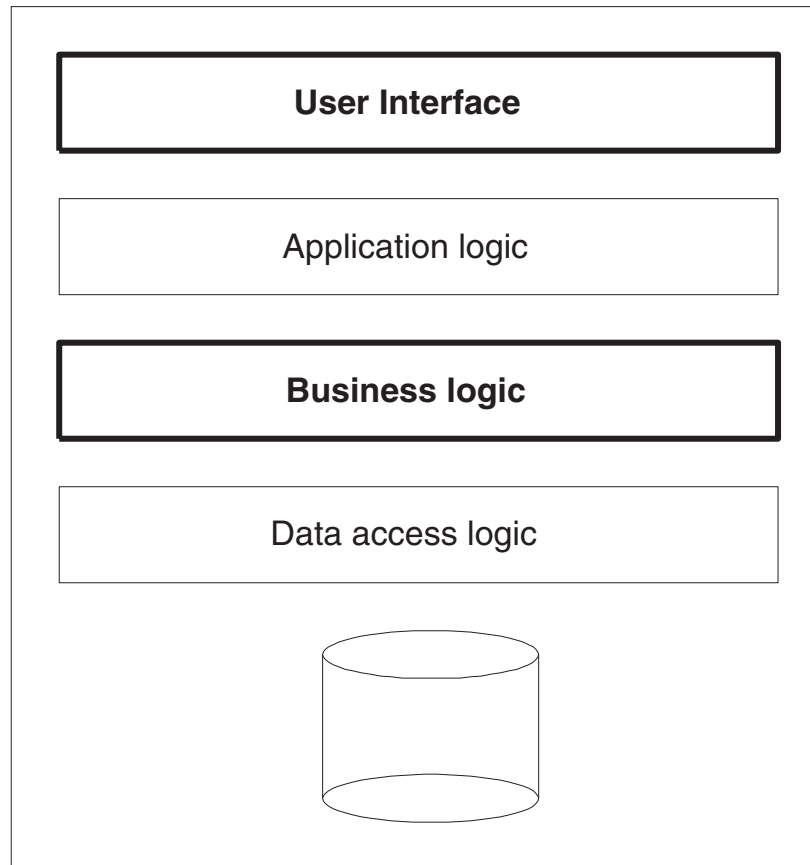
Objectives :

- **Application Development and Distributed Computing**
- **RESTful Web Services / APIs**
- **JSON vs XML**

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Structuring an Application: Layers

1.1



1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Think further than just this 1 application you want to build

Several interfaces on same business layer/domain model

Creating modular applications

- no “spaghetti” code
- reuse of parts (even code written in other programming languages)

Easy maintenance

Extensibility

“Different components and objects making up an application can be located on different computers connected to a network”

Advantages:

- **Reuse (multiple-client applications)**
- **Location transparency (standard mechanisms for lookup)**
- **Higher reliability and availability (clustering)**
- **Scalability (deploying on powerful servers)**
- **Higher performance (parallel processing)**
- **Extensibility (dynamic configuration)**
- **Higher productivity and lower development time (small programs)**
- **Interoperability and reduced vendor dependence**

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

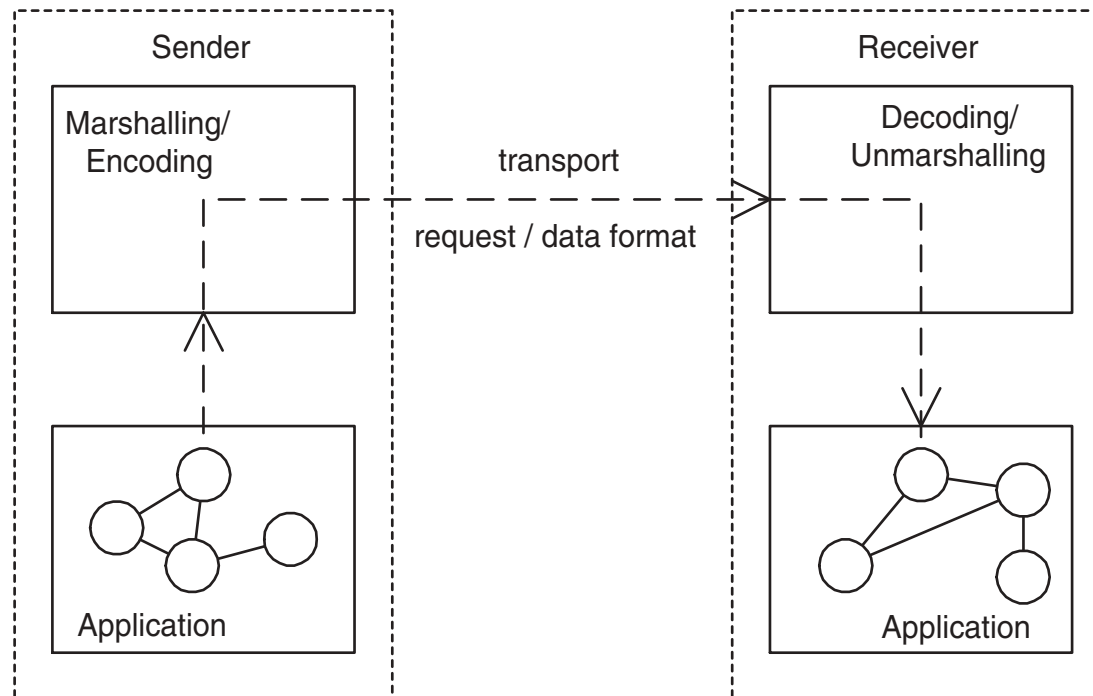
1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Where does each layer run?

Evolution:

- **monolithic**
- **client / server: 2-tier, 3-tier,... (e.g. Java + Oracle server)**
- **n-tier, distributed components (Corba, Java RMI, MS DCOM)**
- **fully distributed, heterogeneous systems:**
 - **independent services**
 - **standard internet technology (web / cloud / mobile / IoT /...)**

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON



- **remote communication (remote procedure call / RPC)**
- **request / (complex) data structure / data types**
- **marshal / serialize / encode**

Remote 'Object' Protocols

CORBA (IIOP)

MS DCOM

Java RMI (Remote Method Invocation)

WDDX (Web Distributed Data Exchange)

XML-RPC (XML Remote Procedure Call)

SOAP

REST (Representational State Transfer)

JSON-RPC

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Before: classic web services with SOAP and WSDL

- well structured & governable, but complex

From +/- 2010: shift to more lightweight 'API'

- easier access to services outside the enterprise
- more agile, less ceremony
- often using REST and JSON vs. SOAP/XML
- more suited for consumer apps, new channels, mashups (Web 2.0) etc...

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

REST = Representational State Transfer

- **Representational** - resources can be represented in virtually any form, including XML, JavaScript Object Notation (JSON), or even HTML, whatever form best suits the consumer of resources
- **State** - more concerned with the state of a resource than with the actions you can take against resource
- **Transfer** - transfer of resource data, in some representational form, from one application to another

A basic philosophy: “everything is a resource on the web”, should be addressable (URI)

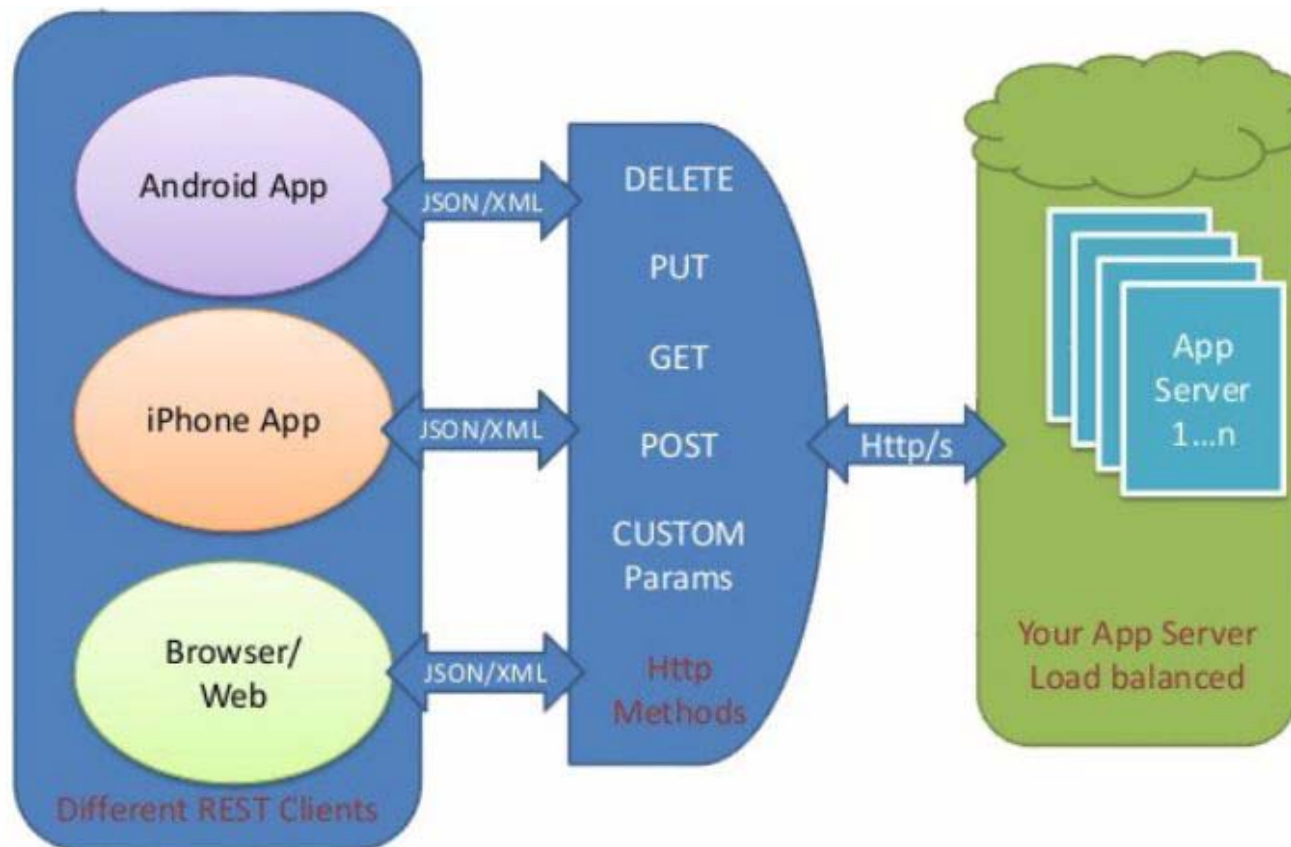
Example: `http://twitter.com/users?screen_name=james`

Uses HTTP to access resources:

4 basic operations (cf. CRUD): GET, PUT, POST, DELETE

REST is an architectural style, not a protocol like SOAP

REST API Architecture



Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

REST is not an architecture, but a set of constraints that creates a software architectural style

Following constraints define a RESTful system:

- **Client-server:** client and the server should be loosely coupled
- **Stateless:** there should be no need for the service to keep users' sessions
- **Cacheable:** network infrastructure should support a cache to avoid repeated round trips between the client and the server for retrieving the same resource
- **Uniform interface:** resource exposed must have a unique address and should be accessible through a generic interface
- **Layered:** server can have multiple layers for implementation
- **Code on demand (optional):** functionality of the client applications can be extended at runtime by allowing a code download from the server and executing the code

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

RESTful resource is anything that is addressable over the web

Resource is a logical, temporal mapping to a concept in the problem domain

Examples:

- a news story
- the temperature in Leuven at 4:00 p.m. EST
- a person working for a company
- list of participants to a course

URI in a RESTful web service is a hyperlink to a resource

URI is not meant to change over time as it may break the contract between a client and a server

Resource's mapping is unique, but different requests for a resource can return the same underlying binary representation from the server

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Map CRUD actions on the resources to the appropriate HTTP methods

Data Action	HTTP equivalent
CREATE	POST
READ	GET
UPDATE	PUT (or PATCH)
DELETE	DELETE

Will be linked with **@GetMapping**, **@PostMapping**,... in the Spring controller classes

Other methods: **HEAD** and **OPTIONS**

Representation (of resource) =

- what is sent back and forth between clients and servers
- temporal state of the actual data located in some storage device at the time of a request
- binary stream together with its metadata that describes how the stream is to be consumed by the client
- can take various forms, e.g. image, text file, XML, JSON

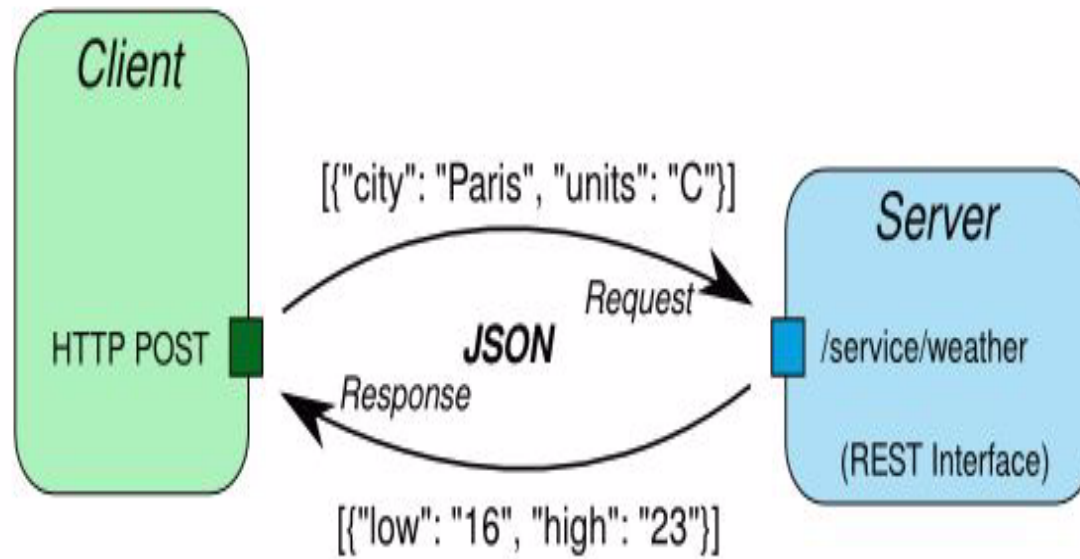
All clients will use the same URI with appropriate accept header values for accessing the same resource in different representations

Metadata can also contain extra information about the resource

- validation
- encryption information
- extra code to be executed at runtime

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Returning resource via HTTP request



Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Use nouns to represent resources

Consistency is the key

Never use CRUD function names in URIs

Use query component to filter URI collection

Use nouns to represent resources

Resource is a thing (noun) instead of an action (verb)

Four categories of resource archetypes

- **document**
 - single resource inside resource collection
 - use “singular” name to denote document resource archetype
 - **examples:**
 - `http://api.example.com/device-management/managed-devices/{device-id}`
 - `http://api.example.com/user-management/users/{id}`
 - `http://api.example.com/user-management/users/admin`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Use nouns to represent resources (cont.)

- **collection**
 - **server-managed directory of resources**
 - **clients may propose new resources to be added to a collection, but is up to the collection to choose to create a new resource or not**
 - **Use “plural” name to denote collection resource archetype**
 - **examples:**
 - `http://api.example.com/device-management/managed-devices`
 - `http://api.example.com/user-management/users`
 - `http://api.example.com/user-management/users/{id}/accounts`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Use nouns to represent resources (cont.)

- **store**

- **client-managed resource repository**
- **lets an API client put resources in, get them, decide to delete**
- **store never generates new URIs, each stored resource has a URI that was chosen by a client when it was initially put into the store**
- **use “plural” name to denote store resource archetype**

- **examples**

`http://api.example.com/cart-management/users/{id}/carts`
`http://api.example.com/song-management/users/{id}/playlists`

- **controller**

- **models resource as a procedural concept**
- **like executable functions, with parameters and return values**
- **use “verb” to denote controller archetype**

- **examples:**

`http://api.example.com/cart-management/users/{id}/cart/checkout`
`http://api.example.com/song-management/users/{id}/playlist/play`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Consistency is the key

Use forward slash (/) to indicate a hierarchical relationship

- forward slash (/) character is used in the path portion of the URI to indicate a hierarchical relationship between resources
- examples:
 - `http://api.example.com/device-management`
 - `http://api.example.com/device-management/managed-devices`
 - `http://api.example.com/device-management/managed-devices/{id}`
 - `http://api.example.com/device-management/managed-devices/{id}/scripts`
 - `http://api.example.com/device-management/managed-devices/{id}/scripts/{id}`

Do not use trailing forward slash (/) in URIs

- adds no semantic value and may cause confusion
- better to drop them completely
- examples:
 - NOT:
`http://api.example.com/device-management/managed-devices/`
 - BUT:
`http://api.example.com/device-management/managed-devices`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Consistency is the key (cont.)

Use hyphens (-) to improve the readability of URIs

- easier for people to scan and interpret
- use to improve the readability of names in long path segments
- examples:

NOT:

`http://api.example.com/inventory-management/
managedEntities/{id}/installScriptLocation`

BUT:

`http://api.example.com/inventory-management/
managed-entities/{id}/install-script-location`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Consistency is the key (cont.)

Do not use underscores (_)

- possible that the underscore (_) character can get partially obscured or completely hidden in some browsers or screens
- use hyphens (-) instead
- examples

NOT:

`http://api.example.com/inventory_management/managed_entities/{id}/
install_script_location`

BUT:

`http://api.example.com/inventory-management/managed-entities/{id}/
install-script-location`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Consistency is the key (cont.)

Use lowercase letters in URIs

- lowercase letters should be consistently preferred in URI paths
- RFC 3986 defines URIs as case-sensitive except for the scheme and host components

- examples

`http://api.example.org/my-folder/my-doc //1`

`HTTP://API.EXAMPLE.ORG/my-folder/my-doc //2`

`http://api.example.org/My-Folder/my-doc //3`

in above examples, 1 and 2 are the same but 3 is not as it uses My-Folder in capital letters

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Consistency is the key (cont.)

Do not use file extensions

- looks bad and does not add any advantage
- removing them decreases the length of URIs
- if you want to highlight the media type of API using file extension then rely on the media type communicated through the Content-Type header
- examples:
NOT:
`http://api.example.com/device-management/managed-devices.xml`
BUT:
`http://api.example.com/device-management/managed-devices`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Never use CRUD function names in URIs

URIs should not indicate that a CRUD function is performed

URIs should be used to uniquely identify resources and not any action upon them

HTTP request methods should be used to indicate which CRUD function is performed

Examples:

//get all devices

HTTP GET `http://api.example.com/device-management/managed-devices`

//create new device

HTTP POST `http://api.example.com/device-management/managed-devices`

//get device for given id

HTTP GET `http://api.example.com/device-management/managed-devices/{id}`

//update device for given id

HTTP PUT `http://api.example.com/device-management/managed-devices/{id}`

//delete device for given id

HTTP DELETE `http://api.example.com/device-management/managed-devices/{id}`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Use query component to filter URI collection

In case you need a collection of resources sorted, filtered or limited based on some certain resource attribute(s)

Do not create new APIs – rather enable in resource collection API

Pass the input parameters as query parameters

Examples

`http://api.example.com/device-management/managed-devices`

`http://api.example.com/device-management/managed-devices?region=USA`

`http://api.example.com/device-management/managed-devices?region=USA&brand=XYZ`

`http://api.example.com/device-management/managed-devices
?region=USA&brand=XYZ&sort=installation-date`

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

XML

3.1

- **XML = Extensible Markup Language**
- **a universal way of structuring text documents**
- **XML 1.0 (1998/2000)**
- **from World Wide Web Consortium (W3C)**
- **markup language, cf. SGML, HTML,...**
- **objectives:**
independent, interoperable, simple, human readable,...
- **importance of XML**
 - **data definition (well formed)**
 - **data validation (valid)**
 - **data interchange**
 - **hierarchical**
 - **namespaces**

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

XML example

```
<?xml version="1.0" encoding="UTF-8"?>
<CourseParticipants>
  <Participant>
    <firstName>Maria</firstName>
    <lastName>Meuris</lastName>
    <age>52</age>
  </Participant>
  <Participant>
    <firstName>Bert</firstName>
    <lastName>Mak</lastName>
    <age>34</age>
  </Participant>
  <Participant>
    <firstName>Peter</firstName>
    <lastName>Buenk</lastName>
    <age>26</age>
  </Participant>
</CourseParticipants>
```

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

JavaScript Object Notation:

- ECMA international standard since 2013 (but already exists longer)
- open-standard file format
- serialization of data/objects (simpler than XML)
- also used outside JavaScript
- used in REST, JSON-RPC, Ajax,...

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

JSON syntax

- **name-value pairs, enclosed by curly braces (`{}`), to form objects**
- **name-value pairs are separated by commas (`,`)**
- **colon (`:`) between name and value**
- **name must be a string, and put between double quotes (`"`)**
- **values must be one of the following data types:**
 - a string
 - a number
 - an object (JSON object)
 - an array
 - a boolean
 - null
- **curly braces (`{}`) also used to denote further nesting**
- **square brackets (`[]`) hold arrays**

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

JSON example

```
[  
  {"firstName": "Maria", "lastName": "Meuris", "age": 52},  
  {"firstName": "Bert", "lastName": "Mak", "age": 34},  
  {"firstName": "Peter", "lastName": "Buenk", "age": 26}  
]
```

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Introduction to REST and JSON

1. Application Development and Distributed Computing
2. RESTful Web Services / APIs
3. XML and/vs. JSON

Spring REST basics

Objectives :

- **Spring's REST basics: Hello World REST API**
- **Testing APIs**

- builds upon Spring MVC
- controllers can handle requests for all HTTP methods. Spring 3.2 and higher also supports the PATCH method
- **@PathVariable** annotation enables controllers to handle requests for parameterized URLs
- resources can be represented in a variety of ways using Spring views and view resolvers, including implementations for rendering model data as XML, JSON (default), Atom, and RSS
- view-based rendering can be bypassed altogether using the **@ResponseBody** annotation and various **HttpMethodConverter** implementations
- **@RequestBody** annotation, along with **HttpMethodConverter** implementations, can convert inbound HTTP data into Java objects passed in to a controller's handler methods
- applications can consume REST resources using **RestTemplate** or **WebClient**
- Spring Boot helps to simplify the configuration

- **Maven Dependencies**

- **Spring MVC / REST + tomcat-starter + jackson-bind + spring-web**

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

- **Packaging**

- **jar: embedded Tomcat Server will be used**
 - **war: can deploy on own server (but extra configuration needed)**

Spring Boot auto-configuration

- **only have to write the controllers, rest is handled automatically**
- **use application.properties (in src/main/resources) to set up the port and context root of your web application**

`server.port=8080`

`server.servlet.context-path=/helloapi`

Spring REST basics

1. Spring REST basics
2. Testing REST APIs

The Basics

- configuration is annotation-based (main one: `@RestController`)
- `@RequestMapping` maps to the url requested by the client
 - default HTTP method = GET
 - also possible to use `@GetMapping` instead
- wire in your Service class(es)
- request-handling method can have any name
- return will be object
 - text/plain when returning String
 - application/json when returning other object type

Accessing Request Input

- use `@RequestParam` attribute in controller method
- pass in URL via `localhost:8080/helloapi/welcome?name=John`

- **Controller**

```
@RestController
public class HelloRestController {

    @Autowired
    HelloService helloService;

    @GetMapping("/welcome")
    public String returnMessage(@RequestParam String name){
        Guest guest = new Guest(name);
        return helloService.sayHelloTo(guest);
    }

    @GetMapping("/welcomeGuest")
    public Guest returnGuest(@RequestParam int id){
        return helloService.findGuest(id);
    }
}
```

- **Test via:**

- localhost:8080/helloapi/welcome?name=John
- localhost:8080/helloapi/welcomeGuest?id=2

Testing REST APIs

2

SoapUI

2.1

Open-source web service testing application for service-oriented architecture (SOA) and representational state transfer (REST)

Owned by SmartBear Software

Core features:

- web services inspection
- web services invoking
- web services development
- web services simulation and mocking
- web services functional, compliance and security testing

SoapUI Pro (commercial enterprise version) adds a number of productivity enhancements to the SoapUI core

<https://www.soapui.org/downloads/soapui.html>

Spring REST basics

1. Spring REST basics
2. Testing REST APIs

1. Spring REST basics
2. Testing REST APIs

Postman company

Originally only as Chrome plugin (deprecated),
nowadays a standalone app

More REST oriented, but SOAP can be tested as well (with some effort)

Need to create a (free) account

Postman Pro and Postman Enterprise add extra features

<https://www.getpostman.com/downloads/>

More REST API functionality

Objectives :

- Defining REST resources with **@RequestMapping**
- Mapping the HTTP request methods
- Request parameters: **@PathVariable**, **@RequestParam** and **@RequestBody**
- More JSON mapping
- Producing and Consuming XML

Basics of @RequestMapping

- URI path to which a resource class or method will respond
- value is relative to URI of the server where REST resource is hosted
- annotation can be applied at both the class and the method levels

Specifying @RequestMapping on a controller class or method

- path defined on class level is relative to the base path
- base path typically takes the following URI pattern:
`http://host:port/<context-root>`
- path defined on method level is relative to the path defined at the class level

```
@RestController
```

```
@RequestMapping("guests")
```

```
public class RestReceptionController { }
```

```
@RequestMapping(path = "/welcome")
```

```
public String returnMessage(@RequestParam("name") String name) { }
```

URI:

```
http://localhost:8081/springrest/guests/welcome?name=John
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Defining RESTful Resources (cont.)

Specifying variables in the URI path template

- **path template allows you to define variables that appear as placeholders in the URI**
- **define variable between curly braces ({ })**
- **variable will be linked with @PathVariable**
`@RequestMapping("{id}")`
`public Guest returnGuestById(@PathVariable("id") int id) {`
 `return receptionService.findGuest(id);`
`}`
- **variable will be replaced at runtime with the values set by the client**
`http://localhost:8081/springrest/guests/2`

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Two ways to pass HTTP method

- **@RequestMapping(path="", method=RequestMethod.GET)**
(or POST, PUT, DELETE instead of GET)
- abbreviated syntax (since Spring 4.3)
@GetMapping, @PostMapping, @PutMapping, @DeleteMapping

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- for retrieving the resources referenced in the URI path
- often used together with the request parameters
- return Object
- Object will by default be transformed to JSON
- example:

```
@GetMapping("{id}")
public Guest returnGuestById(@PathVariable int id) {
    return receptionService.findGuest(id);
}

@GetMapping("")
public List<Guest> returnall(){
    return receptionService.findAllGuests();
}

@GetMapping("/query")
public Guest returnGuestByName(@RequestParam String name) {
    return receptionService.findGuest(name);
}
```

URI:

```
http://localhost:8081/springrest/guests/1
http://localhost:8081/springrest/guests
http://localhost:8081/springrest/guests/query?name=John
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- to send data to the server (new object to be created)
- linked with @RequestBody
- transforms the incoming JSON to the Object type specified
- no-arg constructor needs to be present in model class!
- return void
- example

```
@PostMapping("")  
public void addGuest(@RequestBody Guest guest){  
    receptionService.addGuest(guest);  
}
```

JSON passed:

```
{"id": 3, "name": "Bart", "age": 43 }
```

URI:

```
http://localhost:8081/springrest/guests
```

- cannot be tested any more by simply putting URI in browser!

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- for updating the resource pointed to by the URI
- generally has a message body carrying the payload
- payload will be mapped to the method parameter as appropriate by the framework

- return void

- example:

```
@PutMapping("{id}")
public void changeGuest(@PathVariable int id, @RequestBody Guest guest) {
    receptionService.changeGuest(guest);
}
```

JSON passed:

```
{"id": 3, "name": "Bart", "age": 34 }
```

URI:

```
http://localhost:8081/springrest/guests/3
```

- remark: whole object passed!
- alternative: @PatchMapping

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- to delete a resource
- pass the id of the resource
- return void
- example

```
@DeleteMapping("{id}")  
public void deleteGuest(@PathVariable int id){  
    receptionService.deleteGuest(id);  
}
```

URI:

http://localhost:8081/springrest/guests/3

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Three possibilities to pass data in request

- **@PathVariable**
- **@RequestParam**
- **@RequestBody**

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- **will be added on the URL requested**
`http://localhost:8081/springrest/guests/1`
- **passed via @RequestMapping variable**
`@RequestMapping("{id}")`
- **pass to method via:**
`public Guest findGuest(@PathVariable("id") int id)`

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- will be added after the ? in the URL

`http://localhost:8081/springrest/guests/query?name=John`

- pass to method via:

```
public Guest returnGuestByName(@RequestParam("name") String name) {  
    return receptionService.findGuest(name);  
}
```

- parameter is required, unless a defaultValue is specified

```
@GetMapping(path = "/query")  
public Guest returnGuestByName  
    (@RequestParam(value="name", defaultValue="An") String name) {  
    return receptionService.findGuest(name);  
}
```

URI:

`http://localhost:8081/springrest/guests/query`

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

@PathVariable vs. @RequestParam - When to use which?

General remarks

- no official standard rules, but good practices
- @PathVariable and @RequestParam can be used together

@PathVariable

- to link to a specific resource
- to denote hierarchy
- links can be easily bookmarked and cached
- parameters are mandatory

@RequestParam

- for filtering, pagination
- for none-hierarchical data

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- indicates a method parameter should be bound to the body of the request
- maps the `HttpRequest` body to an object, enabling automatic deserialization of the inbound `HttpRequest` body onto a Java object
- Spring automatically deserializes the JSON into a Java type assuming an appropriate one is specified
- by default, the type we annotate with the `@RequestBody` annotation must correspond to the JSON sent from our client-side
- other media-types can be specified as well via the `consumes` attribute on `@RequestMapping`

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

@ResponseBody

4.1

- tells a controller that the object returned is automatically serialized into JSON and passed back into the `HttpResponse` object
- could be used on top of a controller method
- example

```
@GetMapping("{id}")
@ResponseBody
public Guest returnGuestById(@PathVariable("id") int id) {
    return receptionService.findGuest(id);
}
```
- actually not needed any more since

```
@RestController = @Controller + @ResponseBody
```
- other media-types can be specified as well via the `produces` attribute on `@RequestMapping`

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

3 techniques to modify JSON outcome

- annotations
- adding wrapper elements for lists
- programmatic mapping with ObjectMapper class

Based on Jackson library, which is automatically included in spring-boot-starter-web

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Library: `com.fasterxml.jackson.annotation`

Different categories, depending on the goal of the annotation

- **property naming and inclusion**
`@JsonProperty`, `@JsonIgnore`, `@JsonIgnoreProperties`
- **deserialization and serialization details**
`@JsonFormat`, `@JsonUnwrapped`, `@JsonPropertyOrder`,
`@JsonRootName`
- ...

Placed on class or property level

More info, see:

<https://github.com/FasterXML/jackson-annotations/wiki/Jackson-Annotations>

<https://www.javadoc.io/doc/com.fasterxml.jackson.core/jackson-annotations>

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Jackson Annotations - Example

```
@JsonRootName("guest")
@JsonPropertyOrder({"id","name","enrolDate"})
@JsonIgnoreProperties(ignoreUnknown=true)
public class Guest {
    private int id;
    @JsonProperty("first")
    private String name;
    @JsonIgnore
    private int age;
    @JsonFormat(pattern="dd/MM/yyyy")
    private LocalDate enrolDate=LocalDate.now();
    public Guest(){}
    public Guest(int id, String name, int age) {
        this.id=id;
        this.name = name;
        this.age=age;
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
    public LocalDate getEnrolDate() { return enrolDate; }
    public void setEnrolDate(LocalDate enrolDate) { this.enrolDate = enrolDate; }
}
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

JSON list does not get an associated wrapper element by default

Solution:

- create an extra class that contains a list of the object wanted
- annotate the class with `@JsonRootName`
- add some (de)serialization variables in `application.properties`
- return the wrapper object from the `RestController`

This technique also needs to be applied when using JAXB

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Example

Guests.java

```
@JsonRootName("guests")
public class Guests {
    private List<Guest> guests;
    public Guests() {}
    public Guests(List<Guest> guests) { this.guests = guests; }
    public List<Guest> getGuests() { return guests; }
    public void setGuests(List<Guest> guests) { this.guests = guests; }
}
```

application.properties

```
spring.jackson.serialization.WRAP_ROOT_VALUE=true
spring.jackson.deserialization.UNWRAP_ROOT_VALUE=true
spring.jackson.serialization.indent-output=true
```

RestReceptionController.java

```
@GetMapping(path="all")
public Guests returnAllGuestsViaWrapper(){
    return new Guests(receptionService.findAllGuests());
}
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Library: `com.fasterxml.jackson.databind`

Used to (de)serialize JSON String to/from Object programmatically

Methods:

- `readValue()`
- `writeValue()`, `writeValueAsString()`
- `configure()` -> adds extra (de)serialization options

Also possible to read into a list using
`new TypeReference<List<Guest>>(){}`

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

ObjectMapper - Example

```
@JsonRootName("person")
@JsonPropertyOrder({"id","name","enrolDate"})
@JsonIgnoreProperties(ignoreUnknown=true)
public class Guest {
    private int id;
    @JsonProperty("first name")
    private String name;
    @JsonIgnore
    private int age;
    @JsonFormat(pattern="dd/MM/yyyy")
    private LocalDate enrolDate=LocalDate.now();
    public Guest(){ }
    public Guest(int id, String name, int age) {
        this.setId(id);
        this.name = name;
        this.age=age;
    }
    //getters and setters
}
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

ObjectMapper - Example (cont.)

```
@RunWith(SpringRunner.class)
@SpringBootTest
public class Ch3ObjectMapperApplicationTests {

    @Autowired AbisReceptionService serv;

    @Test
    public void testMapperRead() {
        String jsonString =
            "{\"person\" : {\"id\" : 5,\"first name\" : \"John\", \"enrolDate\" : \"11/06/2019\"}}";

        ObjectMapper om = new ObjectMapper();
        om.configure(DeserializationFeature.UNWRAP_ROOT_VALUE, true);
        try {
            Guest g = om.readValue(jsonString, Guest.class);
            assertEquals("John", g.getName());
        } catch (IOException e) { e.printStackTrace(); }
    }

    @Test
    public void testMapperWrite() {
        Guest g = serv.findGuest(1);
        ObjectMapper om = new ObjectMapper();
        om.configure(SerializationFeature.WRAP_ROOT_VALUE, true);
        try {
            String s = om.writeValueAsString(g);
            System.out.println(s);
        } catch (JsonProcessingException e) { e.printStackTrace(); }
    }
}
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- possible to specify which media types a resource can produce or consume by using the produces and consumes attributes on the @...Mapping annotations
- consumes mostly linked with @PostMapping and @PutMapping
- produces and consumes can also be placed at class level
- method-level attributes override the class-level attributes
- Maven dependency:

```
<dependency>  
    <groupId>com.fasterxml.jackson.dataformat</groupId>  
    <artifactId>jackson-dataformat-xml</artifactId>  
</dependency>
```
- will use the Jackson mappings added before, but can give “strange” results
- extra mappings can be added to map java objects to the correct representation using JAXB

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

- **Java Architecture for XML binding**
- **conversion of XML to and from Java types**
- **marshal/unmarshal XML content into/from Java representation**
- **access, update and validate Java representation against schema constraint**
- **javax.xml.bind package**
- **uses annotations to map Objects to XML elements**
 - **@XmlRootElement**
 - **@XmlAccessorType**
 - **@XmlElement**
 - **@XmlTransient**
 - ...
- **automatically supported in Spring REST**

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

JAXB Jackson Mapping

Jackson library adds annotations on top of JAXB

- **@JacksonXmlRootElement**
- **@JacksonXmlProperty**
- **@JacksonXmlWrapper**
- ...

JSON annotations are also supported for XML

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

JAXB Mapping - Example

```
<?xml version="1.0" encoding="UTF-8"?>
<persons>
  <person>
    <firstname>Maria</firstname>
    <lastname>Meuris</lastname>
  </person>
  <person>
    <firstname>Bert</firstname>
    <lastname>Mak</lastname>
  </person>
  <person>
    <firstname>Peter</firstname>
    <lastname>Buenk</lastname>
  </person>
</persons>
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

JAXB Mapping - Example (cont.)

@JacksonXmlRootElement(localName="persons")

public class Persons {

@JacksonXmlProperty(localName="person")

@JacksonXmlElementWrapper(useWrapping = false)

private ArrayList<Person> persons= new ArrayList<>();

public ArrayList<Person> getPersons() {return persons;}

public void setPersons(ArrayList<Person> persons) {this.persons = persons;}

}

@JacksonXmlRootElement(localName="person")

public class Person {

private String firstName;

@JacksonXmlProperty(localName="lastname")

private String lastName;

public String getFirstName() {return firstName;}

public void setFirstName(String firstName) {this.firstName = firstName;}

public String getLastName() {return lastName;}

public void setLastName(String lastName) {this.lastName = lastName;}

public String toString() {return firstName+ " " + lastName;}

}

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Producing and Consuming XML - Example

- **Guest.java with Jackson JAXB annotations**

```
@JacksonXmlRootElement(localName="guest")
public class Guest {
    private int id;
    private String name;
    private int age;
    public Guest(){}
    public Guest(int id, String name, int age) {
        this.setId(id);
        this.name = name;
        this.age=age;
    }
    public int getId() {return id;}
    public void setId(int id) {this.id = id;}
    public String getName() {return name;}
    public void setName(String name) {this.name = name;}
    public int getAge() {return age;}
    public void setAge(int age) {this.age = age;}
}
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Producing and Consuming XML - Example (cont.)

- **Controller**

```
@RestController
@RequestMapping(path="/guests", produces="application/json" )
public class RestReceptionController {
    @Autowired ReceptionService receptionService;

    @GetMapping(path = "/query")
    public Guest returnGuestByName(
        @RequestParam(value="name", defaultValue="An") String name) {
        return receptionService.findGuest(name);
    }

    @GetMapping(path = "{id}", produces=MediaType.APPLICATION_XML_VALUE)
    public Guest returnGuestById(@PathVariable("id") int id) {
        return receptionService.findGuest(id);
    }

    @PostMapping(path="", consumes=MediaType.APPLICATION_XML_VALUE)
    public void addGuest(@RequestBody Guest guest){
        receptionService.addGuest(guest);
    }
}
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Producing and Consuming XML - Example (cont.)

- **Test URIs**

http://localhost:8081/springrest/guests/query?name=John (will return JSON)

http://localhost:8081/springrest/guests/1 (will return XML)

can use accept header to state which media-type to return

POST: http://localhost:8081/springrest/guests

xml entered :

```
<guest>
  <id>3</id>
  <name>James</name>
  <age>25</age>
</guest>
```

More REST API functionality

1. Defining RESTful Resources
2. Mapping the HTTP Request Methods
3. Accessing Request Data
4. Returning Responses
5. More JSON Mapping
6. Producing and Consuming XML

Creating REST Clients

Objectives :

- **Using RestTemplate**
- **Getting, Posting, Updating and Deleting data via the client**

- **client is not required to run on a server (but probably will if you attach Thymeleaf/HTML/JavaScript/Angular/...)**
- **Maven Dependencies:**

```
<dependencies>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-test</artifactId>  
    <scope>test</scope>  
  </dependency>  
</dependencies>
```
- **“same” model (including mappings) as in REST API will have to be present in the client**

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

- **org.springframework.web.client.RestTemplate**
- **designed to call REST API**
- **main methods are closely tied to REST's underpinnings, which are the HTTP protocol's methods: HEAD, GET, POST, PUT, DELETE**
- **defines 36 methods for interacting with REST resources (of which 11 unique operations)**
- **e.g.: getObject(), postForObject(), delete(), exchange()**
- **wire RestTemplate in Controller (in case client is a webclient), Service class or JUnit Test**
- **configure in @SpringBootApplication configuration class:**
@Bean
public RestTemplate restTemplate(RestTemplateBuilder builder) {
 return builder.build();
}
- **methods linked to URI of the API, baseUrl passed as String:**
String baseUrl="http://localhost:8081/springrestxml/guests";

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

RestTemplate - Example

- **configuration**

```
@SpringBootApplication
public class SpringRestTemplateClientApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringRestTemplateClientApplication.class, args);
    }

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder.build();
    }
}
```

- **wiring into JUnit**

```
@SpringBootTest
public class SpringRestTemplateClientApplicationTests {
    @Autowired
    private RestTemplate rt;
    private String baseUrl="http://localhost:8081/springrestxml/guests";

    @Test
    public void contextLoads() {}
}
```

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

Basic retrieval of data with `getForObject()` method

3.1

- **parameters:**
 - **URI of the request (based on `baseUrl`)**
 - **class of Object to be returned**
- **model classes corresponding to XML or JSON sent by API needed on the classpath**
- **`getForObject` will then deserialize received JSON/XML into Java**

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

getForObject() - Examples

GET a specific person (will use JSON behind the scenes)

```
@Test
public void testFindGuestById(){
    int id=2;
    Guest g = rt.getForObject(baseUrl+"/"+id, Guest.class);
    System.out.println(g);
    assertEquals("An",g.getName());
}
```

GET all persons (will use XML behind the scenes)

```
@Test
public void testFindAllGuests(){
    Guests g = rt.getForObject(baseUrl, Guests.class);
    ArrayList<Guest> list = g.getGuests();
    System.out.println(list);
    assertEquals(2,list.size());
}
```

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

- **needed when**
 - **passing request parameters**
 - **passing header parameters**
(e.g when you want to specify media-type to return)
- **method parameters**
 - **UriComponentsBuilder: pass URI + request parameters**
 - **HttpMethod**
 - **HttpEntity, which needs a HttpHeaders object in its constructor**
 - **class of Object to be returned**

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

exchange() - Example

GET guests by name

```
@Test
public void testFindGuestByName(){
    String name = "John";
    UriComponentsBuilder uriBuilder =
        UriComponentsBuilder.fromHttpUrl(baseUrl+"/query")
            .queryParams("name", name);

    HttpHeaders requestHeaders = new HttpHeaders();
    requestHeaders.add("accept", MediaType.APPLICATION_JSON_VALUE);
    HttpEntity<Guest> requestEntity = new HttpEntity<>(requestHeaders);
    ResponseEntity<Guest> g =
        rt.exchange(uriBuilder.toUriString(), HttpMethod.GET, requestEntity, Guest.class);
    Guest guest=g.getBody();
    System.out.println(guest);
    assertEquals("John",guest.getName());
}
```

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

- **create a new instance of the type you want to post (could be retrieved via a web form)**
- **call the `postForObject()` method on the `RestTemplate` with parameters:**
 - the URI
 - the object you want to pass
 - the type of object returned (or `Void.class`)
- **the posted object will be transformed to the requested media-type (possibly using JAXB)**
- **return = object created or void**
- **possible to send an `HttpEntity` (which wraps object + headers) instead of the object itself**

Creating REST Clients

1. REST Client Project Settings
2. `RestTemplate`
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

Posting Data - Examples

Create a new person which will be send to the API to be stored

```
@Test
public void testPost(){
    Guest g = new Guest(3,"James",25);
    rt.postForObject(baseUrl, g, Void.class);
    this.testFindAllGuests();
}
```

Create a new person which will be send to the API to be stored, adding headers

```
@Test
public void testPostHeaders(){
    Guest g = new Guest(4,"Jill",27);
    HttpHeaders requestHeaders = new HttpHeaders();
    requestHeaders.add("content-type", MediaType.APPLICATION_JSON_VALUE);
    HttpEntity<Guest> requestEntity = new HttpEntity<>(g,requestHeaders);
    rt.postForObject(baseUrl, requestEntity, Void.class);
    this.testFindAllGuests();
}
```

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

- change the instance you want to update (via setter methods,...)
- call the **put()** method on the RestTemplate with parameters:
 - the URI
 - the object you want to pass
- the object will be transformed to the requested media-type (possibly using JAXB)
- return = void
- use the **exchange()** method if you want to pass headers, parameters,...

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

Updating Data - Example

Change the name of Guest 2 to “xxxxx”

```
@Test
public void testPut(){
    Guest g = new Guest(2,"xxxxx",32);
    rt.put(baseUrl+"/"+g.getId(), g);
    this.testFindGuestById();
}
```

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

- **call delete() on the restTemplate**
- **only parameter to pass: URI of the object to delete**
- **return = void**
- **example**

```
@Test
public void testDelete(){
    rt.delete(baseUrl+"/2");
    this.testFindAllGuests();
}
```

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

Creating REST Clients

1. REST Client Project Settings
2. RestTemplate
3. Getting Data
4. Posting Data
5. Updating Data
6. Deleting Data

Exception Handling and Validation

Objectives :

- **HTTP status codes**
- **ResponseEntity class**
- **Exception Handling**
- **Validation**

Not Java of course but necessary to understand for

- exception handling in our services/clients
- notifying the client when a post/put/delete was processed
- debugging

Different categories

- **1xx (informational):** request was received, continuing process
- **2xx (successful):** request was successfully received, understood, and accepted
- **3xx (redirection):** further action needs to be taken in order to complete the request
- **4xx (client error):** request contains bad syntax or cannot be fulfilled
- **5xx (server error):** server failed to fulfil an apparently valid request

200: OK

- standard response for successful HTTP requests
- actual response will depend on the request method used
- in a GET request, response will contain an entity corresponding to the requested resource
- in a POST request, the response will contain an entity describing or containing the result of the action

201: Created

- request has been fulfilled, resulting in creation of a new resource

202: Accepted

- request accepted for processing, but processing has not been completed
- request might or might not be eventually acted upon, and may be disallowed when processing occurs

HTTP Status Codes - 2xx Success (cont.)

203: Non-Authoritative Information

- server is a transforming proxy that received a 200 OK from its origin, but is returning a modified version of the origin's response

204: No Content

- request successfully processed, but no content is returned

205: Reset Content

- request successfully processed, but no content is returned
- unlike a 204 response, this response requires that the requester resets the document view

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

HTTP Status Codes - 2xx Success (cont.)

206: Partial Content

- only part of the resource delivered due to range header sent by the client
- range header is used by HTTP clients to enable resuming of interrupted downloads, or split a download into multiple simultaneous streams

207: Multi-Status

- message body that follows is by default an XML message and can contain a number of separate response codes, depending on how many sub-requests were made

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

400: Bad Request

- **server cannot or will not process the request due to an apparent client error (e.g., malformed request syntax, size too large)**

401: Unauthorized

- **similar to 403 Forbidden, but specifically for use when authentication is required and has failed or has not yet been provided**
- **response must include a WWW-Authenticate header field containing a challenge applicable to the requested resource**
- **semantically means "unauthenticated",i.e. the user does not have the necessary credentials**

402: Payment Required (reserved for future use)

- **original intention was that it might be used as part of some form of digital cash or micro payment scheme**
- **Google Developers API uses this status if a particular developer has exceeded the daily limit on requests**

HTTP Status Codes - 4xx: Client Errors (cont.)

403: Forbidden

- request was valid, but the server is refusing action
- user might not have the necessary permissions for a resource, or may need an account of some sort

404: Not Found

- requested resource could not be found but may be available in the future
- subsequent requests by the client are permissible

405: Method Not Allowed

- request method is not supported for the requested resource
e.g. a GET request on a form that requires data to be presented via POST, or a PUT request on a read-only resource

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

HTTP Status Codes - 4xx: Client Errors (cont.)

406: Not Acceptable

- requested resource is capable of generating only content not acceptable according to the **Accept** headers sent in the request

407: Proxy Authentication Required

- client must first authenticate itself with the proxy

408: Request Timeout

- server timed out waiting for the request

409: Conflict

- request could not be processed because of conflict in the current state of the resource, such as an edit conflict between multiple simultaneous updates

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

HTTP Status Codes - 4xx: Client Errors (cont.)

410: Gone

- resource requested is no longer available and will not be available again
- should be used when a resource has been intentionally removed and the resource should be purged

411: Length Required

- request did not specify the length of its content, which is required by the requested resource

412: Precondition Failed

- server does not meet one of the preconditions that the requester put on the request

413: Payload Too Large

- request is larger than the server is willing or able to process

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

HTTP Status Codes - 4xx: Client Errors (cont.)

414: URI Too Long

- URI provided was too long for the server to process
- often result of too much data being encoded as a query-string

415: Unsupported Media Type

- request entity has a media type which the server or resource does not support

418: I'm a teapot

- defined in 1998 as one of the traditional IETF April Fools' jokes
- RFC specifies this code should be returned by teapots requested to brew coffee
- used as an Easter egg in some websites, including Google

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

HTTP Status Codes - 4xx: Client Errors (cont.)

422: Unprocessable Entity

- request was well-formed but was unable to be processed due to semantic errors

429: Too Many Requests

- user has sent too many requests in a given amount of time
- intended for use with rate-limiting schemes

431: Request Header Fields Too Large

- server can't process request because an individual header field, or all the header fields collectively, are too large

451: Unavailable For Legal Reasons

- server operator has received a legal demand to deny access to a resource or to a set of resources that includes the requested resource
- code 451 was chosen as a reference to the novel Fahrenheit 451

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

500: Internal Server Error

- **generic error message, given when an unexpected condition was encountered and no more specific message is suitable**

501: Not Implemented

- **server either does not recognize the request method, or it lacks the ability to fulfil the request**
- **usually this implies future availability (e.g. new feature of a web-service API)**

502: Bad Gateway

- **server was acting as a gateway or proxy and received an invalid response from the upstream server**

503: Service Unavailable

- **server is currently unavailable (because it is overloaded or down for maintenance)**
- **generally, this is a temporary state**

HTTP Status Codes - 5xx: Server Errors

504: Gateway Timeout

- server was acting as a gateway or proxy and did not receive a timely response from the upstream server

505: HTTP Version Not Supported

- server does not support the HTTP version used in the request

507: Insufficient Storage

- server is unable to store the representation needed to complete the request

508: Loop Detected

- server detected an infinite loop while processing the request

511: Network Authentication Required

- client needs to authenticate to gain network access
- intended for use by intercepting proxies used

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

- **represents the whole HTTP response: status code, headers, and body**
- **can use it to fully configure the HTTP response**
- **ResponseEntity is a generic type, so we can use any type as the response body (even void)**
- **also possible to use builder to create the ResponseEntity object**
- **can be used for the translation into info messages and exceptions by the client**
- **used in RestTemplate as well @RestController methods**
- **returned from getForEntity(), postForEntity() and exchange() in the client**
- **most important methods:**
 - **getBody()**
 - **getStatusCode()**

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

ResponseEntity - Example

RestReceptionController.java (API)

```
@GetMapping(path = "{id}")
public ResponseEntity<Guest> returnGuestById(@PathVariable("id") int id) {
    Guest g = receptionService.findGuest(id);
    ResponseEntity<Guest> re = new ResponseEntity<Guest>(g,HttpStatus.OK);
    return re;
}

@GetMapping(path="", produces=MediaType.APPLICATION_XML_VALUE)
public ResponseEntity<?> returnall(){
    Guests g = new Guests();
    g.setGuests(receptionService.findAllGuests());
    HttpHeaders responseHeaders = new HttpHeaders();
    responseHeaders.add("content-type", MediaType.APPLICATION_XML_VALUE);
    return new ResponseEntity<Guests>(g,responseHeaders,HttpStatus.OK);
}

@PostMapping(path="", consumes=MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> addGuest(@RequestBody Guest guest){
    receptionService.addGuest(guest);
    return ResponseEntity.noContent().build();
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

ResponseEntity - Example (cont.)

JUnit Test (Client)

```
@Test
public void testFindGuestById(){
    ResponseEntity<Guest> g = rt.getForEntity(baseUrl+ "/2", Guest.class);
    System.out.println(g.getStatusCode());
    Guest guest = g.getBody();
    System.out.println(guest);
    assertEquals("An",guest.getName());
}

@Test
public void testFindAllGuests(){
    HttpHeaders requestHeaders = new HttpHeaders();
    requestHeaders.add("accept", MediaType.APPLICATION_XML_VALUE);
    HttpEntity<Guest> requestEntity = new HttpEntity<>(requestHeaders);
    ResponseEntity<Guests> guests =
        rt.exchange(baseUrl,HttpMethod.GET,requestEntity, Guests.class);
    ArrayList<Guest> list = guests.getBody().getGuests();
    System.out.println(list);
    assertEquals(2,list.size());
}

@Test
public void testPost(){
    Guest g = new Guest(3,"James",25);
    ResponseEntity<Void> resp = rt.postForEntity(baseUrl, g, Void.class);
    System.out.println("status: " + resp.getStatusCode() + ", body: " + resp.getBody());
    this.testFindAllGuests();
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

- **when no result found, empty page will be returned and response status is still 200 (OK)**
- **status should be 404 in fact, and response body should carry an error message**
- **Spring offers some options for dealing with exceptions**
 - **creation of Error objects that can carry the error messages**
 - **use ResponseEntity that carries Error object and HttpStatus**
 - **create a centralized exception handler (@RestControllerAdvice)**

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

- **not only pass HTTP status code when something goes wrong**
- **use object to pass more information about the error**
- **client can then interpret the message**
- **RFC 7807 provides a standard format for returning problem details from HTTP APIs**
 - **error responses MUST use standard HTTP status codes in the 400 or 500 range to detail the general category of error**
 - **error responses will be of the content-type application/problem, appending a serialization format of either json or xml: application/problem+json, application/problem+xml**
 - **error responses will have each of the following keys:**
 - title (String): short, human-readable title for the general error
 - statuscode (int): HTTP status code of the error
 - detail (String): human-readable description of the specific error
 - type (String, optional): URL to a document describing the error
 - instance(String, optional): URI pointing to error log for that specific response

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Error Object - Example

```
public class ApiError {
    private String title;
    private int status;
    private String description;
    private String type = "about:blank";
    private String instance = "";
    public ApiError(){}
    public ApiError(String title, int status, String description) {
        this.title = title;
        this.status = status;
        this.description = description;
    }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public int getStatus() { return status; }
    public void setStatus(int status) { this.status = status; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public String getInstance() { return instance; }
    public void setInstance(String instance) { this.instance = instance; }
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

- **ResponseEntity** can be used to pass a **HttpStatus.NOT_FOUND** (404) (or another code) when some exception occurs
- **ApiError** passed as the entity, client checks on the 404
- add response header with content-type **MediaType.APPLICATION_PROBLEM_JSON_VALUE**
- also possible to pass XML of course
- **RestTemplate** will throw **HttpStatusCodeException** when 4xx or 5xx status code passed
- error body read as **String**, need to parse it yourself into object via **ObjectMapper**
- approach works, but:
 - all resource methods will have to return **ResponseEntity**
 - need to transform all your own exceptions to **ResponseEntity** with error object and status code
 - error body needs to be parsed from **String** -> **ApiError**

1. HTTP Status Codes
2. The **ResponseEntity** class
3. Exception Handling
4. Validation

Exception Handling with ResponseEntity - API Example

GuestNotFoundException.java

```
public class GuestNotFoundException extends Exception {  
    public GuestNotFoundException(String message) {  
        super(message);  
    }  
}
```

AbisReceptionService.java

```
@Service  
public class AbisReceptionService implements ReceptionService {  
    private ArrayList<Guest> guests = new ArrayList<>();  
    public AbisReceptionService() {  
        guests.add(new Guest(1,"John",35));  
        guests.add(new Guest(2,"An",42));  
    }  
    @Override  
    public Guest findGuest(int id) throws GuestNotFoundException {  
        return guests.stream().filter(g->g.getId()==id).findFirst()  
            .orElseThrow(() -> new GuestNotFoundException  
                ("guest with id " + id + " could not be found"));  
    }  
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Exception Handling with ResponseEntity - API Example (cont.)

RestReceptionController.java

```
@RestController
@RequestMapping(path="guests", produces="application/json" )
public class RestReceptionController {
    @Autowired ReceptionService receptionService;
    @GetMapping("{id}")
    public ResponseEntity<? extends Object> returnGuest(@PathVariable int id) {
        try {
            Guest g = receptionService.findGuest(id);
            return new ResponseEntity<Guest>(g,HttpStatus.OK);
        } catch (GuestNotFoundException e) {
            HttpStatus status = HttpStatus.NOT_FOUND;
            ApiError err = new ApiError("guest not found", status.value(),e.getMessage());
            HttpHeaders responseHeaders = new HttpHeaders();
            responseHeaders.add("content-type",
                               MediaType.APPLICATION_PROBLEM_JSON_VALUE);
            return new ResponseEntity<ApiError>(err,responseHeaders,status);
        }
    }
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Exception Handling with ResponseEntity - Client Example

JUnit Test

```
@SpringBootTest
public class Ch5ResponseEntityExceptionClientApplicationTests {

    @Autowired private RestTemplate rt;
    private String baseUrl = "http://localhost:8081/springresponseentity/guests";

    @Test
    public void testFindGuestById() throws JsonParseException,
                                           JsonMappingException, IOException {

        ResponseEntity<? extends Object> re=null;
        try {
            re = rt.getForEntity(baseUrl+ "/3", Guest.class);
            Guest guest = (Guest) re.getBody();
            System.out.println(guest);
            assertEquals("An", guest.getName());
        } catch (HttpStatusCodeException e) {
            if (HttpStatus.NOT_FOUND == e.getStatusCode()) {
                String serr = e.getResponseBodyAsString();
                System.out.println(serr);
                ApiError ae=new ObjectMapper().readValue(serr,ApiError.class);
                System.out.println(ae.getDescription());
            } else {
                System.out.println("some other error occurred");
            }
        }
    }
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

- possible to centralize exception handling based on AOP advice
- create a class annotated with `@RestControllerAdvice`
- make class extend `ResponseBodyExceptionHandler`, to already treat the most common HTTP Exceptions (e.g. `HttpRequestMethodNotSupportedException`, `HttpMediaTypeNotSupportedException`)
- implement a method with annotation `@ExceptionHandler`, which contains the code you previously wrote in the controller
- client stays the same of course

1. HTTP Status Codes
2. The `ResponseBody` class
3. Exception Handling
4. Validation

Centralizing Exception Handling - Example

RestResponseEntityExceptionHandler.java

```
@RestControllerAdvice
public class RestResponseEntityExceptionHandler
    extends ResponseEntityExceptionHandler {

    @ExceptionHandler(value = GuestNotFoundException.class)
    protected ResponseEntity<? extends Object> handleGuestNotFound
        (GuestNotFoundException gnfe, WebRequest request) {

        HttpStatus status = HttpStatus.NOT_FOUND;
        ApiError err = new ApiError("guest not found", status.value(), gnfe.getMessage());
        HttpHeaders responseHeaders = new HttpHeaders();
        responseHeaders.add("content-type",
            MediaType.APPLICATION_PROBLEM_JSON_VALUE);
        return new ResponseEntity<ApiError>(err, responseHeaders, status);
    }
}
```

RestReceptionController.java

```
@GetMapping("{id}")
public Guest returnGuestById(@PathVariable("id") int id)
    throws GuestNotFoundException {

    return receptionService.findGuest(id);
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

- **syntax validation can be done via the JSR-303 Bean Validation standard, add spring-boot-starter-validation to the pom.xml**
- **uses annotations on the model classes**
 - e.g. **@NotNull, @NotEmpty, @Size, @Min, @Pattern, @Email, ...**
- **error messages can be added**
- **mostly used for validating POST and PUT requests**
- **use @Valid on the @RequestBody argument to trigger validation**
- **RFC 7807**
 - **create a class ValidationError which contains name and reason**
 - **add a list (called invalid-params) of ValidationError objects to the ApiError class**
- **handling of validation can be added to your ExceptionHandler class**
- **validation errors will throw MethodArgumentNotValidException**
- **ResponseEntityExceptionHandler already contains method handleMethodArgumentNotValid(), which you have to override**

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Validation API - Example

Guest.java

```
public class Guest {  
    private int id;  
    @NotEmpty(message="name must be filled in")  
    private String name;  
    @Min(value=2, message="age must be bigger than 2")  
    private int age;  
    public Guest(){}  
    public Guest(int id, String name, int age) {  
        this.setId(id);  
        this.name = name;  
        this.age=age;  
    }  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public int getAge() { return age; }  
    public void setAge(int age) { this.age = age; }  
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Validation API - Example (cont.)

ValidationError.java

```
public class ValidationError {  
    private String name;  
    private String reason;  
    public ValidationError(){}  
    public ValidationError(String name, String reason) {  
        this.name = name;  
        this.reason = reason;  
    }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public String getReason() { return reason; }  
    public void setReason(String reason) { this.reason = reason; }  
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Validation API - Example (cont.)

ApiError.java

```
public class ApiError {
    private String title;
    private int status;
    private String description;
    private String type = "about:blank";
    private String instance = "";
    @JsonProperty("invalid-params")
    private List<ValidationError> invalidParams = new ArrayList<ValidationError>();
    public ApiError(){}
    public ApiError(String title, int status, String description) {
        this.title = title;
        this.status = status;
        this.description = description;
    }
    // getters and setters
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Validation API - Example (cont.)

RestReceptionController.java

```
@RestController
@RequestMapping(path="guests", produces="application/json" )
public class RestReceptionController {
    @Autowired
    ReceptionService receptionService;
    @PostMapping(path="", consumes=MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<Void> addGuest(@Valid @RequestBody Guest guest){
        receptionService.addGuest(guest);
    }
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Validation API - Example (cont.)

RestResponseEntityExceptionHandler.java

@RestControllerAdvice

```
public class RestResponseEntityExceptionHandler
    extends ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {
        ApiError err = new ApiError("invalid arguments", status.value(), ex.getMessage());
        List<FieldError> fieldErrors = ex.getBindingResult().getFieldErrors();
        List<ValidationError> validationErrorList = err.getInvalidParams();
        for (FieldError fe : fieldErrors) {
            ValidationError validationError = new ValidationError();
            validationError.setName(fe.getField());
            validationError.setReason(fe.getDefaultMessage());
            validationErrorList.add(validationError);
        }
        return new ResponseEntity<Object>(err, headers, status);
    }
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Validation Client - Example

JUnit Test

```
@Test
public void testPostValidation()
    throws JsonParseException, JsonMappingException, IOException {
    Guest g = new Guest(3, "", -25);
    ResponseEntity<? extends Object> re = null;
    try {
        re = rt.postForEntity(baseUrl, g, Object.class);
        System.out.println("post succeeded, code " + re.getStatusCodeValue());
    } catch (HttpStatusCodeException e) {
        if (HttpStatus.BAD_REQUEST == e.getStatusCode()) {
            String serr = e.getResponseBodyAsString();
            ApiError ae = new ObjectMapper().readValue(serr, ApiError.class);
            List<ValidationError> ve = ae.getInvalidParams();
            if (ve.size() != 0) {
                System.out.println("validation errors:");
                ve.stream().map(v -> v.getReason()).forEach(System.out::println);
            }
        } else {
            System.out.println("some other error occurred");
        }
    }
}
```

Exception Handling and Validation

1. HTTP Status Codes
2. The ResponseEntity class
3. Exception Handling
4. Validation

Additional Topics

Objectives :

- **Swagger for documenting REST APIs**
- **Circuit Breakers**

Additional Topics

1. Swagger for documenting REST APIs
2. Circuit Breakers

Additional Topics

1. Swagger for documenting REST APIs
2. Circuit Breakers

Additional Topics

1. Swagger for documenting REST APIs
2. Circuit Breakers

Introduction to ORM and (Spring) JPA

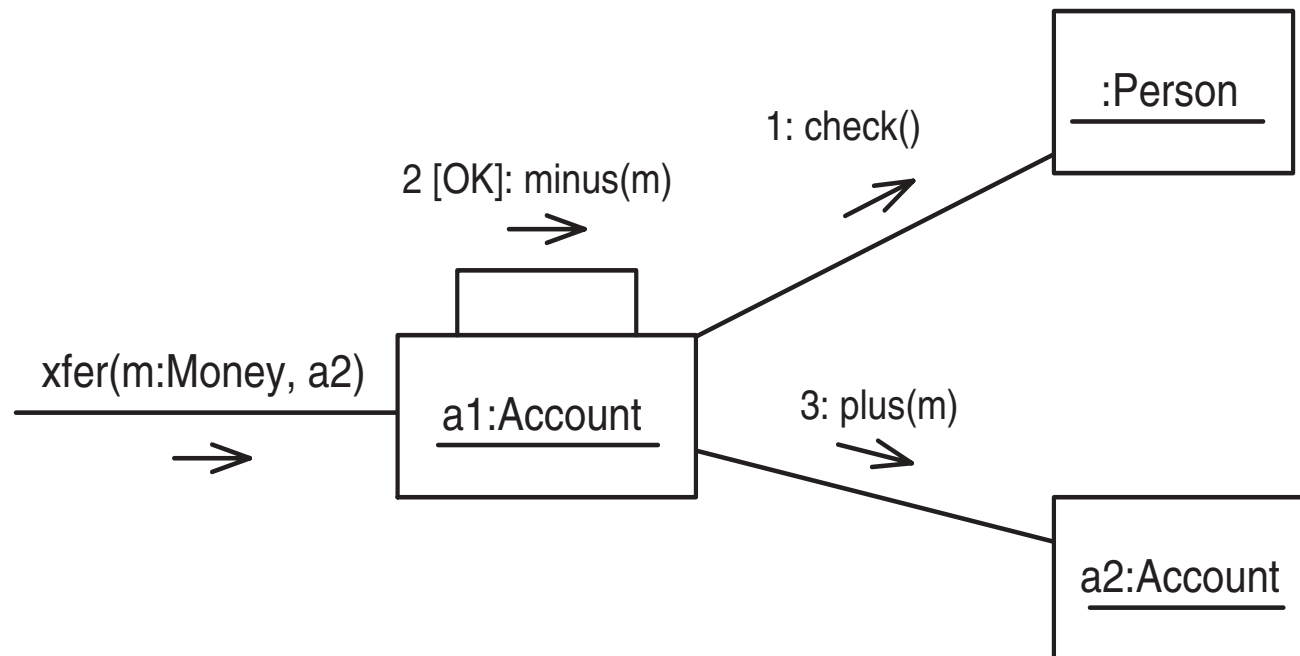
Objectives :

- **Persisting objects**
- **The object-relational mismatch**
- **Features of persistency mechanisms**
- **(Spring) JPA**

OO-applications are composed of objects which

- consist of **data and behaviour**
- are connected to each other
- send messages to each other

Example: transfer money between accounts



Object life cycle

- **objects are described in classes (domain model)**
- **classes are instantiated at runtime**
- **instances are populated with data**
- **these data must be preserved i.e. persisted/saved**

Introduction to ORM and (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

O/R mismatch

- **granularity**
 - object attributes
- **mapping of inheritance trees**
 - inheritance is (mostly) not supported in RDBMSs
 - map tree to table(s)
 - polymorphic queries
- **identification of entities**
 - in OO: not directly supported
 - in RDBMS: through primary keys (PK)

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

Object - relational mapping (cont.)

- **relationship management between entities (associations)**
 - in OO: (navigational) links between objects
 - in RDBMS: PK - FK relations
 - **directional problem in 1-many (or many-many) relationship**
- **transaction management**
 - in application (application environment) or by data store?
 - **concurrency control**
- **synchronization**
 - **how to keep data in objects in sync with database and vice-versa**
- **performance**
 - **caching**
 - **lazy loading**

Introduction to ORM and (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

Object - relational mapping solutions

How to specify O/R mapping

- **hard coding of DB schema in Java**
- **meta-data definitions in XML**
- **annotations**

O/R mapping solutions (frameworks)

- **do it yourself: extremely difficult and cost intensive**
- **use existing solutions:**
 - **JPA - Java Persistence Architecture**
 - **Java/Jakarta EE: Entity EJB (based on JPA)**
 - **Spring Data JPA (based on Hibernate)**
 - **alternatives: Hibernate, OJB, EclipseLink, Cayenne, MyBatis, ...**

based on Unit Of Work Enterprise Application Architecture pattern

<https://java-source.net/open-source/persistence>

Introduction to ORM and (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

- **persist the information (data) in the object model, i.e.**
 - the data in the objects described as attributes in class model
 - links between objects described as relationships in class model
- **synchronization between application and data**
 - data in memory must be synchronized with data in data store
 - data in data store must be synchronized with data in memory
 - important if different applications access the same data
- **transactions**
 - set of actions that move data from one consistent state to another
 - key features: **Atomicity, Consistency, Isolation, Durability**
- **concurrency control**
 - different users/applications must reach same data at same time...
 - ...while keeping the data in a consistent state

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

Features of persistence mechanisms (cont.)

- **query mechanism**
 - **need for some mechanism to retrieve data in a selective way from the data store**
- **identity support**
 - **unique identification of object**
 - **avoid multiple copies of the same data**
- **security**
 - **unauthorized people must not see sensitive data**
- **performance optimisation**
 - **use of caching**

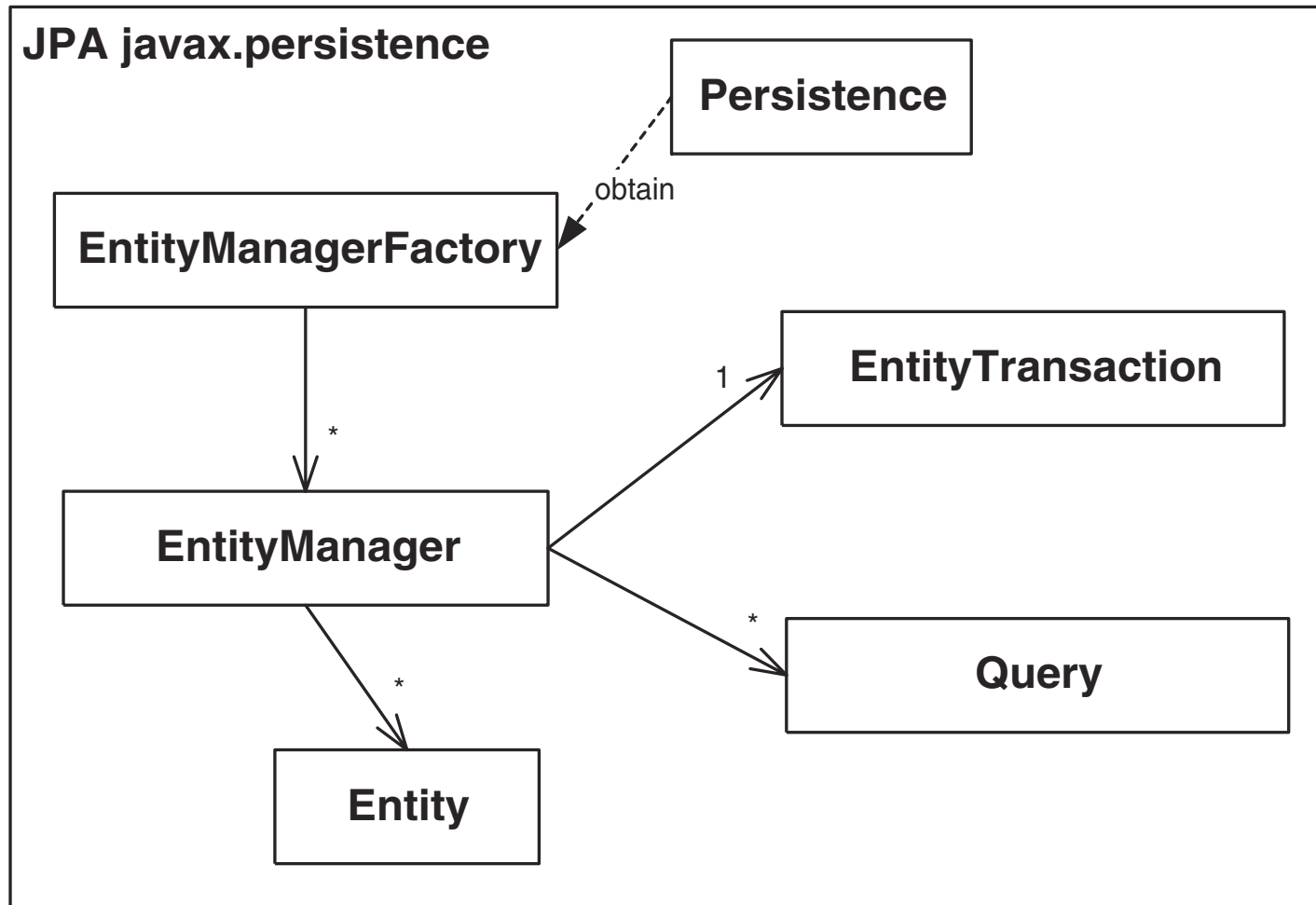
Introduction to ORM and (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

- **JPA: Java Persistence API (2006)**
 - **Java specification for accessing, persisting, and managing data between Java objects / classes and a RDBMS**
 - **defined as part of the EJB 3.0 specification as a replacement for the EJB 2 CMP Entity Beans specification**
 - **jakarta.persistence package (before javax.persistence)**
 - **specification, not implementation**
 - **implementations aka. JPA providers**
EclipseLink, OpenJPA, **Hibernate**, Data Nucleus
- **Hibernate**
 - **started in 2001 as open source project**
 - **later further developed under JBoss (Red Hat)**
 - **used SessionFactory as basic interface**
 - **became certified implementation of JPA since version 3.5 (2010)**
 - **now mainly uses EntityManager as “interface”**

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA



package jakarta.persistence

- **Persistence:** get EntityManagerFactory instances (vendor-neutral)
- **EntityManagerFactory:** factory for EntityManagers
- **EntityManager:** manages a set of persistent objects
acts also as factory for Query instances
- **Entity:** persistent object that represents datastore records
- **EntityTransaction:** (associated with 1 EntityManager) group operations on persistent data into consistent units of work
- **Query:** interface to find persistent objects that meet certain criteria
uses the Java Persistence Query Language (JPQL), the Criteria API and/or the native Structured Query Language (SQL)

Introduction to ORM and (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

- **Spring Data JPA**
 - **powerful repository and custom object-mapping abstractions**
 - **Query DSL:**
 - dynamic query derivation from repository method names
 - **implementation domain base classes providing basic properties**
 - **support for transparent auditing (created, last changed)**
 - **possibility to integrate custom repository code**
 - **easy configuration in Spring Boot**
 - **advanced integration with Spring MVC controllers**
 - **experimental support for cross-store persistence**

Introduction to ORM and (Spring) JPA

1. Persisting objects
2. Object - relational mapping (ORM)
3. Features of persistence mechanisms
4. (Spring) JPA

Configuration

Objectives :

- **Configuring data sources**

- **Maven dependency**

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

- **Spring Boot will do autoconfiguration**

- will try to use embedded database by default
- when own DataSource defined, this is disabled
- disable manually via:
@SpringBootApplication(exclude={DataSourceAutoConfiguration.class})

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

Configuring a data source

2

General

2.1

- **data source defines driver, database url, username and password**
- **central configuration makes it easy to change later on**
- **driver depends on the database, can be chosen when setting up Spring Boot Application for several supported DBs**

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

H2

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
</dependency>
```

Oracle

```
<dependency>  
  <groupId>com.oracle.database.jdbc</groupId>  
  <artifactId>ojdbc11</artifactId>  
</dependency>
```

PostgreSQL

```
<dependency>  
  <groupId>org.postgresql</groupId>  
  <artifactId>postgresql</artifactId>  
</dependency>
```

SQL Server

```
<dependency>  
  <groupId>com.microsoft.sqlserver</groupId>  
  <artifactId>mssql-jdbc</artifactId>  
</dependency>
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

Embedded Data Source

3.1

- **embedded database runs as part of your application instead of a separate database**
- **invisible to end-user**
- **perfect choice for development and test purposes**
- **test data is reset every time you restart your application**
- **pass script that is used to set up the DB**
- **products: Derby, H2,..**

- **configuration:**

```
@Bean
@Profile("development")
public DataSource testDataSource(){
    return new EmbeddedDatabaseBuilder()
        .setType(EmbeddedDatabaseType.H2)
        .addScript("classpath:h2.sql")
        .build();
}
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

- **simplest possible, but limited data source**
- **provided by Spring (`org.springframework.jdbc.datasource`)**
 - **DriverManagerDataSource**
new connection every time one is requested
 - **SingleConnectionDataSource**
returns same connection every time
- **DBMS providers also have implementations available (e.g. `OracleDataSource`)**
- **no pooling possibilities!**
- **for small applications or at development phase**

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

Basic Data Source - Configuration

- **application.properties**

```
spring.datasource.url=jdbc:oracle:thin:@//delphi:1521/TSTA
spring.datasource.username=tu00010
spring.datasource.password=tu00010
spring.datasource.driver-class-name = oracle.jdbc.OracleDriver
```

- **Java config (deprecated)**

@Bean

```
public DataSource dataSource() throws SQLException {
    OracleDataSource ds = new OracleDataSource();
    ds.setURL(env.getProperty("spring.datasource.url"));
    ds.setUser(env.getProperty("spring.datasource.username"));
    ds.setPassword(env.getProperty("spring.datasource.password"));
    ds.setDriverType(env.getProperty("spring.datasource.driver-class-name"));
    return ds;
}
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

- **directly configured in Spring**
- **provided by 3rd party (Apache Commons DBCP, c3po,Hikari,...)**
- **Hikari preferred (for Oracle/PostgreSQL/...), already provided in spring-boot-starter-data-jpa**
- **Maven dependencies for Apache Commons:**

- **Apache Commons DBCP**

```
<dependency>  
  <groupId>org.apache.commons</groupId>  
  <artifactId>commons-dbcp2</artifactId>  
</dependency>
```

RELIES ON:

- **Apache Commons Pool**

```
<dependency>  
  <groupId>org.apache.commons</groupId>  
  <artifactId>commons-pool2</artifactId>  
</dependency>
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

Pooled Data Source - Configuration

- **application.properties**

```
// basic connection settings like before
spring.datasource.hikari.minimumIdle=5
spring.datasource.hikari.maximumPoolSize=20
spring.datasource.hikari.idleTimeout=30000
spring.datasource.hikari.maxLifetime=2000000
spring.datasource.hikari.connectionTimeout=30000
```

- **Java Config (deprecated)**

```
@Bean
public DataSource prodDataSource() throws SQLException {
    HikariDataSource hds = new HikariDataSource();
    hds.setJdbcUrl(env.getProperty("spring.datasource.url"));
    hds.setUsername(env.getProperty("spring.datasource.username"));
    hds.setPassword(env.getProperty("spring.datasource.password"));
    hds.setDriverClassName
        (env.getProperty("spring.datasource.driver-class-name"));
    hds.setMinimumIdle
        (Integer.parseInt(env.getProperty("spring.datasource.hikari.minimumIdle")));
    hds.setMaximumPoolSize(Integer.parseInt
        (env.getProperty("spring.datasource.hikari.maximumPoolSize")));
    hds.setIdleTimeout
        (Long.parseLong(env.getProperty("spring.datasource.hikari.maxLifetime")));
    hds.setConnectionTimeout(Long.parseLong
        (env.getProperty("spring.datasource.hikari.connectionTimeout")));
    return hds;
}
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

- **best to use in production**
- **data source managed by server (WebSphere, Tomcat,...) and retrieved via JNDI**
- **configuration**
 - **application.properties**
`spring.datasource.jndi-name=java:comp/env/jdbc/MyJndiName`
 - **Java Config (deprecated)**

```
@Bean
public DataSource dataSource() throws NamingException {
    return (DataSource) new JndiTemplate().lookup
        (env.getProperty("java:comp/env/jdbc/MyJndiName"));
}
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

- **Spring Boot can autoconfigure datasources based on properties files**
- **don't need to configure them any more and load resources!**
- **create application-myprofilename.properties files when profiles used**
- **use pre configured properties of Spring Boot**
- **driver will be deduced from database url**
- **when nothing configured, automatically embedded db used based on definitions in pom.xml**
- **EntityManager will be used behind the scenes**

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

application.properties file can contain many extra properties

- **DB initialization: spring.jpa.hibernate.ddl-auto**
 - **create:** existing tables are first dropped, and then recreated
 - **create-drop:** tables are created, and dropped after application is finished
 - **validate:** validates whether the tables and columns exist; otherwise, it throws an exception
 - **update:** object model created based on the mappings is compared with the existing schema, schema is updated according to the diff
 - **none:** no DDL is being done

Spring Boot defaults the parameter value to create-drop if no schema manager has been detected, otherwise none for all other cases

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

Extra configuration options (cont.)

- **formatting/logging SQL**
 - **spring.jpa.show-sql = true**
 - **spring.jpa.properties.hibernate.format_sql = true**
- **standard logging**
 - **logging.level.org.hibernate.SQL=DEBUG**
 - **logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE**

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

- **DataSourceTest.java**

```
@SpringBootTest
public class DataSourceTest {

    @Autowired
    DataSource dataSource;

    @Test
    public void testConnectionViaDataSource() {
        try (Connection c = dataSource.getConnection()){
            System.out.println("connection succeeded via "
                               + c.getMetaData().getDatabaseProductName() + " . " );
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Configuration

1. Spring JPA: Basic Configuration
2. Configuring a data source
3. Data Source Types
4. Simplifications from Spring Boot
5. Extra configuration options
6. Testing The Connection

Basic OR Mapping and JPA repositories

Objectives :

- **Mapping of tables and properties**
- **Object identity**
- **JPA repositories and Query Methods**
- **Spring Data JUnit Testing**

What to map?

- map class to relational table
- map obj attribute / property to relational column + type conversion
- provide object identity
- map class association to relational FK
- inheritance/hierarchy mapping

Mapping is done using (JPA) annotations (since Hibernate 3)

using configuration by exception,
the number of annotations required to do simple mappings
can be very limited

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Use annotations on class level

- **@Entity** enables class to become persistent
- **@Table** associates with RDBMS table name (default = class name)

Example

@Entity

@Table(name="GUESTS")

```
public class Guest { }
```

Define schema

- **use schema attribute in table annotation**
@Table(name="GUESTS", schema="SPRINGJPA")
- **default schema in application.properties**
spring.jpa.properties.hibernate.default_schema = springjpa

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Distinction between

- **identical objects:** `a == b`
- **equal objects:** `a.equals(b)`
- **database identity:** same primary key (PK) in database

Distinction between

- **natural keys** (e.g. email)
- **synthetic or surrogate keys** (e.g. generated keys)

Criteria to choose a primary key

- **not null**
- **unique**
- **value never changes, immutable** (i.e. no `set()` method provided)

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Annotation @Id to

(placement is important for access type for all attributes)

- key field -> *field access type* (for all attributes!)
- get method (property) -> *property access type* (for all attributes!)

If no key generator is provided

- application is responsible for providing/defining a value for the PK
- define constructor with argument for PK, e.g.

```
public Guest(String name) { this.name=name; }
```

Unique key/identifier value can be generated:

specify strategy -> @GeneratedValue

- **AUTO**: selects generation strategy based on the DB specific dialect
- **IDENTITY**: use identity columns (if supported in RDBMS)
- **SEQUENCE**: sequential unique key
- **TABLE**: key based on value in separate table

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Examples

- **Auto and Identity**

```
@GeneratedValue(strategy = GenerationType.AUTO)
@Column(name="gno")
private int id;

@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="gno")
private int id;
```

- **Sequence generator**

```
@SequenceGenerator(name="mySeqGen", sequenceName="guestsequence",
                    allocationSize=1)
@Id @GeneratedValue (strategy=GenerationType.SEQUENCE,
                    generator="mySeqGen")

@Column(name="gno")
private Long id;
```

- **Table generator (**

```
@TableGenerator(name="myTabGen", table="keytable",
                pkColumnName="keygenName", pkColumnValue="myGen",
                valueColumnName="newValue", initialValue=100,
                allocationSize=10)

@Id @GeneratedValue (strategy=GenerationType.TABLE, generator="myTabGen")
@Column(name="gno")
private Long id;
```

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

- **2 techniques for creating a composite key**
 - **@IdClass**
 - **@EmbeddedId**
- **in both cases, an extra class is added for the Id “Object”**
- **class has to**
 - **implement Serializable (since possible session caching could occur)**
 - **override equals/hashCode (for testing key equality)**
- **usage in JpaRepository class will be slightly different for both techniques!**

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Composite key via @IdClass

```
public class PersonKey implements Serializable {
    private int nr;
    private String email;
    public PersonKey() { }
    public PersonKey(int nr, String email) {
        this.nr=nr;
        this.email=email;
    }
    public boolean equals(Object o){
        Person p= (Person)o;
        return this.nr==p.nr && this.email.equals(p.email);
    }
    public int hashCode(){
        return Objects.hash(nr,email);
    }
}

@Entity
@IdClass(PersonKey.class)
public class Person {
    @Id
    private int nr;
    @Id
    private String email;
    ...
}
```

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Composite key via @IdClass (cont.)

```
public interface PersonJpaRepository extends JpaRepository<Person, PersonKey> {  
  
    // Query DSL  
    Person findByNrAndEmail(int nr, String email);  
  
    //JPQL  
    @Query("select p from Person p where p.nr=:nr and p.email = :email")  
    Person findPersonById(@Param("nr") int nr, @Param("email") String email);  
}
```

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Composite Key via @EmbeddedId

@Embeddable

```
public class PersonKey implements Serializable {  
    private int nr;  
    private String email;  
    public PersonKey() {}  
    public PersonKey(int nr, String email) {  
        this.nr=nr;  
        this.email=email;  
    }  
    public boolean equals(Object o){  
        Person p= (Person)o;  
        return this.nr==p.nr && this.name.equals(p.name);  
    }  
    public int hashCode(){  
        return Objects.hash(nr,email);  
    }  
}
```

@Entity

```
public class Person {  
    @EmbeddedId  
    private PersonKey persKey;  
    ...  
}
```

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Composite Key via @EmbeddedId (cont.)

```
public interface PersonJpaRepository extends JpaRepository<Person, PersonKey> {  
  
    // Query DSL  
    Person findByPersKey_NrAndPersKey_Email(int nr, String email);  
  
    //JPQL  
    @Query("select p from Person p where p.persKey.nr=:nr and p.persKey.email = :email")  
    Person findPersonById(@Param("nr") int nr, @Param("email") String email);  
  
}
```

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

- **persistent field (direct field access), or**
- **persistent property (property access via get/set)**

`@Column (name="column_name")`

- **name of column in relational table (default = property name)**
- **private, protected, package visibility for fields**

Example

```
@Column(name="ctitle")  
private String title;
```

```
@Column(name="cdur")  
public short getDuration() { return duration; }
```

Configuration by exception: **i.e. default configuration/specification for each (non static non transient) field!**

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Property mapping (cont.)

Additional attributes on `@Column`

- **length (optional): column length (default 255 !)**
- **updatable/insertable:** whether or not the column will be part of the update/insert statement (default true)
- **nullable:** set the column as nullable (default true)
- **unique:** set a unique constraint on the column (default false)
- **precision:** column decimal precision (default 0)
- **scale:** column decimal scale if useful (default 0)
- **table:** define the targeted table (default primary table)
- **columnDefinition:** e.g. `CHAR(45)`

Notes:

- `@Transient` -> field will not be persisted
- make use of Bean Validation annotations, e.g. `@NotNull`

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Type conversion - built-in Hibernate types

4.1

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Mapping type	Java type	ANSI Standard SQL type
big_decimal	java.math.BigDecimal	NUMERIC
big_integer	java.math.BigInteger	NUMERIC
boolean	boolean or java.lang.Boolean	BIT
byte	byte or java.lang.Byte	TINYINT
character	java.lang.Character	CHAR(1)
double	double or java.lang.Double	DOUBLE
float	float or java.lang.Float	FLOAT
integer	int or java.lang.Integer	INTEGER
long	long or java.lang.Long	BIGINT
short	short or java.lang.Short	SMALLINT
string	java.lang.String	VARCHAR
yes_no	boolean or java.lang.Boolean	CHAR(1) ('Y' or 'N')
true_false	boolean or java.lang.Boolean	CHAR(1) ('T' or 'F')
date	java.util.Date or java.sql.Date	DATE
time	java.util.Date or java.sql.Time	TIME
local date	java.time.LocalDate (Java 8)	DATE
local time	java.time.LocalTime (Java 8)	TIME
timestamp	java.util.Date or java.sql.Timestamp	TIMESTAMP
calendar	java.util.Calendar	TIMESTAMP

Type conversion - built-in Hibernate types (cont.)

Mapping type	Java type	Standard SQL type
calendar_date	java.util.Calendar	DATE
locale	java.util.Locale	VARCHAR
currency	java.util.Currency	VARCHAR
timezone	java.util.TimeZone	VARCHAR
class	java.lang.Class	VARCHAR
binary	byte[]	VARBINARY (or BLOB)
blob	java.sql.Blob	BLOB
clob	java.sql.Clob	CLOB
serializable	java.io.Serializable implementor	VARBINARY (or BLOB)
text	java.lang.String	CLOB

- **type is derived by using reflection**
- **@Temporal (TemporalType.DATE, TIME, or TIMESTAMP)** for date/time fields
- **@Lob** for large object (or serializable) properties
- **@Enumerated** for enums (EnumType.STRING or .ORDINAL)
- **User-defined types can be specified with @Type annotation**

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Property not mapped to database column

- annotation `@Formula`
- value of property calculated at runtime
- no column attribute (no insert or delete) -> read-only (Select)
- are allowed in formula (SQL fragment):
 - columns of database table
 - SQL functions
 - subselects

Example

```
@Formula("cprice*1.21")  
private float finalPrice;
```

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

General

5.1

- in any reasonable-sized application, same methods (add, remove,...) will reappear (for different classes) over and over again
- only domain types will be different
- solution: only write repository interface that extends JpaRepository
public interface GuestJpaRepository extends JpaRepository<Guest,Long> {}
- above example will persist **Guest** objects with id of type **Long**
- JpaRepository contains 18 methods for performing common persistence operations, such as save(=merge!), delete, find by id,...
- extra methods can be added, which are translated to SQL
- implementation is generated at application startup
- configure via
 - **Java Config**
@EnableJpaRepository(basePackages="be.abis.ch8jparepo.dao")
 - automatically configured in @SpringBootApplication

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

- **CrudRepository<T,ID>**
 - **<S extends T> S save(S var1): saves/updates a given entity**
 - **Optional<T> findById(ID var1): retrieves an entity by its id**
 - **boolean existsById(ID var1): does entity with given id exists**
 - **Iterable<T> findAll(): all instances of the type**
 - **long count(): number of entities available**
 - **void deleteById(ID var1): deletes the entity with the given id**
 - **void delete(T var1): deletes a given entity**
 - **void deleteAll(): deletes all entities managed by the repository**

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Repository Interfaces (cont.)

- **PagingAndSortingRepository (extends CrudRepository)**
 - **Iterable<T> findAll(Sort var1):** returns all entities sorted by the given options
 - **Page<T> findAll(Pageable var1):** returns a Page of entities meeting the paging restriction provided in the Pageable object
- **JpaRepository (extends PagingAndSortingRepository)**
 - **List<T> findAll()**
 - **List<T> findAll(Sort var1)**
 - **void flush():** flushes all pending changes to the database
 - **<S extends T> S saveAndFlush(S var1):** saves an entity and flushes changes instantly
 - **void deleteInBatch(Iterable<T> var1):** deletes the given entities in a batch which means it will create a single query
 - **void deleteAllInBatch():** deletes all entities in a batch call
 - **T getById(ID var1):** returns a **reference** to the entity with the given identifier

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

- **possibility to add methods to interface**
- **Spring can determine implementation based on the method name**
- **example**
`List<Company> getCompanyByCountryOrderByName(String country);`
- **four verbs allowed in method name: get, read, find and count**
- **subject of repository is mostly optional**
- **syntax: query verb + subject (optional) + predicate + order by**
- **predicate**
 - **condition must reference one or more properties (combined with And and Or)**
 - **if no comparison operator, than equals is used**
 - **other comparison operators:**
`IsAfter, IsGreaterThan, IsLessThan, IsBetween, IsNull, IsLike, Containing, IsNotIn, IsStartingWith, Contains, IsTrue,...`
 - **IgnoringCase can also be added**

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Example

```
public interface GuestJpaRepository extends JpaRepository<Guest, Long> {  
    Guest findById(long id);  
    List<Guest> findByName(String name);  
    List<Guest> findByNameStartsWith(String name);  
    List<Guest> findByAge(int age);  
    List<Guest> findByAgelsLessThan(int age);  
    List<Guest> findByNameStartsWithAndAgelsLessThan(String name, int age);  
    long countByName(String name);  
}
```

Basic OR Mapping and JPA repositories

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

- **put `@Transactional` on test class or methods (`org.springframework.transaction.annotation.Transactional`)**
- **test-managed transactions will automatically be rolled back**
- **test-managed transaction is one that is managed by `TransactionalTestExecutionListener`**
- **when running tests while starting up the Spring application context, a `TransactionalTestExecutionListener` is automatically configured**
- **`@Transactional` creates a new transaction that is then automatically rolled back after test completion**
- **can be changed, `@Commit` or `@Rollback` at the class or method level**
- **can use the static methods in `TestTransaction`, to start and end transactions, flag them for commit or rollback or check the transaction status**
- **Watch out: tests via `RestTemplate` or Postman are not automatically rolled back!**

1. Mapping concepts
2. Class to table mapping
3. Object identity
4. Property mapping
5. JPA repositories
6. Spring Data JUnit Testing

Mapping Associations

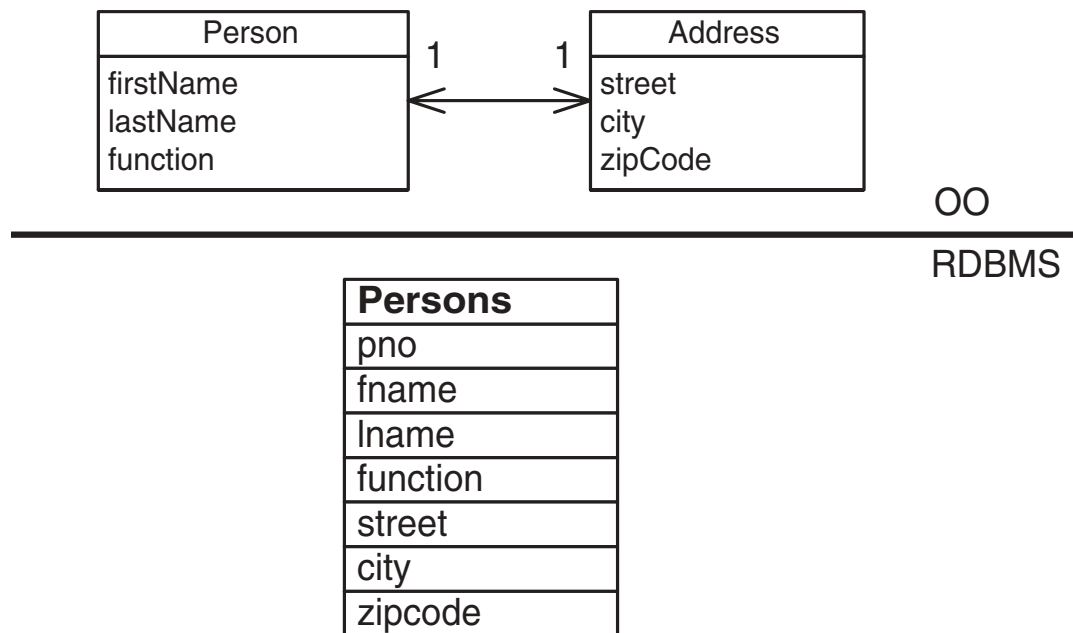
Objectives :

- Value types
- One class for two tables
- Mapping associations
- Cascading and fetching strategies

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Difference between

- **entity type: has its own database identity**
 - **value type: depends on database identity of owning entity type**
- 2 (or more) classes map to 1 database table**



Embedded objects

Use `@Embedded` annotation (in 'parent' class) on contained property

```
@Entity
@Table(name="Persons")
public class Person {
    @Embedded
    private Address address;
    ...
}
```

Use `@Embeddable` annotation on contained ('child') class

```
@Embeddable
public class Address {
    ...
    private String street;
    private String city;
}
```

Notes:

- shared references to Address object is not possible (composition)
- `@Embedded` is optional if `@Embeddable` is defined

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One class for two tables

2

Use @SecondaryTable annotation

@Entity

@Table(name="Persons")

@SecondaryTable(name="Address",
pkJoinColumns = @PrimaryKeyJoinColumn
(name = "pno"))

```
public class Person {
```

```
...
```

```
@Column(name="lname")
```

```
private String lastName;
```

```
...
```

```
@Column(table="Address", name="street")
```

```
private String street;
```

```
@Column(table="Address", name="city")
```

```
private String city;
```

```
...
```

Person
firstName
lastName
function
street
city
zipCode

OO

Persons
pno <pk>
fname
lname
function

Address
street
city
zipCode
pno <fk>

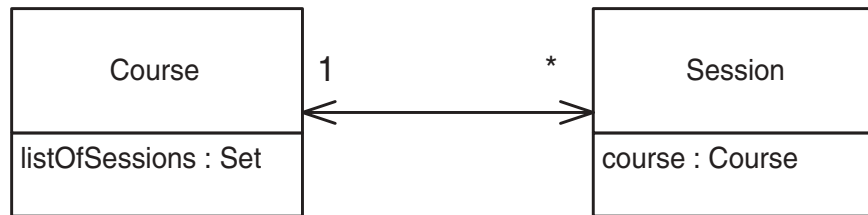
RDBMS

Mapping Associations

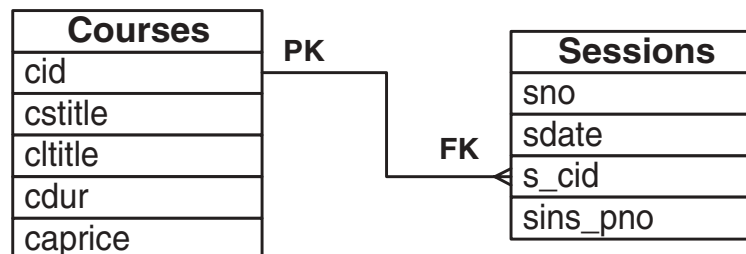
1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Association mapping

3



- **multiplicity (one-to-one, one-to-many, many-to-one, many-to-many)**
- **directionality (navigational access to data)**
 - uni- or bidirectional
 - must be implemented in Java code
 - no managed associations in JPA -> owner class is responsible for updates
- **PK - FK relationship (no direction) in RDBMS**



Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-one mapping

3.1

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

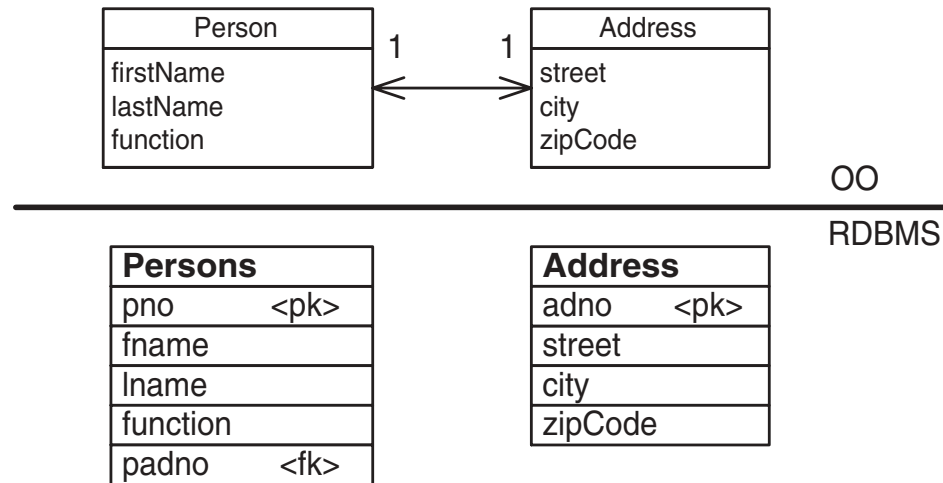
Use when:

- value-type mapping (embedding) not possible
- entity mapping needed - each class mapped to a table

Possibilities:

- foreign key association
- primary key association
- association table (rarely used)

One-to-one mapping (cont.)



- **FK in Persons table refers to PK of Address table**
- **FK (padno) must be unique -> add unique constraint in database**
- **for bidirectional mapping:**
 - **use also one-to-one mapping in Address mapping**
 - **class Person, containing FK, is owning side**

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-one mapping annotations

- **@OneToOne** annotation on association property
- **naming of FK - PK columns**

@JoinColumn(name= ...) on FK property

@Column(name= ...) on PK Id property

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-one mapping - Example

```
@Entity
@Table(name="Persons")
public class Person {

    ...

    @OneToOne
    @JoinColumn(name="padno")
    private Address homeAddress;

    ...
}
```

```
@Entity
@Table(name="Address")
public class Address {

    ...

    @Id @GeneratedValue
    @Column(name="adno")
    private int addressNr;

    ...
}
```

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Bidirectional association

- *Owner (owning side)* class is responsible for the association column(s) update
- Specify additional property on the *target (inverse)* side
- Annotate with `@OneToOne` and attribute `mappedBy` at inverse side of the association (refer to property name in *owner* class)

Example

`@Entity`

`@Table(name="Address")`

`public class Address {`

`...`

`@OneToOne(mappedBy="homeAddress")`

`private Person pers;`

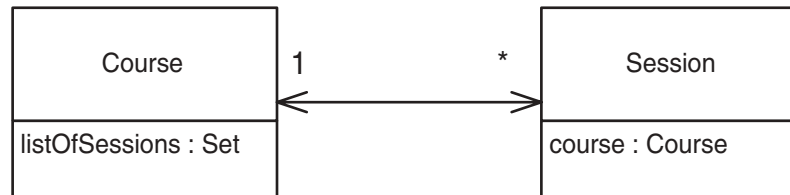
`}`

Mapping Associations

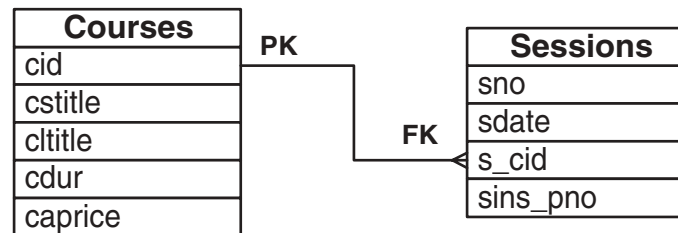
1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Many-to-one mapping

3.2



PK - FK relationship (no direction) in RDBMS



- Use `@ManyToOne` annotation on the **Course** object in the **Session** class

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Many-to-one mapping - Example

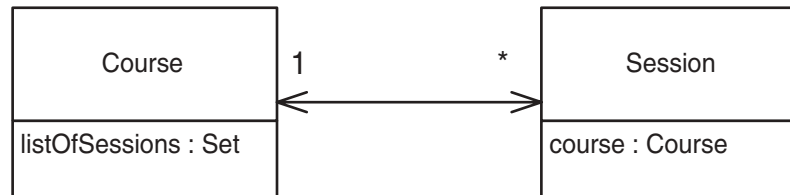
```
public class Course {  
    ...  
    @Id @GeneratedValue  
    @Column(name="cid")  
    private int courseId;  
    private String coursetitle;  
    ...  
}  
  
public class Session {  
    ...  
    @ManyToOne  
    @JoinColumn(name="s_cid")  
    private Course course;  
    ...  
}
```

Mapping Associations

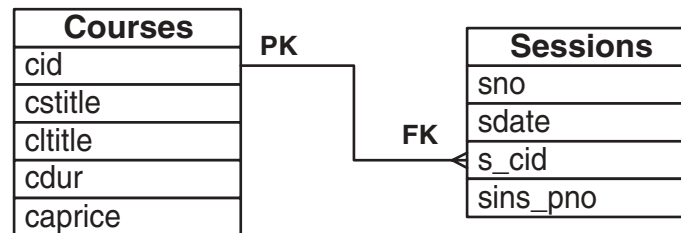
1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-many mapping

3.3



PK - FK relationship (no direction) in RDBMS



Use @OneToMany annotation (on property with interface Set or List)

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-many mapping (cont.)

- Define foreign key column via `mappedBy` attribute, and (optionally) target of association via `targetEntity` attribute
(not required if generic type is specified)
- reverse direction: use `@ManyToOne` and `@JoinColumn` annotations
- 'many' side is owning side

Example

```
public class Course {  
    ...  
    @OneToMany(targetEntity=Session.class, mappedBy="course")  
    private Set<Session> courseSessions = new HashSet<Session>();  
}  
  
public class Session {  
    ...  
    @ManyToOne  
    @JoinColumn(name="s_cid")  
    private Course course;  
}
```

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-many mapping - remarks

- **don't forget to initialise the collection !**
- **override `equals()` and `hashCode()` for the entities used in a set**
identification based on “business key equality”
- **control association inside application, e.g. via convenience method**

```
public class Course{  
    ...  
    public void addSession(Session sess) {  
        this.getCourseSessions().add(sess);  
        sess.setCourse(this);  
    }  
    public void removeSession(Session sess) {  
        this.getCourseSessions().remove(sess);  
        sess.setCourse(null);  
    }  
}
```

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

One-to-many mapping - remarks (cont.)

If the many-collection is implemented as a `java.util.Map`, use the `@MapKey` annotation for the primary key.

Example: person with multiple email addresses

`@Entity`

```
public class Person {
```

```
    . . . .
```

`@OneToMany`

`@MapKey(name="number")`

```
private Map<String, Email> emails;
```

```
    . . . .
```

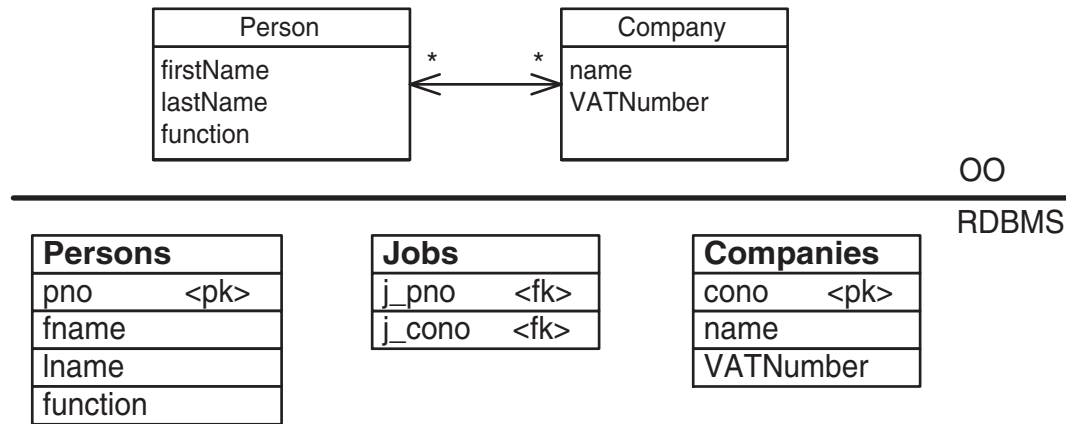
```
}
```

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Many-to-many associations

3.4



Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Can always be mapped with two many-to-one associations
(recommended)

Use `@ManyToMany` annotation

Unidirectional and bidirectional associations possible

In the database, one additional table is needed
(intermediate association class)

Many-to-many mapping

Mapping table generated for each direction by default

- use `mappedBy` attribute to point to same mapping table
- PK is combination of both foreign keys

Definition of join table (and columns)

- `@JoinTable(name="...")`
- `@JoinColumns(joinColumns=@JoinColumn(name="..."), inverseJoinColumns=@JoinColumn(name="..."))`

Bidirectional mapping

- repeat `@JoinTable` and switch join column names

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Example

```
public class Person {  
    ...  
    @ManyToMany  
    @JoinTable(name = "jobs",  
               joinColumns = @JoinColumn(name = "j_pno"),  
               inverseJoinColumns = @JoinColumn(name = "j_cono"))  
    private List<Company> listOfCompanies = new ArrayList<Company>();  
    ...  
}  
  
public class Company {  
    @ManyToMany(targetEntity=Person.class, mappedBy="listOfCompanies")  
    private List<Person> listOfEmployees = new ArrayList<Person>();  
    ...  
}
```

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Cascade strategy

4.1

- how will cascading happen along the association path?
- default = no cascading !
- define `cascade=CascadeType` . (javax.persistence package)
 - **PERSIST**: save/persist corresponding record in other table
 - **REMOVE**: delete corresponding record in other table
 - **MERGE**: merge (after detach) corresponding record in other table
 - **REFRESH**: refresh corresponding record from other table
 - **DETACH**: detach corresponding record in other table
 - **ALL**: all the above options
- If cascade type does not correspond with database action:
TransientObjectException

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Additional options (cont.)

Fetching strategy

4.2

- how are associated records fetched?
- define `fetch=FetchType` .
 - **EAGER**: load corresponding record (default for one-to-one and many-to-one)
 - **LAZY**: do not load corresponding record (default for one-to-many and many-to-many)also called lazy initialisation

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Mapping Associations

1. Value types
2. One class for two tables
3. Association mapping
4. Additional options

Querying in Spring JPA

Objectives :

- **@Query**
- **JPQL**
- **Named queries**
- **Native queries**
- **Using DTOs and mappers**

- **@Query can be defined on repository methods to specify your own query, instead of using a generated one**

- **query can be written in JPQL or SQL**

```
@Query("select c from Course c where c.title = 'Java' ")  
List<Course> findJavaCourses();
```

```
@Query( value="select * from courses where title = 'Java' ",  
        nativeQuery=true)  
List<Course> findJavaCoursesNative();
```

- **positional or named parameters can be passed**

```
@Query("select c from Course c where c.title = ?1 ")  
List<Course> findCoursesWithTitle(String title);
```

```
@Query("select c from Course c where c.title = :title ")  
List<Course> findCoursesWithTitle(@Param("title") String title);
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

JPA Query Language (JPQL) characteristics

- use classes + properties instead of tables + columns
- polymorphic queries
- automatic join for associations
- full support of relational operations
- paging features

Querying is done in an ‘object oriented’ way!

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

projection **SELECT** **specify selected data**

selection **FROM** **data source**

restriction **WHERE** **match records to criteria**

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **Defines the class name + an alias**
`select cc from ConcreteCourse [as] cc`
- **JPQL is case sensitive for class names and properties**
- **if @Entity(name="abc") attribute is used on class, use abc in JPQL from clause**
- **alias will be used in where clause**

JPQL: polymorphic queries

```
Select c from Course c
    //selects all ConcreteCourse and CourseGroup objects

Select o from java.lang.Object o
    //selects all persistent objects

Select s from java.io.Serializable s
    //selects all serializable objects
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **Comparison operators**

- **where column [= | <> | < | > | <= | >=] expression**

- **use of NOT: negates the expression**

```
select c from Course c where c.title = 'Java'
```

```
select c from Course c where c.title <> 'Java'
```

```
select c from Course c where c.duration > 3
```

```
select c from Course c where c.duration <= 4
```

```
select c from Course c where not(c.title = 'Java')
```

- **(NOT) IN operator**

```
select c from Course c where c.title IN ('JAVAPROG','SPRINGJPA')
```

```
select c from Course c NOT IN ('JAVAPROG','SPRINGJPA')
```

- **(NOT) Between operator (border values are inclusive)**

```
select c from Course c where c.price between 200 and 300
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

Restriction: Where (cont.)

- **(NOT) like operator**

```
select c from Course c where c.title like 'Java%'
select c from Course c where c.title like '%Java%'
select c from Course c where c.title not like '%Java%'
select c from Course c where lower(c.title) like '%java%'
```

- **Is (NOT) Null checking**

```
select c from Course c where c.title is null
select c from Course c where c.title is not null
```

- **AND, OR, NOT operators**

```
select c from Course c
where c.title not like '%java%'
    and (c.duration < 2 or c.duration > 4)
```

- **Collection operations**

```
select c from Company c, Person p
where c.employees is not empty

select c from Company c, Person p
where p member of c.employees

select c from Company c, Person p
where size(c.employees) > 100
```

- **functions :** https://en.wikibooks.org/wiki/Java_Persistence/JPQL

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **inner join automatically generated when traversing entities**

`select p from Person p where p.address.country = 'B'`

- **join can also be mentioned explicitly**

- **“comma” notation, with join condition in where statement**

`select d from Employee e, Department d where e.department = d`

- **(inner) join**

`select distinct p from Person p join p.enrolments e where e.pricePerDayPaid > 400`

- **left join**

`select p from Person p left join Company c where p.age > 40`

- **join fetch (to override FetchType.LAZY)**

`select p from Person p join fetch p.companies c where p.age > 40`

mainly used in combination with @OneToMany and @ManyToMany associations

watch out: if no where condition or traversing entity used, a cartesian product is generated!

`select p from Person p join Address a`

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **Named parameters**

- parameter name preceded by a “:”
- in method, use `@Param` which states the name of the parameter

```
@Query("select c from Course c  
      where c.duration = :dur and c.title like %:title%")  
List<Course> getCourseByDurationAndTitle  
    (@Param("dur") int duration, @Param("title") String title)
```

- **Positional parameters (deprecated!)**

```
@Query("select c from Course c  
      where c.duration = ?1 and c.title like %?2%")  
List<Course> getCourseByDurationAndTitle  
    (int duration, String title)
```

Note: watch out for SQL injection !

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **Predefine commonly used queries in the persistent class**
- **Add `@NamedQuery` annotation before entity class**
- **add a method in the repository interface which has exactly the same name as the name of the named query**
- **name obligatory prefixed with the class name**
- **uses JPQL by default**
- **possible to add parameters**
- **also `@NamedNativeQuery`, which will use SQL instead**
- **Example**

```
@NamedQuery(name="Course.findByDurAndTitleNQ",
            query="select c from Course c
                  where c.duration = :dur and c.title like %:title%")

public class Course {
    ...
}

public interface CourseJpaRepository extends JpaRepository<Course,Integer> {
    List<Course> findByDurAndTitleNQ
        (@Param("dur") int duration, @Param("title") String title);
}
```

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- use `@Query(value="", nativeQuery=true)`
- via `@NamedNativeQuery`
- specify result class in the repository method
- plain SQL that uses table names and columns
- JPA will not generate the query/queries in this case
- mostly used in case of limiting results when using joins
- gives you more control, but are DBMS specific
- Example

```
@Query( value="select * from courses where cname like %:name% order by cname",  
        nativeQuery=true)
```

```
List<Course> findCoursesOrderedByName(@Param("name") String name);
```

remark: it is possible that you will need to use a concat function when using a “like” combined with a parameter in native queries (certainly when used together with other functions):

```
select * from courses where cname like concat('%',:name,'%') order by cname
```

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **returned entity will change when select doesn't fetch the full object**
 - **aggregate functions -> numerical value returned**
 - **select specific columns or group by -> List<Object[]> returned**
- **use Data Transfer Objects (DTOs) and mappers to convert to/from the “original” entity**
- **mainly needed when using native queries with joins, or limiting the result shown via a REST API**
- **better for performance, but comes with the “cost” of mapping the result**

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **avg**

```
@Query("select avg(c.duration) from Course c")  
Double getAverageDurationOfCourses();
```

- **min**

```
@Query("select min(c.duration) from Course c")  
Integer getMinimumDurationOfCourses();
```

- **max**

```
@Query("select max(c.duration) from Course c")  
Integer getMaximumDurationOfCourses();
```

- **sum**

```
@Query("select sum(c.duration) from Course c")  
Integer getTotalDurationOfCourses();
```

- **count**

```
Integer countByDuration(int duration);
```

- **count distinct**

```
@Query("select count(distinct c.duration) from Course c")  
Integer getNumberOfDifferentDurationOfCourses();
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **Object[] or List<Object[]> returned instead of original object**

- **Examples**

```
@Query("select c.title, c.duration from Course c where c.id=:id")
```

```
Object[ ] getCourseTitleAndDurationById(@Param("id") int id);
```

```
@Query("select c.title, count(c) from Course c group by c.domain having count(c)>2")
```

```
List<Object[ ]> getNumberOfCoursesPerDomain();
```

```
@Query(value="p.pfname, p.plname, co.coname  
            from persons p inner join companies c on p.pa_cono=c.cono",  
        nativeQuery=true)
```

```
List<Object[ ]> getPersonsWithCompanyName();
```

- **will need to be mapped to a DTO**
- **techniques:**
 - custom Mapper class
 - @Named(Native)Query and @SqlResultSetMapping
 - Interface-based DTO projections
 - Class-based DTO projections

Only the first 2 techniques can be used for native queries!

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

Data Transfer Objects (DTOs)

6.1

- class that maps the result of a query
- provide a constructor with all arguments that will be passed
- used to limit the size of objects passed to the client (web or via JSON)
- will increase performance
- better design: separates entity layer and “view” layer objects
- will need transformation/mapping!
- **Example**

```
public class PersonDTO {  
    private String firstName;  
    private String lastName;  
    private String companyName;  
    public PersonDTO(String firstName, String lastName, String companyName){  
        ...  
    }  
    ...  
}
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **user-defined class that maps the `Object[ ]` to a `PersonDTO` object**
- **contains a static method `PersonDTO toDTO(Object[ ])`**
- **service will then call the mapper object, and returns the `PersonDTO` object**
- **all array elements will need to be converted from `Object` to their correct type!**

Querying in Spring JPA

1. `@Query`
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

Custom Mapper Class - Example

```
public PersonJpaRepository extends JpaRepository<Person,Integer>{
    @Query(value="select p.pfname, p.plname, co.coname
        from persons p inner join companies c on p.pa_cono=c.cono",
        nativeQuery=true)
    List<Object[ ]> getAllPersonsWithCompanyName();
}
```

```
public class PersonDTOMapper {
    public static PersonDTO toDTO(Object[ ] objArray){
        PersonDTO p = new PersonDTO(objArray[0].toString(),
                                    objArray[1].toString(),
                                    objArray[2].toString());

        return p;
    }
}
```

```
public class MyPersonService implements PersonService {
    @Autowired PersonJpaRepository pjr;
    public List<PersonDTO> findAllPersonsWithCompanyName(){
        List<PersonDTO> persons = new ArrayList<PersonDTO>();
        List<Object[ ]> resList = pjr.getAllPersonsWithCompanyName();
        for (Object[ ] objArray : resList){
            persons.add(PersonDTOMapper.toDTO(objArray));
        }
        return persons;
    }
}
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- use a named (native) query on the class
- @SqlResultSetMapping then used to map the columns
- List of DTO objects can then directly be returned from the repository
- Example

```
@NamedNativeQuery(name="Person.getAllPersonsWithCompanyNameNQ",
    query="p.pfname first, p.plname last, co.coname comp
    from persons p inner join companies c on p.pa_cono=c.cono",
    resultSetMapping = "PersonDtoMapping")

@SqlResultSetMapping(name = "PersonDtoMapping",
    classes = @ConstructorResult(targetClass = PersonDto.class,
        columns = {@ColumnResult(name = "first"),
            @ColumnResult(name = "last"),
            @ColumnResult(name = "comp")}))

@Entity
public class Person {
    ...
}

public PersonJpaRepository extends JpaRepository<Person,Integer>{
    @Query(nativeQuery=true)
    List<PersonDTO> getAllPersonsWithCompanyNameNQ();
}
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

- **Class-based DTO projections**

- **use constructor call directly in the JPQL query**
- **fully qualified name needed!**

```
public PersonJpaRepository extends JpaRepository<Person,Integer>{  
    @Query  
    (value="new be.abis.demo.dto.PersonDTO(p.firstName, p.lastName,  
                                             p.company.name) from Person p")  
    List<PersonDTO> getAllPersonsWithCompanyName();  
}
```

- **Interface-based DTO projections**

- **instead of using DTO class, use an interface with the columns in the exact order**
- **watch out: easy to write, but bad performance!**

```
public interface PersonDTO {  
    String getFirstName();  
    String getLastName();  
    String getCompanyName();  
}  
public PersonJpaRepository extends JpaRepository<Person,Integer>{  
    @Query(value="p.firstName, p.lastName,p.company.name from Person p")  
    List<PersonDTO> getAllPersonsWithCompanyName();  
}
```

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the "select" object
6. Using DTOs and mappers

Querying in Spring JPA

1. @Query
2. JPQL
3. Named queries
4. Native queries
5. Changing the “select” object
6. Using DTOs and mappers

Transaction Management and Exception Handling

Objectives :

- **Exception Handling**
- **Modifying queries**
- **Transaction Management**
- **Declarative and programmatic transactions**

- Spring JPA predefines many (around 35 of them!) exceptions
- platform-agnostic
- super class = `org.springframework.dao.DataAccessException`
- descriptive names :
 `IncorrectResultSizeDataAccessException`,
 `DataIntegrityViolationException`,...
- all of them are `RuntimeException` -> unchecked
- still important to do correct exception handling!!!
- documentation:

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/dao/DataAccessException.html>

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- JPQL/native queries can be used for DML
- add **@Modifying** annotation on the repository method
- obligatory to add **@Transactional** when calling them!
- Examples

```
public PersonJpaRepository extends JpaRepository<Person,Integer>{
    @Modifying
    @Query(value="update persons set ppwd=:newpwd where pno=:id ,
        nativeQuery=true)
    Person updatePassword(@Param("newpwd") String newpwd, @Param("id") int id);
}

public class MyPersonService implements PersonService {
    @Autowired PersonJpaRepository pjr;
    @Transactional
    public void updatePassword(String newpwd, int id){
        pjr.updatePassword(newpwd,id);
    }
}
```

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- **transactions ensure all-or-nothing behaviour**
- **data never left in inconsistent state**
- **ACID test**
- **Hibernate/JPA imposes transaction management**
- **global vs local transactions**
 - **global transactions**
 - enable you to work with multiple transactional resources, typically relational databases and message queues
 - application server manages global transactions through JTA
 - **local transactions**
 - resource-specific, e.g. transaction linked to a JDBC connection
 - simpler, but no application server involved, so limited

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- **consistent programming model in any environment (whether global or local)**
- **only need an application server's JTA capability if application needs to handle transactions across multiple resources**
- **Spring supports both programmatic and declarative transaction management**
- **abstract away actual transactional implementation from code**
- **transaction strategy based on PlatformTransactionManager**
- **TransactionException is a RuntimeException**
- **By default everything is autocommitted, unless @Transactional is used**

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- **different implementations of PlatformTransactionManager**
- **acts as facade to platform-specific transaction implementation**
 - **DataSourceTransactionManager (JDBC) -> dataSource**
 - **HibernateTransactionManager -> sessionFactory**
 - **JpaTransactionManager -> entityManager**
 - **JtaTransactionManager -> need dataSource via JNDI**
- **getTransaction() will return a TransactionStatus based on a TransactionDefinition**
- **Spring Boot automatically configures a JpaTransactionManager (if no JTA context found)**
- **use via**
 - **@Transactional (declarative transactions)**
 - **wired in transactionTemplate (programmatic transactions)**

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- might represent a new transaction, or can represent an existing transaction if a matching transaction exists in the current call stack
- associated with a thread of execution
- code can use this to retrieve status information, and to programmatically request a rollback
- provides access to savepoint management facilities
- methods
 - `flush()`: flush the underlying session to the datastore, if applicable
 - `hasSavepoint()`: does transaction internally have a savepoint
 - `isCompleted()`: is transaction completed (commit or rollback)
 - `isNewTransaction()`: is present transaction new or nested
 - `isRollbackOnly()`: is transaction marked as rollback-only
 - `setRollbackOnly()`: set the transaction rollback-only.

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- **isolation level**
 - **degree to which transaction is isolated from other transactions**
 - **possibilities:**
 - ISOLATION_READ_COMMITTED: dirty reads are prevented; non-repeatable reads and phantom reads can occur
 - ISOLATION_READ_UNCOMMITTED: dirty reads, non-repeatable reads and phantom reads can occur.
 - ISOLATION_REPEATABLE_READ: dirty reads and non-repeatable reads are prevented; phantom reads can occur.
 - ISOLATION_SERIALIZABLE: dirty reads, non-repeatable reads and phantom reads are prevented
 - (default!) ISOLATION_DEFAULT: use the default isolation level of the underlying datastore

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Transaction Definition (cont.)

- **propagation behaviour**
 - **defines the transaction scope**
 - **Spring offers all of the transaction propagation options familiar from EJB CMT**
 - **possibilities**
 - PROPAGATION_NESTED: execute within a nested transaction if a current transaction exists, PROPAGATION_REQUIRED like behaviour else
 - PROPAGATION_NEVER: do not support a current transaction; throw an exception if a current transaction exists.
 - PROPAGATION_NOT_SUPPORTED: do not support a current transaction; rather always execute non-transactionally
 - (default!) PROPAGATION_REQUIRED: support a current transaction; create a new one if none exists.
 - PROPAGATION_REQUIRES_NEW: create a new transaction, suspending the current transaction if one exists.
 - PROPAGATION_SUPPORTS: support a current transaction; execute non-transactionally if none exists.

Transaction Management and Exception Handling

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Transaction Definition (cont.)

- **timeout**
 - **how long does transaction runs before timing out and being rolled back automatically**
 - **defined as int**
 - **default = - 1: use default timeout of the underlying transaction system**
- **read-only**
 - **defines whether underlying transaction is readonly**
 - **default = false**
- **rollback rules**
 - **defines for which Exceptions the transaction should be rolled back**
 - **default: RuntimeException and Error**

Transaction Management and Exception Handling

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- implemented through Spring's AOP framework
- annotation based:
 - `@Transactional (org.springframework.transaction.annotation)`
 - attributes define the transaction definition properties
- only need to override non-default options
- method annotations override class annotations
- use in the Service Layer for DML methods
- for delete: use `myRepo.flush()` such that the exception is thrown within the transaction boundary
- in JUnit tests
 - automatic rollback when putting `@Transactional` on class/method, use `@Commit` on test to change this behaviour
 - Make sure that the `@Transactional` annotation on the test does not change the behaviour of the method in the Service layer!
 - put `@Transactional` only on “happy flow” tests, exceptions should be handled from the Service layer

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Example

- **AbisReceptionService.java**

```
@Service
public class AbisReceptionService implements ReceptionService {
    @Autowired
    private CompanyJpaRepository cjr;

    ...

    @Transactional(isolation = Isolation.SERIALIZABLE,
                    rollbackFor = CompanyException.class)
    public void doStuffWithCompanies() throws CompanyException{
        try {
            Company c = new Company
                (1000, "My Company", "some street", "30", "3000", "Leuven", "B");
            cjr.save(c);
            c.setStreet("Diestsevest");
            cjr.save(c);
            System.out.println(cjr.findById(1000));
            cjr.deleteById(3);
            List<Company> companies = cjr.findAll();
            for (Company comp : companies) {System.out.println(comp);}
            cjr.flush();
        } catch (EmptyResultDataAccessException | DataIntegrityViolationException e){
            throw new CompanyException("oops");
        }
    }
}
```

Transaction Management and Exception Handling

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Example (cont.)

- **ReceptionTest.java**

```
@SpringBootTest
public class ReceptionTest {
    @Autowired
    private ReceptionService receptionService;

    @Test
    @Transactional
    public void testdoStuffWithCompanies() {
        assertDoesNotThrow(()->receptionService.doStuffWithCompanies());
    }

    @Test
    public void testdoStuffWithCompanies() {
        assertThrows(CompanyException.class,
            ()->receptionService.doStuffWithCompanies());
    }
}
```

Transaction Management and Exception Handling

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

- **control every step in the transaction yourself**
- **can be more fine-grained than declarative transactions**
- **uses TransactionTemplate that wires in TransactionManager**
- **implement TransactionCallback interface**
- **call doInTransaction() method**
- **wire template into your Service classes**
- **intrusive in your code!**

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Example

@Service

public class SimpleService implements Service {

private final TransactionTemplate transactionTemplate;

@Autowired

public SimpleService(PlatformTransactionManager transactionManager) {
 this.transactionTemplate = new TransactionTemplate(transactionManager);
}

public Object someServiceMethod() {

return transactionTemplate.execute(new TransactionCallback() {

public Object doInTransaction(TransactionStatus status) {

try {

updateOperation1();

updateOperation2();

} catch (SomeBusinessException ex) {

status.setRollbackOnly();

}

}

});

}

}

Transaction Management and Exception Handling

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Transaction Management and Exception Handling

1. Exception Handling
2. Modifying queries
3. Understanding transactions
4. Transaction Status
5. Transaction Definition
6. Declarative transactions
7. Programming transactions

Securing REST APIs

Objectives :

- **API keys**
- **HTTP basic and digest authentication**
- **Authorizing Access**
- **OAuth 2.0 and OIDC**
- **HTTPS - Trust stores and Key Stores**
- **CORS**

Security is an important part of any enterprise application

Not all APIs are/should be public!

Two essential elements

- **Authentication:**
verifying the identity of the user trying to access the API
- **Authorization:**
verifying what an authenticated user is permitted to do in the API

Ways to secure your REST API

- **API keys**
- **HTTP basic / digest authentication**
- **OAuth 2.0**
- **Authorize the RESTful web service accesses**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

Application Security vs Server Security

Application Security

- **application has its own way to check authentication & authorization**
- **custom database**
- **some possibilities:**
 - authentication via login screen
 - other web pages check whether login was done correctly
 - authorization via roles of users
- **“custom” encoding mechanisms via (java) frameworks**

Server Security

- **SERVER defines users and roles (possibly via LDAP)**
- **can be used for different applications at once**
- **application not called when authentication fails**
- **encryption via HTTPS, Digest authentication,...**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

API key is a token that a client provides when making API calls

Supplied by the API provider (e.g. when creating an account)

An API Key has these properties:

- **token, in the form of a relatively long random string (e.g. 32 characters)**
- **identifier, for storage and unique identification**
- **transmitted with the request (requires SSL for in production) in the HTTP header or via a cookie (convention: call it api-key)**
- **known to the client**
- **can be validated by the server**
- **unique to a device or software (e.g. UUID)**
- **bound to a user if necessary**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

RestReceptionController.java

```
@GetMapping("{id}")
public Guest returnGuestById(@PathVariable("id") int id,
                             @RequestHeader MultiValueMap<String,String> headers)
    throws GuestNotFoundException, ApiKeyNotCorrectException {
    boolean keyOK = false;
    if (headers.containsKey("api-key")){
        String auth = headers.get("api-key").get(0);
        System.out.println("key passed: " + auth);
        boolean tokenOK = this.checkToken(auth);
        if (tokenOK) keyOK=true;
    }
    if (keyOK) {
        return receptionService.findGuest(id);
    } else {
        throw new ApiKeyNotCorrectException("please provide a valid API KEY");
    }
}

private boolean checkToken(String token) {
    boolean tokenOk = false;
    ArrayList<String> acceptedTokens = new ArrayList<String>();
    acceptedTokens.add("012345678901234567890123456789ab");
    acceptedTokens.add("51ddcd22-09be-422a-8ec1-41a1179c4b72");
    acceptedTokens.add(UUID.randomUUID().toString());
    if (token != null && acceptedTokens.contains(token)) tokenOk = true;
    return tokenOk;
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

API Keys in the API (cont.)

ApiKeyNotCorrectException.java

```
public class ApiKeyNotCorrectException extends Exception {  
    public ApiKeyNotCorrectException(String message) {  
        super(message);  
    }  
}
```

RestResponseEntityExceptionHandler.java

```
@RestControllerAdvice  
public class RestResponseEntityExceptionHandler  
    extends ResponseEntityExceptionHandler {  
    @ExceptionHandler(value = ApiKeyNotCorrectException.class)  
    protected ResponseEntity<? extends Object> handleGuestNotFound  
        (ApiKeyNotCorrectException ance, WebRequest request) {  
        HttpStatus status = HttpStatus.UNAUTHORIZED;  
        ApiError err = new ApiError("unauthorized", status.value(), ance.getMessage());  
        HttpHeaders responseHeaders = new HttpHeaders();  
        responseHeaders.add("content-type",  
            MediaType.APPLICATION_PROBLEM_JSON_VALUE);  
        return new ResponseEntity<ApiError>(err, responseHeaders, status);  
    }  
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

JUnit Test

```
@SpringBootTest
public class Ch6ApiKeyClientApplicationTests {

    @Autowired private RestTemplate rt;
    private String baseUrl = "http://localhost:8081/securitykey/guests";

    @Test
    public void testFindGuestById() throws Exception {
        HttpHeaders requestHeaders = new HttpHeaders();
        requestHeaders.add("api-key", "012345678901234567890123456789ab");
        HttpEntity<Guest> requestEntity = new HttpEntity<>(requestHeaders);
        ResponseEntity<? extends Object> re=null;
        try {
            re = rt.exchange(baseUrl+ "/2", HttpMethod.GET,requestEntity, Guest.class);
            Guest guest = (Guest) re.getBody();
            assertEquals("An", ((Guest) re.getBody()).getName());
        } catch (HttpStatusCodeException e) {
            if(HttpStatus.NOT_FOUND == e.getStatusCode()
                || HttpStatus.UNAUTHORIZED == e.getStatusCode()) {
                String serr = e.getResponseBodyAsString();
                ApiError ae=new ObjectMapper().readValue(serr,ApiError.class);
                System.out.println(ae.getDescription());
            } else {
                System.out.println("some other error occurred");
            }
        }
    }
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- provide declarative security for Spring-based applications
- roots in Acegi Security
- tackles security from two angles
 - secure web requests via servlet filters
 - secure method invocations using AOP
- security namespace
- best put all configuration in separate file
- Maven Dependency

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

contains several modules like spring-security-config, spring-security-web, spring-aop and spring-web

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

BASIC

- **send a Base64 encoded username and password as a pair in the HTTP authorization header**
- **username and password must be sent for every HTTP request made by the client**
- **chain of events:**
 - **client requests a URI**
 - **server replies with a 401 HTTP response**
 - **client receives response, scans through it, and prepares a new request with the necessary data needed to authenticate the user**
 - **new request from the client will contain the authorization header set to a Base64 encoded value of column delimited username and password string, <username>:<password>**
 - **server verifies credentials and replies with requested resource**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

HTTP Basic and Digest Authentication (cont.)

- **deficiencies:**
 - **unauthorized access could take place with cached credentials**
 - **no way for a web service to differentiate authorized requests from unauthorized ones**
 - **not enough for security because usernames and passwords are only encoded using Base64 encoding, which is easy to decipher**
 - **intent of Base64 is not to secure the name-value pair, but to uniformly encode characters when transferred over HTTP**
 - **not recommended to use basic authentication over HTTP for application accessed over the Internet**
 - **solve this potential security hole by using HTTPS**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

HTTP Basic and Digest Authentication (cont.)

DIGEST

- **unlike basic authentication, password is not transmitted in clear text between the client and the server**
- **client sends a one way cryptographic hash of the username, password, and a few other security related fields using the MD5 message-digest hash algorithm**
- **when server receives the request, it regenerates the hashed value for all the fields as done by client and compares it with the one present in the request**
- **if the hashes match, request is treated as authenticated and valid**
- **remark: not all servers (like WAS) support digest authentication!**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **Java Config**

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {}
```

- **default configuration will**

- demand that each HTTP request is authenticated
- since no user store is defined, nobody can login yet
- automatically reroute the user to a default login screen
e.g. go to localhost:8081/security/guests in your browser

- **actions to be taken**

- user store needs to be configured
- specify which requests should and should not require authentication and what authorities are required
- (provide a custom login screen)

- **Watch out: since Spring Boot 3: security config class cannot extend WebSecurityConfigurerAdapter anymore !!!**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **Spring is capable of authenticating against virtually any data store**
 - **in-memory**
 - **relational database**
 - **LDAP**
- **also possible to create custom implementation**
- **since Spring 5: PasswordEncoder required**
- **in-memory**
 - **add PasswordEncoder**
 - **create Bean with type UserDetailsService which returns an InMemoryUserDetailsManager**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

Example of an In-Memory User Store

- since Spring 5: PasswordEncoder encouraged
- SecurityConfig.java

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

```
public BCryptPasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

 @Bean

```
public UserDetailsService userDetailsService(PasswordEncoder encoder){  
    UserDetails user = User.withUsername("user1")  
        .password(encoder.encode("pwd1"))  
        .roles("USER")  
        .build();  
  
    UserDetails admin = User.withUsername("user2")  
        .password(encoder.encode("pwd2"))  
        .roles("USER", "ADMIN")  
        .build();  
  
    return new InMemoryUserDetailsManager(admin, user);  
}
```

}

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **decide which URLs will require login, and which won't**
- **add a bean returning a `SecurityFilterChain` in `SecurityConfig.java`, taking in a `HttpSecurity` object as a parameter**
- **use `authorizeHttpRequests()` method**
- **uses ANT-style matching syntax**
- **shows 401 when not authorized**
- **disable csrf for the moment**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

Intercepting Requests - Example

- access to /guests permitted for everyone (watch out, also POST possible for the moment)
- all subpaths of /guests (like /guests/1) will have to be authenticated

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {  
    http.csrf(AbstractHttpConfigurer::disable)  
        .authorizeHttpRequests(authorizationManagerRequestMatcherRegistry ->  
            authorizationManagerRequestMatcherRegistry  
                .requestMatchers("/guests").permitAll()  
                .requestMatchers("/guests/*").authenticated())  
        .httpBasic(Customizer.withDefaults())  
        .sessionManagement(httpSecuritySessionManagementConfigurer ->  
            httpSecuritySessionManagementConfigurer  
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS));  
  
    return http.build();  
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **username and password will now have to be provided when calling the API for certain requests**
- **create an interceptor class which will find all incoming HTTP requests**
- **interceptor will need the following Maven dependency:**

```
<dependency>  
  <groupId>org.apache.httpcomponents.client5</groupId>  
  <artifactId>httpclient5</artifactId>  
</dependency>
```
- **basic configuration of RestTemplate (with @Bean) not sufficient any more**
- **need to set up a RestTemplate with basic authentication, by creating a custom component which uses a Spring FactoryBean**
- **RestTemplate will then refer the interceptor class that was created**
- **in the request that has to be authenticated: call the interceptor on the RestTemplate object, and pass username and password via the BasicAuthenticationInterceptor class**
- **make sure exception handling treats the 401 and 403 statuses**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

HTTP Basic Authentication in the Client - Example

MyHttpRequestInterceptor.java

```
public class MyHttpRequestInterceptor extends
    HttpComponentsClientHttpRequestFactory {

    HttpHost host;

    public MyHttpRequestInterceptor(HttpHost host) {
        super();
        this.host = host;
    }

    protected HttpContext createHttpContext(HttpMethod httpMethod, URI uri) {
        return createHttpContext();
    }

    private HttpContext createHttpContext() {
        AuthCache authCache = new BasicAuthCache();
        BasicScheme basicAuth = new BasicScheme();
        authCache.put(host, basicAuth);
        BasicHttpContext localcontext = new BasicHttpContext();
        localcontext.setAttribute(HttpClientContext.AUTH_CACHE, authCache);
        return localcontext;
    }
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

HTTP Basic Authentication in the Client - Example (cont.)

RestTemplateFactory.java

@Component

```
public class RestTemplateFactory
    implements FactoryBean<RestTemplate>, InitializingBean {

    private RestTemplate restTemplate;

    public RestTemplate getObject() {
        return restTemplate;
    }

    public Class<RestTemplate> getObjectType() {
        return RestTemplate.class;
    }

    public boolean isSingleton() {
        return true;
    }

    public void afterPropertiesSet() {
        HttpHost host = new HttpHost("localhost", 8081);
        restTemplate = new RestTemplate
            (new MyHttpRequestInterceptor(host));
    }
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

HTTP Basic Authentication in the Client - Example (cont.)

JUnit test

```
@SpringBootTest
public class Ch6BasicAuthenticationClientApplicationTests {
    @Autowired private RestTemplate rt;
    private String baseUrl = "http://localhost:8081/security/guests";
    @Test
    public void testFindAllGuests(){           // OK, no authentication required
        Guests g = rt.getForObject(baseUrl, Guests.class);
        ArrayList<Guest> list = g.getGuests();
        assertEquals(2,list.size());
    }
    @Test
    public void testFindGuestById() throws Exception { //will require authentication
        rt.getInterceptors()
            .add(new BasicAuthenticationInterceptor("user1", "pwd1"));
        ResponseEntity<? extends Object> re=null;
        try {
            re = rt.exchange(baseUrl+ "/2", HttpMethod.GET, null, Guest.class);
            assertEquals("An", ((Guest) re.getBody()).getName());
        } catch (HttpStatusCodeException e) {
            if (HttpStatus.NOT_FOUND == e.getStatusCode()) {
                //page not found exception handling
            } else {
                System.out.println("NOT AUTHORIZED");
            }
        }
    }
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **some actions will need extra authorization on top of authentication (e.g. post instead of get)**
- **authorization will be role-based (roles passed via SecurityConfig)**
- **Spring uses method level security to enforce authorization**
- **client will stay the same as with basic authentication**
- **don't forget to catch 401 (Unauthorized) and 403 (Forbidden) now**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **second line of defence next to Web Security**
- **secure bean methods**
- **3 different kinds of security annotations**
 - **Spring Security's own @Secured**
 - **JSR-250's @RolesAllowed**
 - **fine-grained expression-driven annotations**
- **configuration:**
 - **add @EnableMethodSecurity(jsr250Enabled=true) on your security config class**
 - **Secure the method via @RolesAllowed**

```
@PostMapping("")
@RolesAllowed("ADMIN")
public void addGuest(@Valid @RequestBody Guest guest){
    receptionService.addGuest(guest);
}
```
- **HttpStatus.FORBIDDEN (403) thrown if access not allowed**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

JUnit Test

@Test

public void testPostValidation() throws Exception {

```
    rt.getInterceptors()
        .add(new BasicAuthenticationInterceptor("user2", "pwd2"));
    Guest g = new Guest(3, "James", 32);
    HttpHeaders requestHeaders = new HttpHeaders();
    requestHeaders.add("content-type", MediaType.APPLICATION_JSON_VALUE);
    requestHeaders.add("accept", MediaType.APPLICATION_JSON_VALUE);
    ResponseEntity<? extends Object> re = null;
    try {
        re = rt.exchange(baseUrl, HttpMethod.POST,
            new HttpEntity<Guest>(g, requestHeaders), Object.class);
        System.out.println("post succeeded, code " + re.getStatusCodeValue());
    } catch (HttpStatusCodeException e) {
        if (HttpStatus.BAD_REQUEST == e.getStatusCode()) {
            //validation exception handling
        } else if (HttpStatus.FORBIDDEN == e.getStatusCode() ||
            HttpStatus.UNAUTHORIZED == e.getStatusCode()) {
            System.out.println("GET OUT!");
        } else {
            System.out.println("some other error occurred");
        }
    }
}
```

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

OAuth 2.0

6.1

- **authorization framework**
- **open standard for authorization, used by many enterprises and service providers to protect resources**
- **authentication done via Authorization Server (could be 3rd party, e.g. Google)**
- **goals**
 - **delegate access**
 - **avoid sharing passwords**
 - **use short-lived access tokens**
 - **allow secure app integration**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

OAuth 2.0 (ctd.)

- **main actors**
 - **resource owner: user owning the data**
 - **client: app requesting access**
 - **authorization server: issues token after user consent**
 - **resource server: api that trusts access tokens**
 - **what OAuth is NOT**
 - **OAuth is not authentication**
 - **does not provide identity**
 - **access token only means "you are allowed," not "you are this person."**
- > other protocol needed for authentication!**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

OpenID Connect (OIDC)

6.2

- **build on top of (including) OAuth 2.0, providing authentication**
- **OIDC adds**
 - **ID Token: JSON Web token (JWT) containing user identity**
 - **userInfo endpoint (retrieve profile information)**

standard scopes: openid, profile, email

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

OIDC Authorization Code Flow (Typical)

- **user visits the application**
- **app redirects to Authorization Server (e.g., Google, Keycloak).**
- **user logs in**
- **authorization Server redirects back with authorization code**
- **app exchanges code for**
 - **ID token**
 - **access token**
 - **optional refresh token**
- **app verifies ID token signature**
- **user is logged in**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **Maven Dependency:**

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-client</artifactId>  
</dependency>
```

- **application.properties (Google OIDC example)**

```
spring.security.oauth2.client.registration.google.client-id=YOUR_CLIENT_ID  
spring.security.oauth2.client.registration.google.client-secret  
                                =YOUR_CLIENT_SECRET  
spring.security.oauth2.client.registration.google.scope=openid,profile,email  
spring.security.oauth2.client.provider.google.issuer-uri  
                                =https://accounts.google.com
```

**to get Google OAuth2 / OpenID Connect Client ID and Client Secret,
create a project in Google Cloud Console and enable the OAuth
consent + credentials**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

Integration in Spring (ctd.)

- **SecurityConfig.java**

@Configuration

@EnableWebSecurity

public class SecurityConfig {

@Bean

public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

http

**.authorizeHttpRequests(auth -> auth
.requestMatchers("/public").permitAll()
.anyRequest().authenticated()
)**

**.oauth2Login() // Enables OpenID Connect login
.oauth2Client(); // Enables OAuth2 client capabilities**

return http.build();

**}
}**

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

Integration in Spring (ctd.)

- **API Controller**

```
@RestController
public class ApiController {
    private OAuth2AuthorizedClientService clientService;

    public ApiController(OAuth2AuthorizedClientService clientService) {
        this.clientService = clientService;
    }

    @GetMapping("/call-api")
    public String callApi(@AuthenticationPrincipal OidcUser user) {
        OAuth2AuthorizedClient client =
            clientService.loadAuthorizedClient("google", user.getName());

        String accessToken = client.getAccessToken().getTokenValue();
        return "Access token: " + accessToken;
    }
}
```

- **UserController**

```
@RestController
public class UserController {
    @GetMapping("/me")
    public String me(@AuthenticationPrincipal OidcUser user) {
        return "Hello " + user.getFullName();
    }
}
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

- **CORS = Cross-Origin Resource Sharing**
- **browser security mechanism that restricts HTTP requests from one origin to resources of another origin**
 - **origin = Protocol + Domain + Port**
 - **prevents unauthorized cross-site calls**
 - **servers must explicitly allow cross-origin requests**

- **configuration**

- **in the API (on the Controller)**

```
@CrossOrigin(origins = "http://localhost:4200")
```

- **via a proxy in client (e.g. angular)**

```
//proxy.conf.json
{ "/ngpersonapi": {
  "target": "http://localhost:8080",
  "secure": false
}}

//angular.json
"projects": {
  "your-project-name": {
    "architect": {"serve": {"options": { "proxyConfig": "proxy.conf.json" }
  } } } }
```

Securing REST APIs

1. Securing APIs
2. API Keys
3. Introduction to Spring Security
4. HTTP Basic and Digest Authentication
5. Authorizing Access
6. Introduction to OAuth 2.0 and OpenID Connect
7. HTTPS - Trust stores and Key Stores
8. CORS

APPENDIX A. EXERCISES

1 Spring Getting Started

1. Create a new Spring Boot project called SpringExercises in IntelliJ. Use the following configuration:

```
type=maven, packaging=jar,  
group=be.abis, artifact=SpringExercises,  
package=be.abis.exercise  
java version = 21
```

Set the version of Spring Boot to 3.5.14. Add Web as dependency.

Copy the model and repository directories of the exerciseskeletonBNP.zip file in the project under the be.abis.exercise package.

Put the personsSpring.csv file under your c:\temp\javacourses (check/change the location of the file in the FilePersonRepository class). Check out the code.

2. Change the MemoryCourseRepository and FilePersonRepository classes such that they are managed by Spring. Use @Component to achieve this. Wire the interfaces into the SpringExercisesApplicationTests class, and run the contextLoads() test.

2 Spring Boot Testing

1. Create a CourseRepositoryTest class. Wire the CourseRepository in here this time. Add tests for all methods, using assertEquals.
2. Create a PersonRepositoryTest class. Wire the PersonRepository. Add tests for all methods, including a test to check the exception. Make sure to also use assertNull and assertNotNull somewhere. Use @BeforeEach to create the person to add.
3. Use @TestMethodOrder in the PersonRepositoryTest to fix the order of the tests.

3 Spring Core

1.
 - a. Create a *CourseService* interface with the following methods:
 - List<Course> findAllCourses();
 - Course findCourse(int id) throws CourseNotFoundException;
 - Course findCourse(String shortTitle);
 - void addCourse(Course c);
 - void updateCourse(Course c);
 - void deleteCourse(int id);
 - b. Create a *AbisCourseService* class that implements this interface. Implement the methods by calling the equivalent CourseRepository methods (as far as possible). Add the exception in the findCourse(id) method. Configure the class by using @Component and @Autowired.
 - c. Add a AbisCourseServiceTest class to test the findCourse(int id) method. Make sure that the CourseService interface is wired into your test class. Use Mockito to mock the result of calling the CourseRepository. Make sure to test both the “happy case” and the Exception.
2. Similar as in the previous exercise, create a *PersonService* interface with an *AbisPersonService* implementation class which wires in the PersonRepository, and mimics its methods. In this case, you can implement all methods. Use the “correct” stereotypes for each application layer. Use JUnit again to test. You can use “integration testing” as from now.

3.

- a. Create a *TrainingService* interface with the following methods
 - `String getWelcomeMessage();`
 - `void enrolForSession(Person person, int sessionId)`
 throws `EnrolException`;
 - `List<Session> findSessionsForCourse(String shortTitle);`
 - `List<Enrolment> findEnrolments(int personId);`
 - b. Implement this in an *AbisTrainingService* class. The class will need to be linked to both the *PersonService* and *CourseService*. The real method implementations will be added later. Use constructor injection for the *CourseService* and setter injection to wire the *PersonService*.
 - c. Fill in the `welcomeMessage` using `@Value` (field injection).
 - d. Test via JUnit. Wire in the *TrainingService*. Test `getWelcomeMessage()`. Also test `findPerson(email,password)` and `findCourse(id)` again, but make sure to only wire the *TrainingService* directly in the test class.
 - e. Add an `init` method that writes: "AbisTrainingService is ready for work!".
 - f. Take out all annotations used to configure the *MemoryCourseRepository*, and configure everything via a Java configuration class instead. Use a separate configuration class for this.
4. Create a second class *SomePersonService* implementing *PersonService*. Make sure that `findPerson(email,password)` returns yourself. Use annotations to configure everything.
 5. Use profiles such that *AbisPersonService* is used in production, and *SomePersonService* while in development. Create a new Test class that uses the *dev* profile. All the other tests should still work by defining *prod* as the profile to be used in the *application.properties*.
 6. Pass the file path to be used in the *FilePersonRepository* class via runtime value injection, using a file called *my.properties* to define the value.

4

AOP

1. Create a *LoginAspect* class that logs each time after a person logs in, but also when there was a wrong login. You can just print a simple message in each case.
2. Create a timer for all `find*` methods in the repository package.

5

Spring REST

1. Create a *PersonController* class (in a package *be.abis.exercise.controller*). Add a method `findPersonByMailAndPwd(String email, String password)` that returns the full *Person* object in json format. The URL should be:
`http://localhost:8081/exercise/personapi/persons?mail=x&pwd=x`
Replace the "x" values with real data, and test in a browser. You can "ignore" the *LoginException* for now.
2. Add controller methods that correspond to (and call) all service methods. Choose the correct HTTP methods. Use the "good practices naming rules" for your URIs. Document the chosen URIs (in a simple text file). Test with Postman.
3. Change the `findPersonByMailAndPwd(email,pwd)` method so it uses POST instead of GET.
4. Change the "int age" variable to a "*LocalDate birthDate*" everywhere (you will also need to change the repository and the *persons.csv* file). Use *d/M/yyyy* as standard date format. Also experiment with the other Jackson annotations.
5. Add a method `List<Person> findPersonsByCompanyName(String compName)`. You will also have to implement the method in the repository and the service layers. Add a test in the *PersonRepositoryTest* class. Let the controller return XML instead of JSON when calling the method. Use Jackson JAXB to do the mapping.
6. Change the `addPerson()` method such that it can consume XML (but JSON should also still be possible).

10 ____ *Entity mappings and Basic JPA repositories*

1.
 - a. Add correct mappings in the Course class to mimic the DB correctly. Use GenerationType.IDENTITY for the courseId.
 - b. Replace the CourseRepository interface with a CourseJpaRepository, which has to be able to call the same methods as before. Use provided methods and query methods for this. Make sure to fetch a course (by id), before you update it.
Wire the new repository into the CourseService class, and make sure that all methods are implemented. Let the addCourse() and updateCourse() methods return a Course object as from now.
 - c. Use the CourseServiceTest class to test all methods (including the DML ones). What happens in the DB? Now add @Transactional on the DML tests, and check again. Also make sure that when running all tests, they nicely run in isolation. Don't use @Order annotations anymore.
2. Test via the Course API. Take a look at the generated SQL via the console in IntelliJ. When adding a new course, make sure not to pass the id anymore!

11 _____ *Association Mapping and @Query*

1.
 - a. Add mappings in the Company and Address classes to map to the companies table.
 - b. Create a CompanyJpaRepository class. Make sure to add methods to find a company by id and to find by name.
 - c. Define a method to find a company by name and town. Use @Query with JPQL for this.
 - d. Use a CompanyRepositoryTest class to test these methods, as well as the DML methods.

2.
 - a. Add the correct mappings in the Person class to mimic the DB correctly. Replace the PersonRepository with a PersonJpaRepository, which has to be able to call the same methods as before.
 - b. Wire this in the PersonService. The "add" and "update" methods should return a Person object now.
 - c. Test via JUnit. Consider following use cases when adding a person:
 - person working for a new company
 - person working for an existing company
 - person without a company
 - d. Test via the API.
3. Add mappings in the Session class and create a SessionJpaRepository. Wire the repository in the AbisTrainingService. Implement the findSessionsForCourse(String shortTitle) method. Don't bother about cancelled sessions yet. Use JPQL. Add a JUnit test, and also test via the API to find all courses for DB2BAS.

12 ____ *More querying techniques and performance*

1.
 - a. Add a PersonDTO class, such that the API will only show a person's id, first name, last name, email, birthdate, company name and company town.
 - b. Provide a PersonForm class, which passes all person data (including company and address), but in a more flat structure.
 - c. Make sure that all (necessary) JSON/XML annotations are removed from the Person class, and now only appear in the PersonForm and PersonDTO classes.
 - d. Foresee a PersonMapper class that transforms a Person object to and from the Form/DTO objects. Call the mapper in the PersonApiController methods.
 - e. Test via the API.

2. Adjust exercise 10.3. as to return a SessionDTO object (defined as a record) from the TrainingController, which contains the session number, start date, instructor's first name and last name, kind, cancelled indicator and long course title. Do the mapping from inside the AbisTrainingService this time.
3. Make sure the query now only returns the non-cancelled sessions, by using a native query.
4.
 - a. Add mappings in the Enrolment class. Use a "composite" id for the session and enrolmentInSession columns.
 - b. Add a NamedQuery to find all (non-cancelled) enrolments for a person. Only fetch the e_pno, sdate and s_cid columns in the result. Create an EnrolmentJpaRepository that calls the named query.
 - c. Create an EnrolmentDTO record that mentions the person's first name and last name, his/her company name, the startdate of the session and the course title.
 - d. Implement the *EnrolmentDTO findEnrolments(int personId)* method in the AbisTrainingService and the Controller.
 - e. Add a JUnit test in the TrainingServiceTest class and test the API.

13 Exception Handling and Transactions

1. Let the PersonAlreadyExistsException (thrown when the person's emailaddress already exists in the DB) work again. Test via JUnit and Postman.
2. Create a method to delete a session by id. Throw a SessionCanNotBeDeletedException in case there are still enrolments for the session. Test via JUnit.
3. In the CourseJpaRepository, create a method to change the price of a course. Make sure to use a modifying, native query to do this, which only changes the caprice column. Add the necessary in the AbisCourseService class and test via JUnit.

4. Add a method in the SessionJpaRepository to cancel a session. Make sure to use a modifying, native query to do this, which only changes the scancel column. Also add the necessary in the AbisTrainingService class and test via JUnit. Make sure the test also works if it is not defined as @Transactional. Extra: add a corresponding method in the API and test.
5. Implement the method *enrolForSession(PersonForm person, int sessionId)* in the AbisTrainingService class. Make sure it is transactional. Throw the EnrolException if something goes wrong, and make sure everything is rolled back in that case. Add JUnit test cases in the TrainingServiceTest. Extra: Implement the method in the controller and test via Postman.

14 Spring Security

1. Although everybody can log in (findPerson(email,pwd)) and can call changePassword, a person should only be capable of changing his OWN password (after logging in). Implement this by using API keys:
 - create a ApiKeyStore which holds an API key for each user id
 - send this API key to the client (TrainingAPI) when the user logs in correctly.
 - when a user wants to change a password, he has to send his API key as well
 - service checks whether the API key is correct
2. Configure Spring Security. Create an in-memory user store, with 2 users:
 - abis01 which has role AbisUser
 - abis02 which has roles AbisUser and AbisAdmin
 Call the *findCourse(int id)* method via a browser and via Postman to check what happens. Authenticate yourself to solve the issue.
3. Everybody should be able to find courses, but only the users stated above should be able to perform the other actions. Use basic authentication to achieve this. Test in Postman, and also adjust the client to provide the authentication data when needed. Test via JUnit in the client.
4. Only the user with AbisAdmin role should be able to DELETE courses. Implement authorization to do this. Test via JUnit in the client.
5. Add CORS to the controller, such that it can be reached by Angular applications. These run typically on port 4200.

